

THEORY OF COMPUTATION

Handwritten Notes

VIVEKANAND VERNEKAR

GateWallah Parakram C 2024

(Hinglish Weekday)

Other Notes Uploaded Below

<https://t.me/pwgatenotes>

FINITE AUTOMATA

Basics of TOC

1. Symbol
2. Alphabet
3. String
4. Language

Symbol:

* It is anything.

* It is one length.

0

$|0| = 1$

gate

$|gate| = 4$

Symbol

	Symbol
1. English Language	letter a, b, ... z
2. Hindi Language	अ, आ, ... इ
3. Decimal Language	digit 0, 1, 2, ... 9
4. Binary Language	bit 0, 1
5. C language	char. 0...9, a, b...z, +, -, ...

Alphabet:

* It is a set.

↳ Collection of objects

↳ well defined → distinct, unordered.

Alphabets in TOC

$\Sigma \rightarrow$ Input alphabet: Set of i/p Symbols

$\Delta \rightarrow$ Output alphabet: Set of output Symbols

$\Gamma \rightarrow$ Stack alphabet: Set of Stack Symbols

$T \rightarrow$ Tape alphabet: Set of tape Symbols

eg: English Alphabet $\Sigma = \{a, b, \dots, z, A, B, \dots, Z\}$
 ↳ Symbol

String:

* It is a sequence of symbols

$$\Sigma = \{0, 1\}$$

- Zero length String: λ or ϵ (Empty String)
- One length string: 0, 1
- two length String: 00, 01, 10, 11
- three length strings: 000, 001, 010, 011, 100, 101, 110, 111

eg:: $\Sigma = \{0, 1\}$

How many k-length strings?

$$\rightarrow 2^k = |\Sigma|^k$$

$$\Sigma = \{a, b, c\}$$

How many 100 length strings?

$$\rightarrow 3^{100}$$

Symbols

↓
Alphabet↓
Strings↓
LanguagesSet: Collection of objects $\emptyset$

Empty Set.

 $\{ \}$

$$|\emptyset| = 0$$

Size of $\emptyset$.String: Sequence of symbols ϵ

Empty String

String

$$|\epsilon| = 0$$

Size of length of ϵ Language [Set] over Σ

* Set

* Set of Strings

* Collection of strings

$$\Sigma = \{0, 1\}$$

Language over $\Sigma =$

Language with zero strings: $L = \phi = \{\}$ ^{empty set} empty language

Languages with only 1 string: $L_1 = \{\epsilon\}$, $L_2 = \{1\}$, $L_3 = \{0\}$,
 $L_4 = \{00\}$...

Languages with only 2 strings: $L_1 = \{\epsilon, 0\}$, $L_2 = \{0, 1\}$, $L_3 = \{0, 00\}$...

given Σ .

- No. of strings over $\Sigma =$ Infinite
- No. of languages over $\Sigma =$ Infinite
- No. of k length strings $= |\Sigma|^k$
- No. of languages having 10 strings $=$ Infinite

Languages over $\Sigma = \{0, 1\}$

Finite Languages

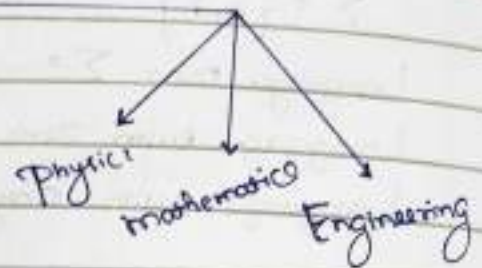
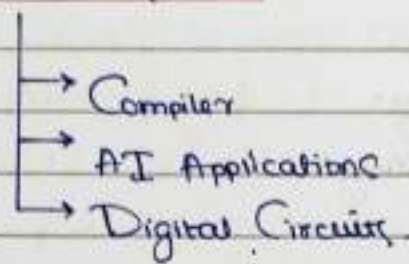
ϕ
 $\{0\}$
 $\{0, 10\}$
 $\{0^n, 1 \mid n \leq 10\}$
 $\Sigma = \{0, 1\}$

Infinite Languages

$\{0^n \mid n \geq 10\}$
 $\{0^n 1 \mid n \geq 1\}$
 form of a string

	English	Binary N. System.	C. Language
Symbols	a, b, ..., z	0, 1	Character
Alphabet	$\{a, b, \dots, z\}$	$\{0, 1\}$	Character Set
Strings	words	numbers	tokens
Language	Sentences	binary language	Programs

Applications of TOC :



Operations on String

(Unary)	(Binary)	
① Reversal $w = abc$ $w^R = cba$ $ w = w^R $	② Concatenation $w_1 = ab$ $w_2 = bc$ $w_1.w_2 = abbc$ $w_2.w_1 = bcab$ $ w_1.w_2 = w_2.w_1 $	③ Prefix $\rightarrow$ start is fixed $w = abcd$ Prefixes of $w =$ ϵ a ab abc $abcd$

④ Suffix $w = abcd$ Suffixes of w : ϵ d cd bcd $abcd$	⑤ SubString $w = abcd$ Substrings of w are: ϵ $\{ \begin{matrix} a & ab \\ b & ab \\ c & abc \\ d & abcd \end{matrix} \}$ $\{ \begin{matrix} ab & abc \\ bc & abcd \\ cd & \end{matrix} \}$ $\{ \begin{matrix} abc & abcd \\ bcd & \end{matrix} \}$ $\{ abcd \}$
---	--

Prefix(w)	Suffix(w)	SubString(w)
$= \{ v \mid uv = w \}$ Beginning sequence of w .	$= \{ v \mid uv = w \}$ Ending sequence of w .	$= \{ y \mid x_1 y x_2 = w \}$ $w = abcd$ $\epsilon, \epsilon, abcd = w$ $\epsilon, a, bcd = w$ $\vdots$

* $w = aaaa$

Prefixed of $w = \epsilon, a, aa, aaa, aaaa - 5$

Suffixes of $w = \epsilon, a, aa, aaa, aaaa - 5$

Substrings of $w = \epsilon, a, aa, aaa, aaaa - 5$

* $w = abcd$

Prefixed of $w = \epsilon, a, ab, abc, abcd - 5$

Suffixes of $w = \epsilon, d, cd, bcd, abcd - 5$

Substrings of $w = \epsilon, a, b, c, d, ab, bc, cd, abc, bcd, abcd - 10$

How many prefixes for n -length string?

→ $n+1$

How many suffixes for n -length string?

→ $n+1$

How many substrings for n -length string?

→ $n+1 \leq \text{No. of Substrings} \leq \frac{n(n+1)}{2} + 1$

↓
If all symbols are same

↓
If all symbols in the string are distinct

How many different length prefixes for n -length string?

→ $n+1$

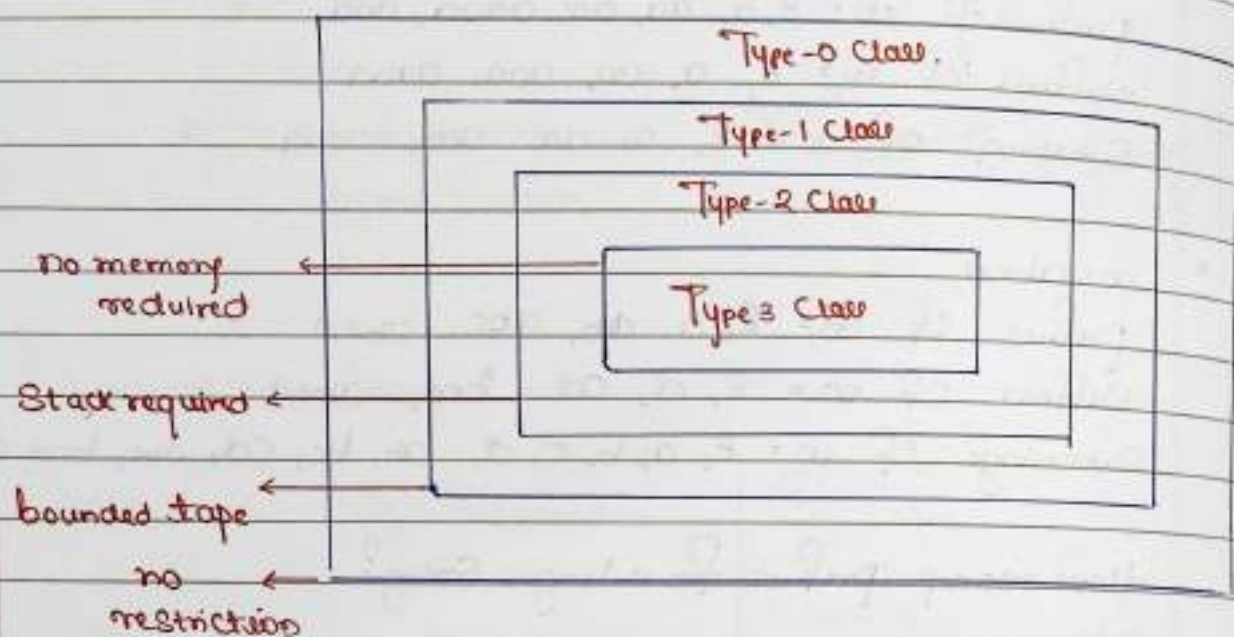
How many different length suffixes for n -length string?

→ $n+1$

How many different length substrings for n -length string?

→ $n+1$

Chomsky Hierarchy: $T_3 < T_2 < T_1 < T_0$



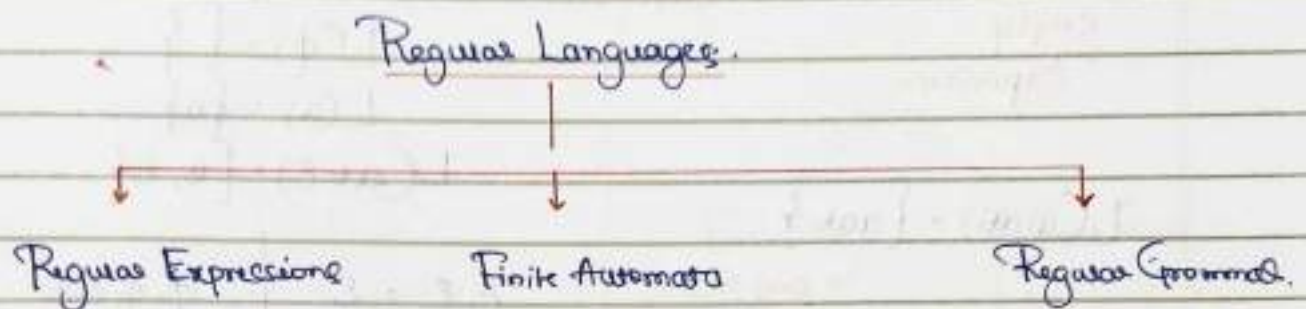
Type 3 class	Type 2 class	Type-1 class	Type 0 class
↓	↓	↓	↓
Set of all regular languages.	Set of all context-free languages	Set of all context-sensitive languages	Set of all recursively enumerable languages.
Finite Automata (FA).	Push Down Automata (PDA)	Linear bound Automata (LBA)	Turing Machine (TM)



Language
 ↳ Set of strings

Automata
 (Machine)
 ↓
 Set of states & transitions

Grammar
 ↳ Set of rules



Regular Expression :

- * It represents a regular language.
- * It uses four operators.

Binary {

- I. OR (+)
- II. Concatenation (.)
- III. Kleene Star [Kleene Closure] (*)
- IV. Kleene Plus [positive closure] (+)

OR	Concatenation	Kleene Star	Kleene Plus
+, U, I	.	*	+
Binary	Binary	Unary	Unary
$R_1 + R_2$	$R_1 . R_2$	R^*	R^+
Either R_1 or R_2	R_1 followed by R_2	Zero or more occurrence of R	One or more occurrences of R
$a + b$	$a . b$	$a^* = a^{20}$	$a^+ = a^{21}$
$\{a, b\}$	$\{ab\}$	$\{a^0, a^1, a^2, \dots\}$	$\{a, aa, aaa, \dots\}$
$a^*, b^*, a^+ b^+$	aa, ba, aab, aba	$\{\epsilon, a, aa, aaa, \dots\}$	

$$L(a + \epsilon) = \{\epsilon, a\} = \{a, \epsilon\}$$

$$L(a + ab) = \{a, ab\} = \{ab, a\}$$

$$L(a + b) = \{a, b\}$$

$$L(\phi) = \{\} = \phi$$

↓ empty expression

$$L(\epsilon) = \{\epsilon\}$$

↓ non empty expression

↓ non empty set

$$L(\phi) = \{\}$$

$$L(a) = \{a\}$$

$$L(a + \epsilon) = \{a, \epsilon\}$$

$$L(a \cdot ab) = \{aab\}$$

↪ one string

$$L(a \cdot \epsilon) = \{a\}$$

$$\epsilon \cdot \epsilon = \epsilon$$

$$a \cdot \epsilon = a$$

$$\epsilon \cdot a = a$$

$$\phi \cdot \phi = \phi$$

$$a \cdot \phi = \phi$$

OR:

$$1. \epsilon + \epsilon = \epsilon$$

$$2. \phi + \phi = \phi$$

$$3. a + a = a$$

$$4. \epsilon + a = \epsilon \cdot a = a + \epsilon$$

$$5. a + \epsilon = a$$

$$6. \epsilon + \phi = \epsilon$$

$$7. \phi + \epsilon = \epsilon$$

$$8. a + \phi = a$$

$$9. \phi + a = a$$

Regular Expression $\cong$ Set

$$\epsilon \cong \{\epsilon\}$$

$$\phi \cong \{\} = \phi$$

$$a \cong \{a\}$$

$$R_1 + R_2 \cong L(R_1) \cup L(R_2)$$

$$R_1 \cdot R_2 \cong L(R_1) \cdot L(R_2)$$

eg: $a + \epsilon = a + \epsilon$

either a or ϵ

$$a \cdot \epsilon = a$$

$$|a| = 1$$

$$|\epsilon| = 0$$

$$L(a + \epsilon) = \{a, \epsilon\}$$

$$a + \epsilon \neq a$$

Concatenation.

$$1. \epsilon \cdot \epsilon = \epsilon$$

$$2. \phi \cdot \phi = \phi$$

$$3. a \cdot a = aa = a^2$$

$$4. \epsilon \cdot \phi = \phi$$

5. $\phi \cdot \epsilon = \phi$

6. $a \cdot \epsilon = a$

7. $\epsilon \cdot a = a$

8. $\phi \cdot a = \phi$

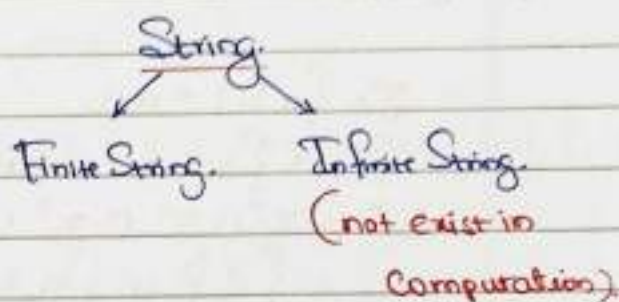
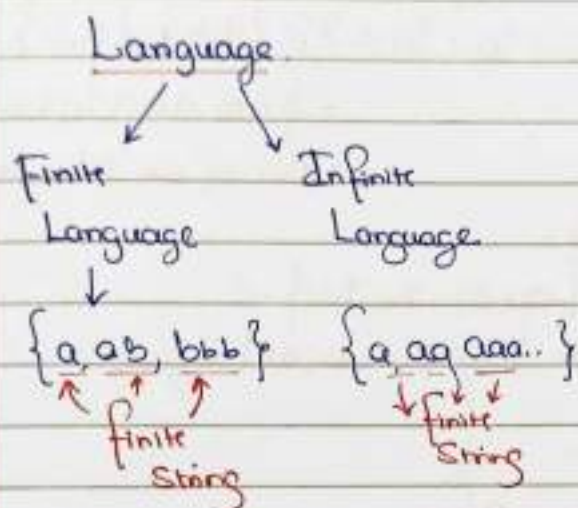
9. $a \cdot \phi = \phi$

$\phi \cdot R = \phi$

$R \cdot \phi = \phi$

$R \cdot \epsilon = R$

$\epsilon \cdot R = R$



$$(a+ab) (\epsilon+aaa+bb) = \{a, ab, abb, a^4, abaaa\}$$

$$a \cdot \epsilon \rightarrow a$$

$$a \cdot aaa \rightarrow aaaa = a^4$$

$$a \cdot b \rightarrow ab$$

$$ab \cdot \epsilon \rightarrow ab$$

$$ab \cdot aaa \rightarrow abaaa$$

$$ab \cdot b \rightarrow abb$$

Kleene Star:

$$R^*$$

$$R^0 + R^1 + R^2 + \dots$$

$$a^* = \{\epsilon, a, aa, \dots\}$$

$$L(a^*) = \{\epsilon, a, a^2, a^3, \dots\}$$

$$= \{a^n \mid n \geq 0\}$$

Language set

$$1. \quad b^* = \{\epsilon, b, bb, bbb, \dots\} = \{b^n \mid n \geq 0\}$$

2. $(ab)^+ = \{ \epsilon, ab, abab, ababab, \dots \} = \{ (ab)^n \mid n \geq 0 \}$
3. $(ba)^+ = \{ \epsilon, ba, baba, bababa, \dots \} = \{ (ba)^n \mid n \geq 0 \}$
4. $(aa)^+ = \{ \epsilon, a^2, a^4, a^6, \dots \} = \{ a^{2n} \mid n \geq 0 \}$
5. $(a+b)^+ = \{ \epsilon, a, b, aa, ab, ba, \dots \} = \{ a, b \}^+$
6. $(a+\epsilon)^+ = \{ \epsilon, a, a^2, a^3, \dots \} = \{ a^n \mid n \geq 0 \} = a^+$
7. $(b+\epsilon)^+ = b^+$

$$\Sigma = \{a, b\} \cdot \{a, b\}$$

$$= \{aa, ab, ba, bb\}$$

= Set of all 2 length strings

$$\Sigma = \{a, b\} = a+b$$

$$\Sigma^+ = \{a, b\}^+ = (a+b)^+$$

$$\Sigma^+ = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

$$= \{ \epsilon \} \cup \{a, b\} \cup \{aa, ab, ba, bb\} \dots$$

Set of all strings over Σ = Universal language

$a+b$

a	✓
b	✓
ab	×
ba	×
bb, aa	×

$(a+b)^+$

$$(a+b)^0 = \epsilon$$

$$(a+b)^1 = a, b$$

$$(a+b)^2 = (a+b)(a+b) = aa, ab, ba, bb$$

$(a+\epsilon)^+$

$$(a+\epsilon)^0 = \epsilon$$

$$(a+\epsilon)^1 = a, \epsilon$$

$$(a+\epsilon)^2 = (a+\epsilon)(a+\epsilon) \rightarrow a^2, a, \epsilon$$

$$(a+\epsilon)^3 = a^3, a^2, a, \epsilon$$

$$\epsilon, a, a^2, a^3, \dots, a^+$$

$$(a+b+\epsilon)^+ = (a+b)^+$$

Kleene Plus.

R^+

$$R^1 + R^2 + R^3 + \dots$$

$$a^+ = \{a, a^2, a^3, \dots\} = \{a^n \mid n \geq 1\}$$

$$b^+ = \{b, b^2, b^3, \dots\} = \{b^n \mid n \geq 1\}$$

$$1. (a+\epsilon)^+ = a^+$$

$$2. (b+\epsilon)^+ = b^+$$

$$3. (a+b+\epsilon)^+ = (a+b)^+$$

$$4. a^+ + \epsilon = a^+ + a^0 = a^+$$

$$5. a^+ + \epsilon = a^+$$

$$\boxed{R^+ = R^+ + \epsilon^0}$$

$$(a+\epsilon)^+ \xrightarrow{a} \epsilon$$

$$(a+\epsilon)^2 = (a+\epsilon)(a+\epsilon) = a, \epsilon, a^2$$

$$\vdots$$

$$\epsilon, a, a^2, a^3, \dots, a^+$$

	OR	Concatenation.
1. Identity	ϕ (Empty expression)	ϵ
2. Associative	$\checkmark$ (holds)	$\checkmark$ (holds)
3. Commutative	$\checkmark$ (holds)	$\times$ (does not hold)
4. Dominator. (Annihilator)	Σ^+	ϕ

Identity: $R + I = I + R = R$

$$R + \phi = \phi + R = R$$

$I = \phi$
for +

ϕ is identity for "OR" operation

$I = \epsilon$
for .

$$R \cdot I = I \cdot R = R$$

$$R \cdot \epsilon = \epsilon \cdot R = R$$

ϵ is identity for concatenation.

Associative: $(R_1 + R_2) + R_3 = R_1 + (R_2 + R_3)$

(left associative)

(Right associative)

$$(a+b)c = a+(b+c)$$

$\hookrightarrow \{a, b, c\}$

$$(R_1 \cdot R_2) \cdot R_3 = R_1 (R_2 \cdot R_3)$$

$$(a \cdot b) \cdot c = a \cdot (b \cdot c)$$

$$\rightarrow \{a, b, c\}$$

Commutative:

$$R_1 + R_2 = R_2 + R_1 \quad \checkmark \text{ holds}$$

$$R_1 \cdot R_2 \neq R_2 \cdot R_1$$

(need not be)

Does not hold.

Dominator:

$$R + D = D + R = D \quad \& \Sigma^+ \text{ is dominator for } +$$

$$R + \Sigma^+ = \Sigma^+ + R = \Sigma^+$$

$$R \cdot D = D \cdot R = D$$

ϕ is dominator for $\cdot$.

$$R \cdot \phi = \phi \cdot R = \phi$$

Distributive:

(I) OR over Concatenation

(i) Left distribution

$$a + (b \cdot c) \quad ? \quad (a + b) \cdot (a + c)$$

$$\{a, bc\} \neq \{a, ac, ab, bc\}$$

(ii) Right distribution

$$(a \cdot b) + c \quad ? \quad (a + c) \cdot (b + c)$$

$$\{ab, c\} \neq \{ab, ac, cb, cc\}$$

Does not hold.

OR should not

be distributed

over Concatenation.

(II) Concatenation over OR.

(i) Left distribution:

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$

$$\{ab, ac\} = \{ab, ac\}$$

(ii) Right distribution

$$(a + b) \cdot c = (a \cdot c) + (b \cdot c)$$

$$\{ac, bc\} = \{ac, bc\}$$

It holds.

Concatenation is

distributed over OR.

Simplification:

1. $\phi^+ = \epsilon$

2. $\epsilon^+ = \epsilon$

3. $\phi^+ = \phi$

4. $\epsilon^+ = \epsilon$

5. $(a+\phi)^+ = a^+$

6. $(a+\phi)^+ = a^+$

7. $(\epsilon+\phi)^+ = \epsilon^+ = \epsilon$

8. $(\epsilon+\phi)^+ = \epsilon^+ = \epsilon$

9. $\phi^+ \cdot \phi^+ = \epsilon \cdot \phi = \phi$

10. $\epsilon^+ \cdot \epsilon^+ = \epsilon \cdot \epsilon = \epsilon$

11. $\phi^+ \cdot \epsilon^+ = \epsilon \cdot \epsilon = \epsilon$

12. $\phi^+ \cdot \epsilon^+ = \phi \cdot \epsilon = \phi$

$$R^+ = R' + R' \dots$$

$$\phi^+ = \phi^0 + \phi^1 + \phi^2 \dots$$

$$= \epsilon + \phi = \underline{\underline{\epsilon}}$$

$$^+ (any)^0 = \epsilon$$

$$^+ Any \cdot \phi = \phi$$

$$^+ \epsilon^+ = \epsilon$$

13. $a^+ \cdot \phi^+ = a^+ \cdot \epsilon = a^+$

14. $a^+ \cdot \phi^+ = \phi$

15. $a^+ \cdot a = a^+$

$$\uparrow \downarrow \quad (a^+ = \epsilon, a, aa \dots \cdot a)$$

16. $= a^1, a^2, a^3 \dots = a^+$

$$a \cdot a^+ = a^+$$

17. $a^+ + a^+ = a^+$

18. $a^+ + a^+ = a^+$

19. $a^+ + a^+ = a^+$

20. $a + a^+ = a + (a + a^+ + \dots) = a^+$

21. $a + a^+ = a^+$

22. $a^{100} + a^+ = a^+$

23. $a^+ + a^+ + a + a^{1000} = a^+ \cdot a^+ \cdot a = (a \cdot a)^+$

24. $(a^+)^2 = a^+ \cdot a^+ = a^+ \cdot a \cdot a = a \cdot a^+$

25. $(a^+)^2 = a^+ \cdot a^+ = a^+$

26. $(a^+)^{100} = a^+$

27. $a^+ a^+ a^+ a^+ = a^+$

28. $(a^+)^+ = (a^+)^0 \cup (a^+)^1 \cup \dots = a^+$

29. $(a^+)^+ = (a^+)^0 \cup (a^+)^2 \dots = a^+$

30. $(a^+)^+ = (a^+)^0 \cup (a^+)^1 \dots = \epsilon + a^+ = a^+$

31. $(a^+)^+ = (a^+)^1 \cup (a^+)^2 \dots = a^+$

Note:

(I) $R + \phi = R$

(II) $R \cdot \phi = \phi$

(III) $R \cdot \epsilon = R$

(IV) $R^0 = \epsilon$

(V) $R + R^+ = R^+$

(VI) $a \cdot a^+ = a^+$

(VII) $a^+ \cdot a = a^+$

(VIII) $(a + \epsilon)^+ = a^+$

(IX) $(a + \epsilon)^+ = a^+$

(X) $a^+ + \epsilon = a^+$

Homework: Find the min String in the following expressions:

1. $(ab)^+ a a a (b + ab) \rightarrow \min = ab a a a b$

2. $(ab)^+ a a a (b + \epsilon) \rightarrow a a a$

3. $(a + ab)^+ (bb + a a a)^+ (ab)^+ \rightarrow b b$

4. $(ab)^+ + \epsilon \rightarrow ab$

5. $(ab) (atb)^+ a a a + b a \rightarrow b a$

6. $[(ab^+ + a^+ + b a) ab]^+ (atb)^+ \rightarrow a a b$

7. $[(ab)^+ a)^+ a a a]^+ \rightarrow a a a$

8. $[(ab a)^+ a b a)^+ ab]^+ \rightarrow \epsilon$

9. $(a^+)^+ \rightarrow a$

10. $(ab)^+ a^+)^+ b b^+ a a^+ \rightarrow ab b a$

11. $[(ab)^+ a b a]^+ a a a \rightarrow a b a b a a a$

12. $(a a a^+ a b a^+)^+ \rightarrow a a a b$

13. $(a + a a + a a a)^+ \rightarrow a$

14. $a^+ b^+ \rightarrow ab$

15. $a^+ b^+ \rightarrow \epsilon$

16. $a^+ b^+ c^+ d^+ \rightarrow c$

17. $(a + b^+ + c^+) \rightarrow a$

18. $(a + b + \epsilon)^+ \rightarrow \epsilon$

19. $[(a + ab) a a + ab]^+ a a a \rightarrow a b a a a$

20. $(ab a a a a + a^+)^+ \rightarrow \epsilon$

Priority: 1. Kluene plus, Kluene star

2. Concatenation

3. OR.

Q1. How to write Regular expressions?

$\rightarrow$ Regular Language $\equiv$ Regular Expression.

1. $L = \{ \} \text{ over } \Sigma = \{a\}$

$R = \phi$.

2. $L = \{ \} \text{ over } \Sigma = \{a, b\}$

$R = \phi$.

Note: To represent one regular language, there are infinite regular expressions.

3. $L = \{\epsilon\}$ over $\Sigma = \{a\}$
 $R = \epsilon = \phi = \phi^+ a^0 \dots$

$$a^+ = (a^+)^0 = (a^+)^1 = (a^+)^2$$

$$= a^+ + \epsilon$$

4. $L = \{\epsilon\}$ over $\Sigma = \{a, b\}$
 $R = \epsilon = \phi^0 \cdot a^0 = b^0 \dots$ (and many more).

$$= (a + \epsilon)^+$$

$$= (a + \epsilon)^+$$

$$= \phi^+ \cdot a^+$$

5. $L = \text{Set of all strings over } \Sigma = \{a\}$
 $R = a^+ = \Sigma^+$

$$= a^+ + \text{any other } \Sigma \text{ exp.}$$

6. $L = \text{Set of all strings over } \Sigma = \{a, b\}$
 $R = (a + b)^+ = \Sigma^+$

$$ab^+ \neq b^+a^+$$

$$(a^+b^+)^+ = (b^+a^+)^+$$

7. $L = \{w \mid w \in \{a, b\}^+, w \text{ starts with } a\}$
 $= \{aw \mid w \in \{a, b\}^+\}$
 $R = a(a + b)^+ = a\Sigma^+$

eg: $A = \{x \mid x \in \mathbb{N}, x \geq 2\}$

Cond. 1 Condition 2.

Every natural number greater than 2 = $\{3, 4, 5, \dots\}$

8. Set of all strings starting with ab over $\Sigma = \{a, b\}$
 $R = ab(a + b)^+$

9. $L = \{w \mid w \in \{a, b\}^+, w \text{ starts with } aaa\}$
 $R = aaa(a + b)^+$

10. $L = \{w \mid w \in \{a, b\}^+, w \text{ ends with } a\}$
 $R = (a + b)^+ a$

11.

Group I.	Group II.
1. $a(ab)^+$	I. $(ab^+)^+ = \{a, aa, ab, \dots\}$
2. $b(ab)^+$	II. $(b^+a)^+ = (a+b)^+a$
3. $(ab)^+a$	III. $(ba^+)^+ = b(ab)^+a$
4. $(ab)^+b$	IV. $(a^+b)^+ = (a+b)^+b$

12. $L = \{w, a, w_2 \mid w_1, w_2 \in \{a, b\}^+\}$
 = Set of all strings containing 'a' as substring.
 = $\{a, aa, ab, \dots\}$
 $\{(ab)^+a(ab)^+, b^+a(ab)^+\}$ Same.

13. $L = \{w \mid w \in \{a, b\}^+, |w| = 2\}$
 exactly 2 length strings.

$$P = ab + ba + aa + bb$$

$$= (ab)(ab)$$

$$= (ab)^2 = \Sigma^2$$

14. $\{w \mid w \in \{a, b\}^+, |w| \leq 2\}$
 Almost 2 length strings.

$$P = \epsilon + a + b + aa + ab + ba + bb$$

$$= \Sigma^0 + \Sigma^1 + \Sigma^2 = (\epsilon + a + b)^2$$

$$|w| = k$$

$$= (a + b + \epsilon)^k$$

String	Length
ϵ	0
a, b	1
aa, ab, ba, bb	0 or 1
$(a + b + \epsilon)^2$	≤ 2 length

$$15. \{w \mid w \in \{a, b\}^+, |w| \geq 2\}$$

Atleast 2 length

$$\begin{aligned} R &= \underbrace{(a+b)^2}_{\text{exactly 2 length}} \underbrace{(a+b)^+}_{\text{any length}} \\ &= (a+b)^+ (a+b)^2 \\ &= (a+b)(a+b)(a+b)^+ \\ &= (a+b)(a+b)^+ \\ &= (a+b)^+ (a+b) \end{aligned}$$

$$16. \{w \mid w \in \{a, b\}^+, n_a(w) = 2\}$$

$$R = b^+ a b^+ a b^+$$

$$= (b^+ a)^2 b^+$$

$$= b^+ (a b^+)^2$$

$$n_a(w)$$

$$\rightarrow \#a(w)$$

$$\rightarrow \text{no. of } a^i \text{ in } (w)$$

$$17. \{w \mid w \in \{a, b\}^+, n_a(w) \leq 2\}$$

$$R = b^+ + b^+ a b^+ + b^+ a b^+ a b^+$$

$$= b^+ (\epsilon + a) b^+ (\epsilon + a) b^+$$

$$\text{Zero } a^i = b^+$$

$$\text{one } a = b^+ a b^+$$

$$2 \text{ } a^i = b^+ a b^+ a b^+$$

$$18. \{w \mid w \in \{a, b\}^+, n_a(w) \geq 2\}$$

$$R = (a+b)^+ a (a+b)^+ a (a+b)^+$$

$$19. \{w \mid w \in \{a, b\}^+, |w| \text{ is even}\}$$

$$R = [(a+b)^2]^+$$

$$20. \{w \mid w \in \{a, b\}^+, |w| \text{ is odd}\}$$

$$R = [(a+b)^2]^+ (a+b)$$

$$= (a+b) [(a+b)^2]^+$$

even

odd

odd

$$21. L = \{xay \mid x, y \in \{a, b\}^+, |x| = 3\}$$

$$= 4^{\text{th}} \text{ symbol from beginning is } a$$

$$= (a+b)^3 a (a+b)^+$$

$$\underbrace{\quad}_{3 \text{ length}} \quad \underbrace{\quad}_{4^{\text{th}} \text{ symbol}}$$

22. $\{ xay \mid x, y \in \{a, b\}^*, |y| = 2 \}$
 any 2 length.
 $= (a+b)^* a (a+b)^*$ } 3rd symbol from end is a.

23. $\{ a^m b^n \mid m, n \geq 0 \}$
 $R = a^* b^*$

24. $\{ a^m b^n \mid m \geq 0, n \geq 1 \}$
 $R = a^* b^+$

25. $\{ a^m b^n \mid m \geq 1, n \geq 1 \}$
 $R = a^+ b^+$

26. $\{ a^m b^n \mid m = \text{even}, n \geq 0 \} = R = (aa)^* b^*$

27. $\{ a^m b^n \mid m = \text{odd}, n \geq 1 \} = R = a(aa)^* b^+$

28. $\{ a^m b^n \mid m \geq 2, n = \text{even} \} = R = aaa^+(bb)^*$

29. $\{ a^m b^n \mid m \geq 2, n \geq 3 \} = R = aaa^+ bbbb^+$

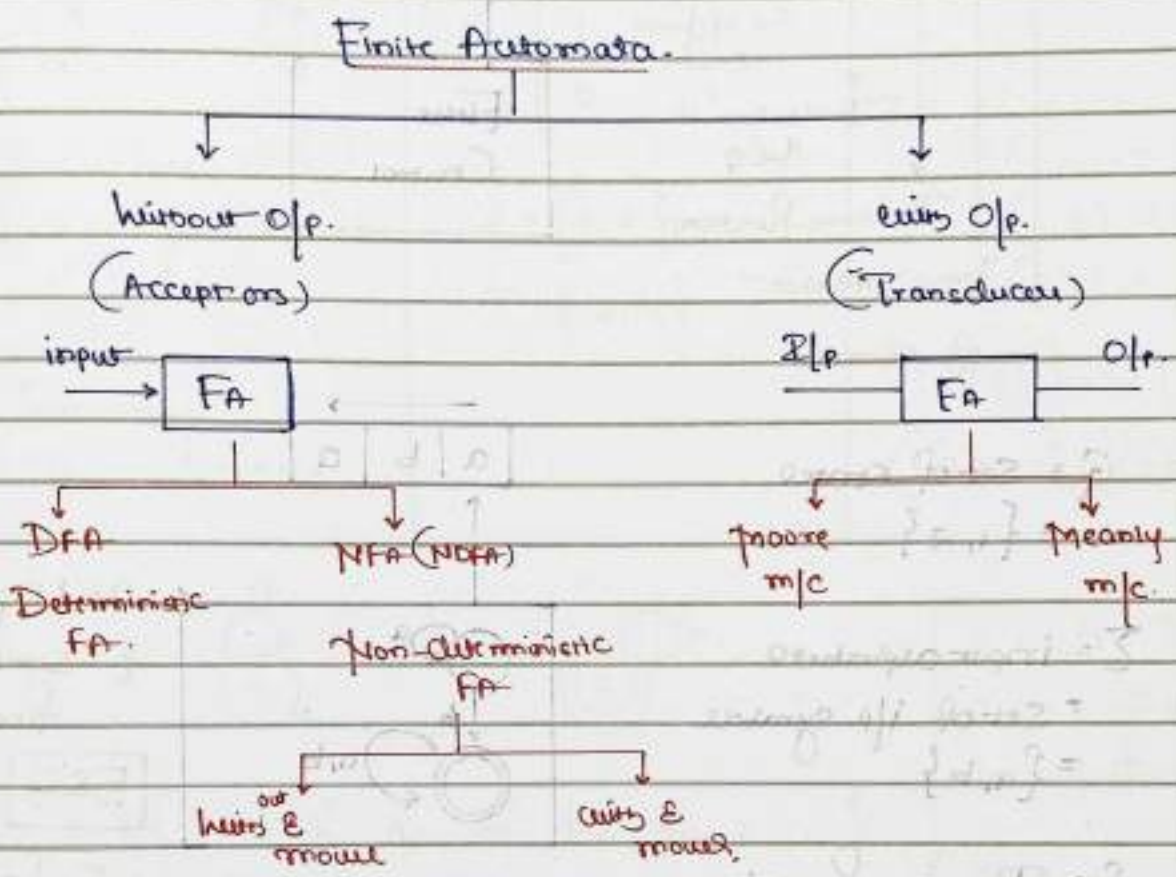
30. $\{ a^m b^n \mid m \leq 1, n \geq 2 \} = R = \underbrace{(\epsilon + a)}_{\text{max } 1a} + \underbrace{bb^+}_{\text{min } 2b}$

31. $\{ w \mid w \in \{a, b\}^*, n_a(w) = \text{odd} \}$
 $= b^*(b^+ a b^+ a b^+)^* b^+ a b^+$
 $= b^+ a b^+(b^+ a b^+ a b^+)^* b^+ a b^+$
 $= b^+ a b^+(b a b a b)^* b^+$

two
find out
other
equivalent
expressions.

32. $\{ w \mid w \in \{a, b\}^*, n_a(w) = \text{even} \}$
 $= b^*(b^+ a b^+ a b^+)^* b^+$
 $= b^+(a b a b^+)^* b^+ = (b^+ a b^+ a)^* b^+$

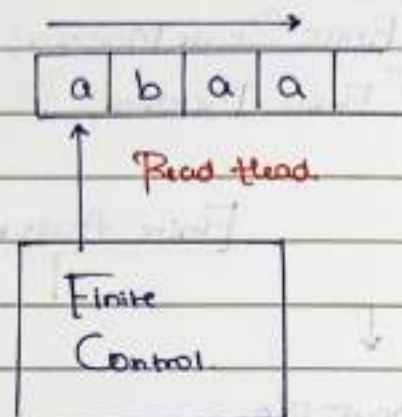
Finite Automata : It represents a regular language
(Finite State Machine) (accepts)
(Finite Machine) (recognises).



FA Configuration

$$FA = (Q, \Sigma, \delta, q_w, F)$$

$\downarrow$ Set of final states
 $\downarrow$ Initial state of $q_w \in Q$
 $\downarrow$ Transition function
 $\downarrow$ Input alphabet
 $\downarrow$ Set of states

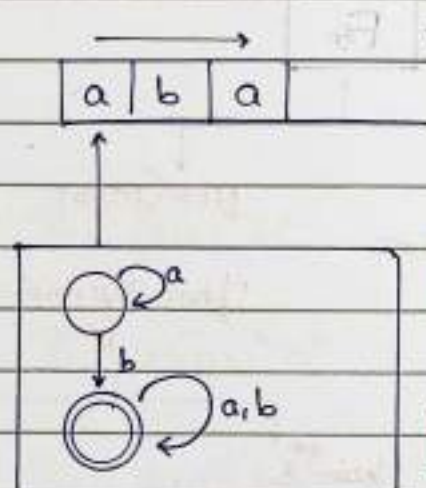


$Q =$ Set of states
 $= \{1, 2\}$

$\Sigma =$ input alphabets
 $=$ Set of i/p symbols
 $= \{a, b\}$

$\delta =$ Transition function

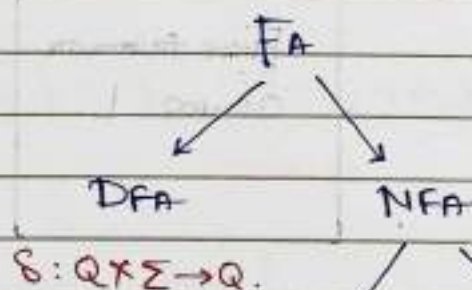
$q_w =$ Initial state
 $= 1$



$0 \leq |F| \leq |Q|$
 $F =$ set of final states

$$F \subseteq Q$$

$F = \{2\}$



FA Representations

1. State Diagram.
2. Transition Table.
3. Set/Function/Relation.

without ϵ move.

$$\delta: Q \times \Sigma \rightarrow 2^Q$$

with ϵ move.

$$\delta: Q \times \Sigma \cup \{\epsilon\} \rightarrow 2^Q$$

Diagram	Table	Set.									
Initial State: Final State / Non Final State: / Transition: $P_s \xrightarrow{i} N_s$	Σ δ $Q = \{1, 2\}$ $\Sigma = \{a, b\}$ <table border="1"> <tr> <th></th><th>a</th><th>b</th></tr> <tr> <th>1</th><td>1</td><td>2</td></tr> <tr> <th>2</th><td>2</td><td>1</td></tr> </table>		a	b	1	1	2	2	2	1	$E = (\{1, 2\}, \{a, b\}, \delta, 1, \{1\})$ δ $\delta(1, a) = 1$ $\delta(1, b) = 2$ $\delta(2, a) = 2$ $\delta(2, b) = 1$ $\delta = \{((1, a), 1), ((1, b), 2), ((2, a), 2), ((2, b), 1)\}$
	a	b									
1	1	2									
2	2	1									

$\Sigma^* = \{ \epsilon, a, b, aa, ab, ba, bb, \dots \}$
 $L = ? = (a+ b)^* b$

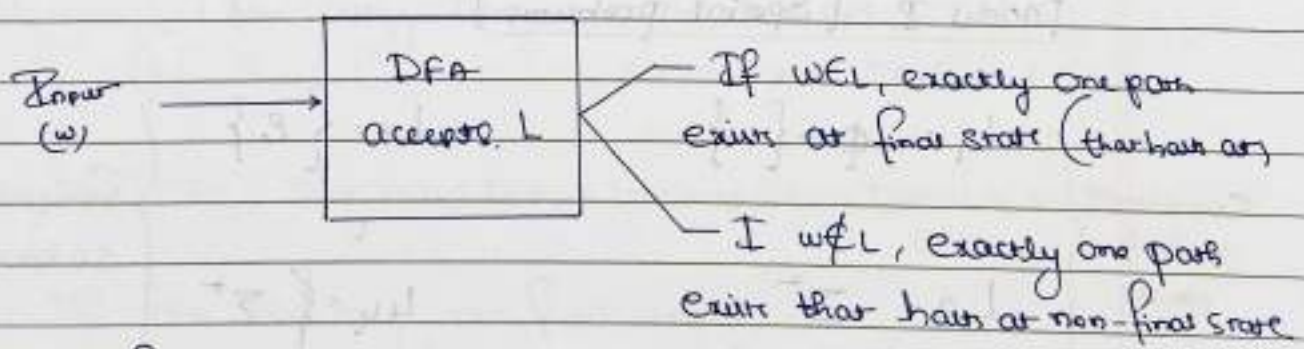
ϵ : 1. (not accepted)

a : $1 \xrightarrow{a} 1$

b : $1 \xrightarrow{b} 2$ (final, accepted)

aa : $1 \xrightarrow{a} 1 \xrightarrow{a} 1$

→ What is DFA?

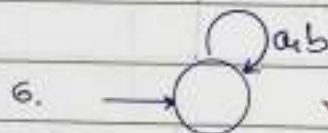
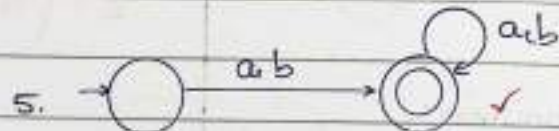
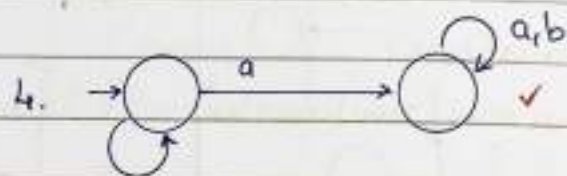
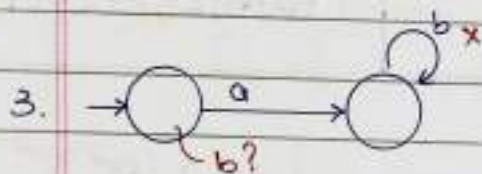
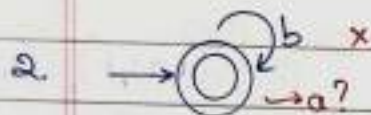
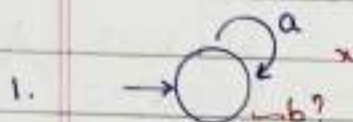


DFA Definitions

- I $\delta: Q \times \Sigma \rightarrow Q$
- II From every state, for every i/p symbol, exactly one transition to next state.

III. $\forall w \in \Sigma^+ \rightarrow$ exactly one path exists.
 [valid string has at final
 invalid string has at nonf.]

Identify Correct DFA's $\Sigma = \{a, b\}$



Note: $|Q| = n$ $|\Sigma| = k$

How many transitions in DFA?

$$= nk$$

$$= |Q \times \Sigma| = |Q| \times |\Sigma|$$

Modu 2 [Special problems]

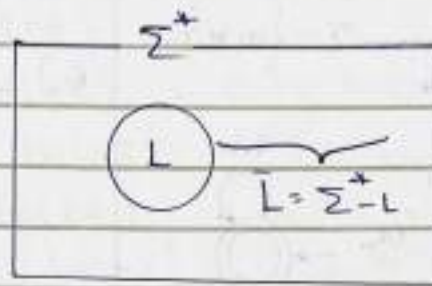
Complement to each other.	{	$L_1 = \phi = \{ \}$	$L_3 = \{ \epsilon \}$	}	Complement to each other.
		$L_2 = \Sigma^+$ Universal Set.	$L_4 = \{ \Sigma^+ \}$		

$$\rightarrow \bar{L} = \Sigma^+ - L$$

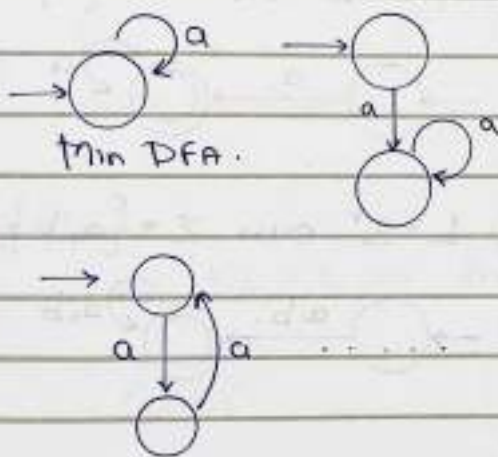
= Universal set - L

$$L \cup \bar{L} = \Sigma^+$$

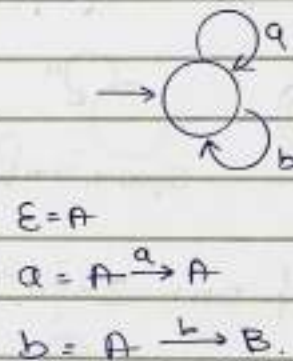
$$L \cap \bar{L} = \phi$$



① $L = \{ \} \text{ over } \Sigma = \{a\}$



② $L = \{ \} \text{ over } \Sigma = \{a, b\}$



Note: (I) For every language, no. of DFAs = Infinite
 (II) " " " " no. of min. DFAs = 1.

$\rightarrow$ Min. DFA = Minimized DFA = minimal DFA = DFA with minimum no. of states

(III) IF DFA has only non final states
 then $L(\text{DFA})$ is $\phi = \{ \}$

(IV) IF DFA has only final states then $L(\text{DFA})$ is Σ^+

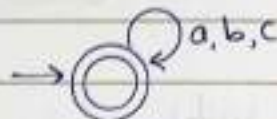
3. $L = \Sigma^+$ over $\Sigma = \{a\}$

$= a^+$



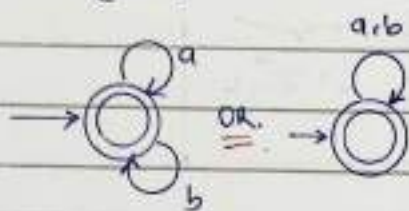
5. $L = \Sigma^+$ over $\Sigma = \{a, b, c\}$

$= (a+b+c)^+$



4. $L = \Sigma^+$ over $\Sigma = \{a, b\}$

$= (a+b)^+$

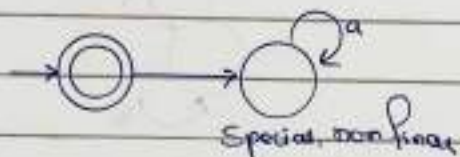


6. $L = \Sigma^+$ over $\Sigma = \{*, @\}$

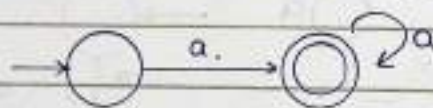
$= (*+@)^+$



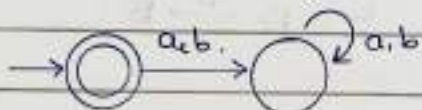
7. $L = \{\epsilon\}$ over $\Sigma = \{a\}$



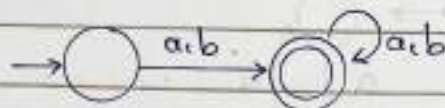
9. $L = \Sigma^+$ over $\Sigma = \{a\}$



8. $L = \{\epsilon\}$ over $\Sigma = \{a, b\}$

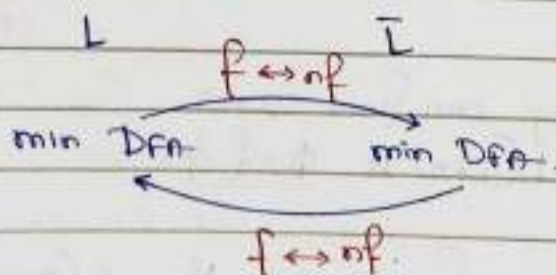


10. $L = \Sigma^+$ over $\Sigma = \{a, b\}$



Note:

If L has min DFA with n states, then $\bar{L}$ has min states DFA with n states.



Module III [Length based problems]

1. $L_1 = \{w \mid w \in \{a, b\}^+, |w| = 2\} = (a+b)^2$
2. $L_2 = \{w \mid w \in \{a, b\}^+, |w| = 3\} = (a+b)^3$
3. $L_3 = \{w \mid w \in \{a\}^+, |w| = 2\} = aa$
4. $L_4 = \{w \mid w \in \{a\}^+, |w| = 3\} = aaa$
5. $L_5 = \{w \mid w \in \{a\}^+, |w| \leq 2\} = \epsilon + a + a = (\epsilon + a)^2$
6. $L_6 = \{w \mid w \in (a+b)^+, |w| \leq 2\} = (\epsilon + a + b)^2$
7. $L_7 = \{w \mid w \in \{a\}^+, |w| \geq 2\} = aaa^+ = aa^+ = a^+a = a^+aa$
8. $L_8 = \{w \mid w \in (a+b)^+, |w| \geq 2\} = (a+b)^2(a+b)^+$

→ $L_1 = \{w \mid w \in \{a, b\}^+, |w| = 2\} = (a+b)^2$

= set of all 2 length strings

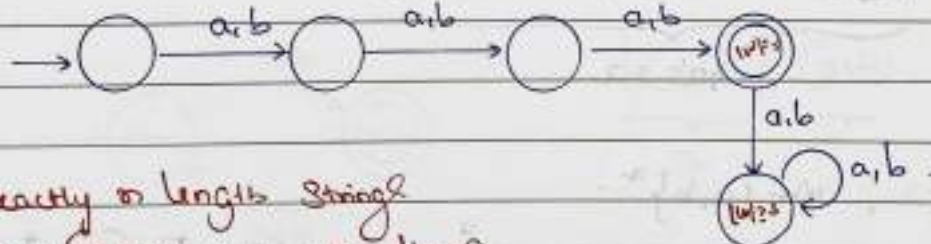
= $\{aa, ab, ba, bb\}$

= $aa + ab + ba + bb = (a+b)(a+b)$
 $= (a+b)^2$

Note:

$|w| = k \Rightarrow k+2$ states in DFA

→ $L = (a+b)^3$



Exactly n length strings
 $(a+b)(a+b) \dots n$ times

↓
 $n+2$ states in DFA.

Exactly n length } Dead state is required $\Rightarrow (n+1) + 1$ dead state.
 At most n length }

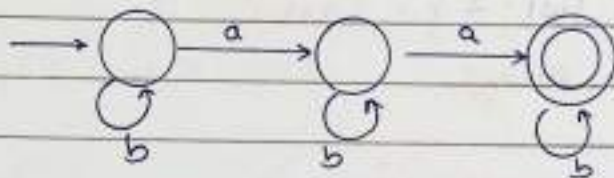
At least n length } dead state not required $\Rightarrow n+1$

+1w

Modu - III [No. of Symbols]

1. $\{w \mid w \in \{a, b\}^+, n_a(w) = 2\}$
2. $\{w \mid w \in \{a, b\}^+, n_a(w) \leq 2\}$
3. $\{w \mid w \in \{a, b\}^+, n_a(w) \geq 2\}$
4. $\{w \mid w \in a^+, n_a(w) = 2\}$
5. $\{w \mid w \in a^+, n_a(w) \leq 2\}$
6. $\{w \mid w \in a^+, n_a(w) \geq 2\}$

1. No. of a 's = 2. $w \in \{a, b\}^+$

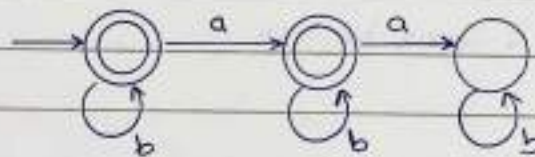


$$\#a's = 2$$

$$\#b's \geq 0$$

$$b^+ a b^+ a b^+$$

2. No. of a 's ≤ 2 $w \in \{a, b\}^+$



$$\#a's = 0$$

$$\#a's = 1$$

$$\#a's = 2$$

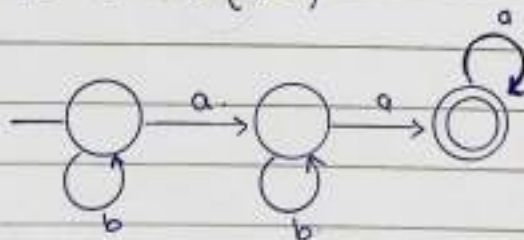
$$\#a's \leq 2$$

$$b^+(a + \epsilon)b^+(a + \epsilon)b^+$$

$$\#a's \leq 2$$

$$\#b's \geq 0$$

3. $\#a's \geq 2$, $w \in \{a, b\}^+$

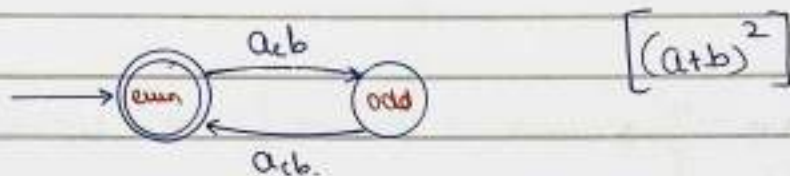


$$(a + b)^+ a (a + b)^+ a (a + b)^+$$

Module - IV Length and Remainder

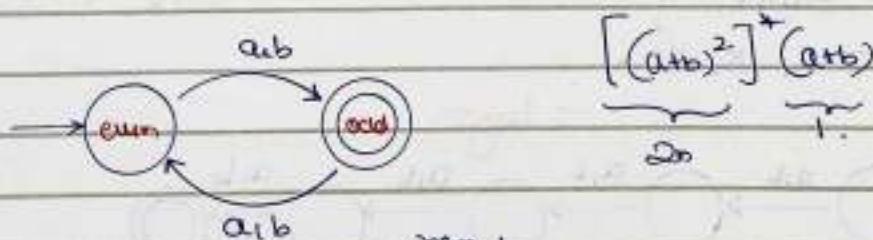
1. $\{w \mid w \in \{a,b\}^*, |w| \text{ is divisible by } 2\}$
 2. $\{w \mid \text{""} \text{ ""} \text{ ""} |w| \text{ is not divisible by } 2\}$
 3. $\{w \mid \text{""} \text{ ""} \text{ ""} |w| \text{ is divisible by } 3\}$
 4. $\{w \mid \text{""} \text{ ""} \text{ ""} |w| = 5n+2 \}$ nzo.
 5. $\{w \mid \text{""} \text{ ""} \text{ ""} |w| = 3n+5 \}$ nzo.
- } Complements to each other.

→ $\{w \mid w \in \{a,b\}^*, |w| = \text{even}\}$



rem=0	rem=1.
$ w =2$	$ w =1$
" 4	" 3
" 6	" 5
	...

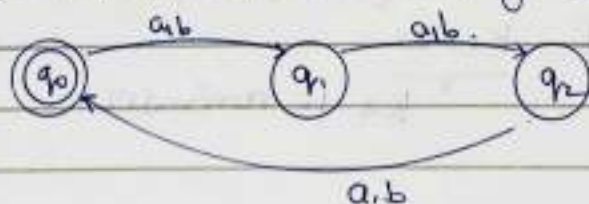
→ $\{w \mid w \in \{a,b\}^*, |w| = \text{odd}\}$



$$\begin{aligned}
 |w| &= 2n+1 \\
 |w| &= 1 \pmod{2} \\
 (|w| \pmod{2}) &= 1
 \end{aligned}$$

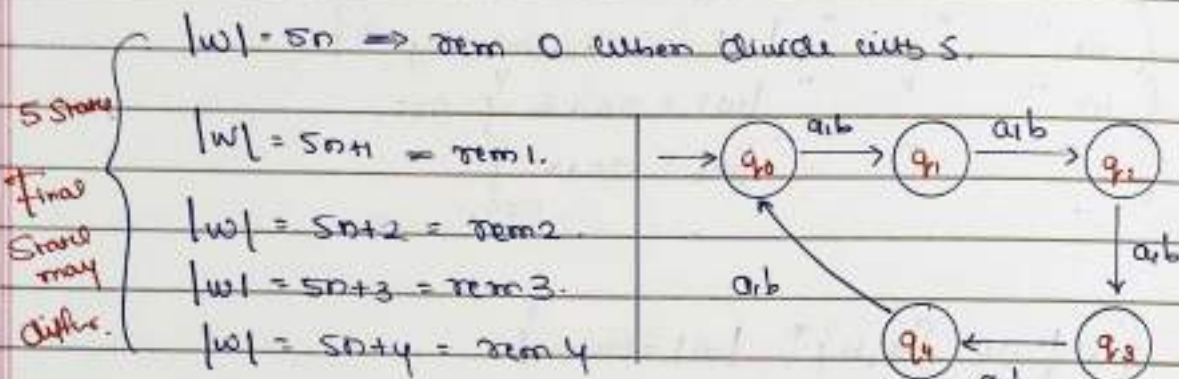
rem=1
$ w =1$
" 3
" 5
...

→ $\{w \mid w \in \{a,b\}^*, |w| \text{ is divisible by } 3\}$



Note: $L = \{w \mid w \in \{a,b\}^+, |w| \text{ is divisible by } k\}$.

No of states in min DFA = k states.



$|w| = 3n+0$
 $= 3n+1$

3 states.

$= 3n+2$

$= 3n+3$

$= 3n+4$

$= 3n+5$

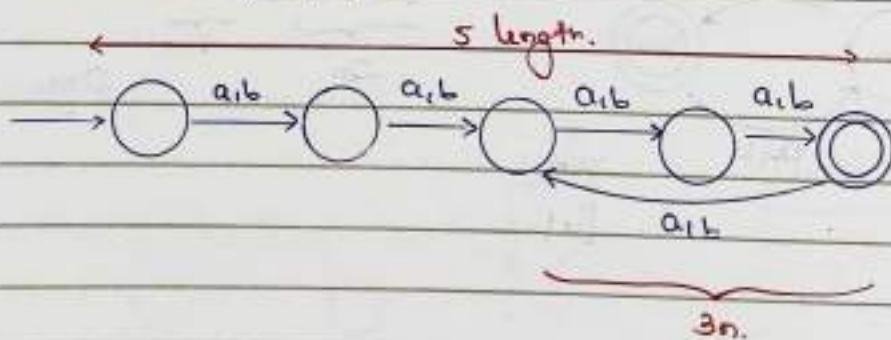
$= 3n + \text{constant } (k) \rightarrow k+1 \text{ States.}$

If $k_1 > k_2$, k_2 is the remainder.

If $k_1 \leq k_2$.

$\rightarrow \{ |w| = 3n+5 \}$
 $= 5, 8, 11 \dots$

$3n+2, n \geq 0.$



Note: $L = \{w \mid w \in \{a,b\}^+, |w| = k_1 n + k_2, n \geq 0\}$

I) If $k_1 > k_2$

k_2 is remainder

$\rightarrow k_1$ states

II) If $k_1 \leq k_2 \Rightarrow k_2 + 1$ states.
 k_2 is not remainder.

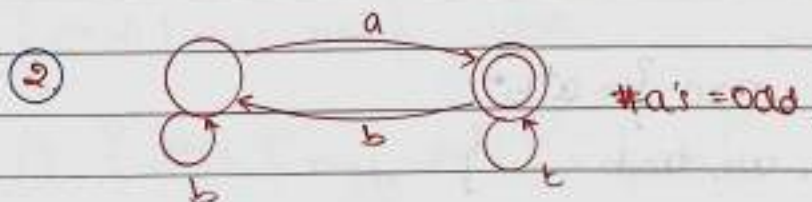
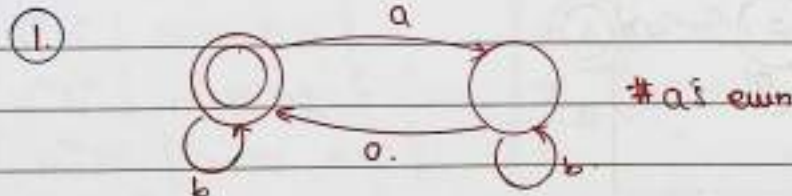
Note: $L = \{w \mid w \in \{a, b\}^*, \#a(w) = k_1 n + k_2, n \geq 0\}$.

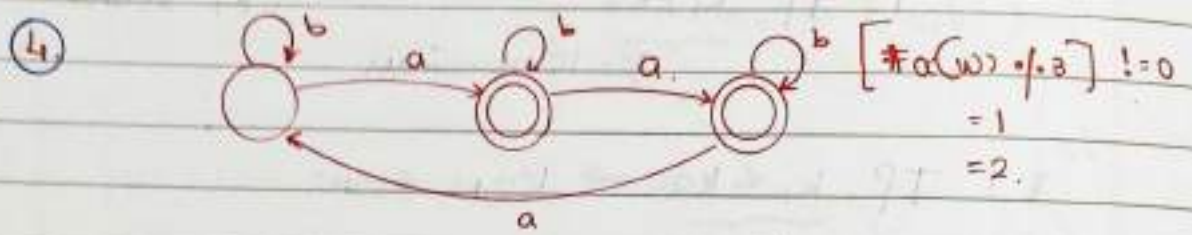
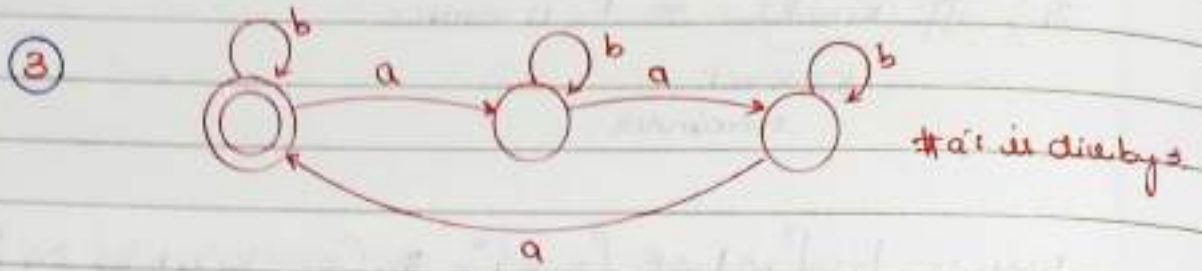
I) If $k_1 > k_2 \Rightarrow k_1$ states.
 k_2 is remainder.

II) If $k_1 \leq k_2 \Rightarrow k_2 + 1$ states.
 k_2 is not remainder.

Model - IV [No. of Symbols and Remainder]

1. $\{w \mid w \in \{a, b\}^+, \#a(w) = \text{even}\}$ } 2 states
2. $\{w \mid \text{" " " " } \#a(w) = \text{odd}\}$ }
3. $\{w \mid \text{" " " " } \#a(w) \text{ divisible by } 3\}$ }
4. $\{w \mid \text{" " " " } \#a(w) \text{ not divisible by } 3\}$ }
5. $\{w \mid \text{" " " " } \#a(w) = 100n + 53, n \geq 0\} \rightarrow 100 \text{ states}$
6. $\{w \mid \text{" " " " } = 123n + 1125, n \geq 0\} \rightarrow 1126 \text{ states}$



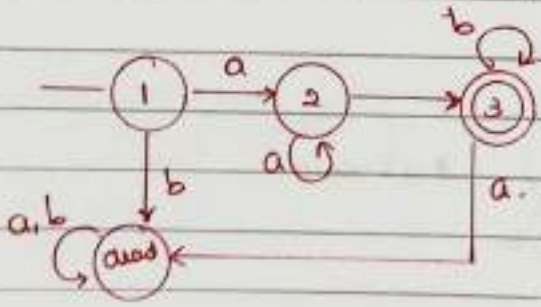


Module VI. [Sequence]

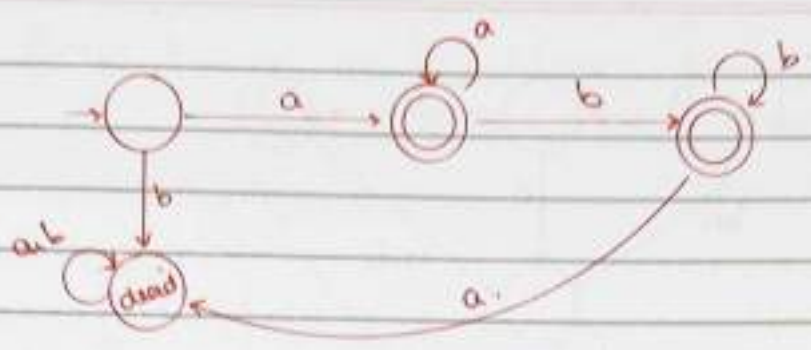
1. a^+b^+
 2. a^+b^+
 3. a^+b^+
 4. a^+b^+
- } a^+ followed by b^+

→ $L = \{ a^m b^n \mid m \geq 1, n \geq 1 \} = a^+ b^+$
 $= \{ ab, aab, abb, \dots \} = a^+ a b b^+ = a^+ a b^+ b = (a^+)^+ (b^+)^+ = (a^+)^+ (b^+)^+$

} a^+ and b^+ appear after b^+ & b^+ and a^+ before a^+ .



→ $\{ a^m b^n \mid m \geq 1, n \geq 0 \} = a^+ b^*$
 $= \{ a, aa, ab, aab, \dots \}$



→ $L = a^+b^+$
 $= \{a^m b^n \mid m \geq 0, n \geq 1\}$
 $= \{b \dots \}$
 (min. $a^1 b^1$)

= 3 state.

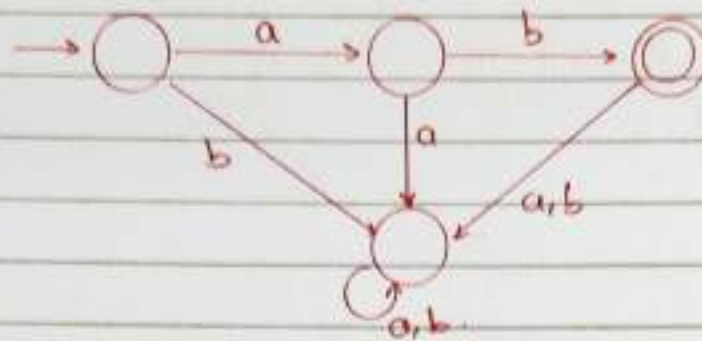
→ $a^+b^+ = \{a^m b^n \mid m \geq 0, n \geq 0\}$
 $= \{e, \dots\}$
 (min. $a^0 b^0$)

How many?
 5. $L = b^+a^+$
 6. $L = b^+a$
 7. $L = b^+a^1$
 8. $L = b^+a^+$
 9. $L = a^+b^+c$
 10. $L = b^+a^+c^+$

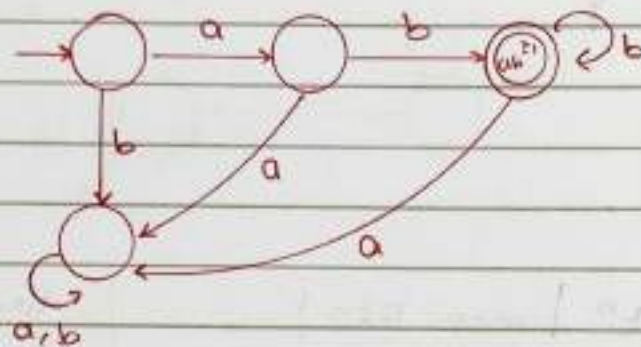
Modu-7: [sequence, no. of symbols]

- | | |
|--|--|
| 1. $\{a^m b^n \mid m=1, n=1\}$ | 6. $\{a^m b^n \mid m=\text{even}, n=1\}$ |
| 2. $\{a^m b^n \mid m=1, n \geq 1\}$ | 7. $\{a^m b^n \mid m=\text{odd}, n=1\}$ |
| 3. $\{a^m b^n \mid m \geq 1, n \geq 1\}$ | 8. " $m=1, n=\text{even}$ |
| 4. $\{a^m b^n \mid m \leq 1, n=1\}$ | 9. " $m=\text{even}, n=\text{even}$ |
| 5. $\{a^m b^n \mid m \leq 1, n \leq 1\}$ | 10. " $m=\text{even}, n=\text{odd?}$ |

① $\{a^m b^n \mid m=1, n=1\} = \{ab\}$

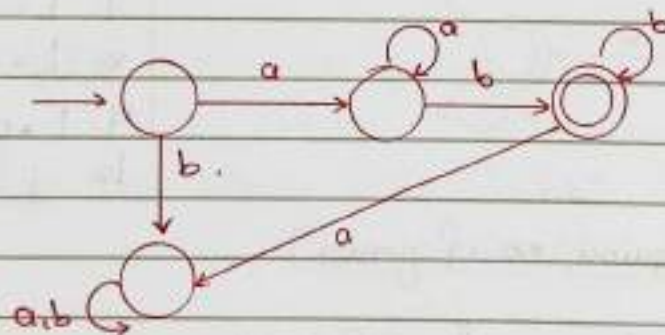


② $\{a^m b^n \mid m \geq 1, n \geq 1\} = \{\underbrace{ab}_{\text{min}}, \dots\}$

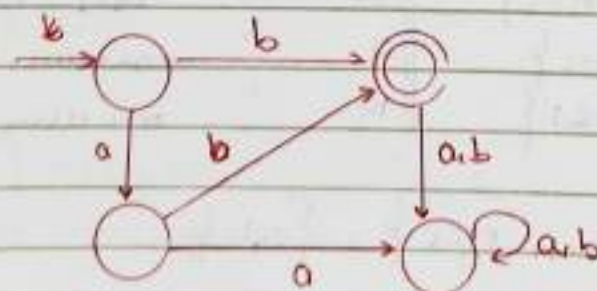


$L = ab^+$

③ $L = \{a^m b^n \mid m \geq 1, n \geq 1\} = a^+ b^+$

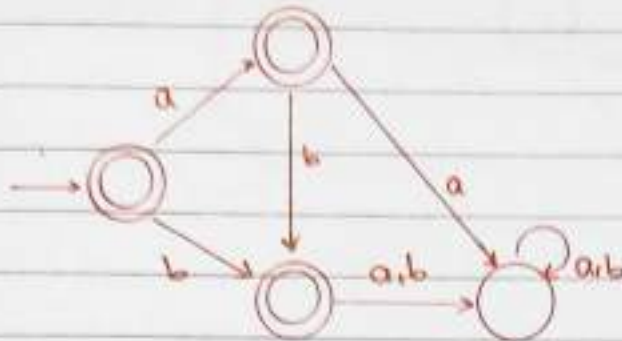


④. $\{a^m b^n \mid m \leq 1, n \geq 1\} = (\epsilon a) b = b + ab$



= 4 states.

$$\textcircled{5} \{a^m b^n \mid m \leq 1, n \leq 1\} = (\epsilon + a) (\epsilon + b) \\ = (a^0 + a^1) (b^0 + b^1) = \{\epsilon, a, b, ab\}$$

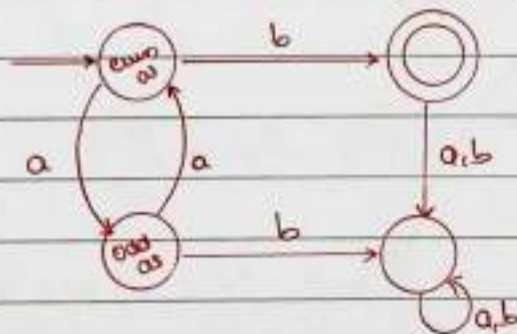


Ques

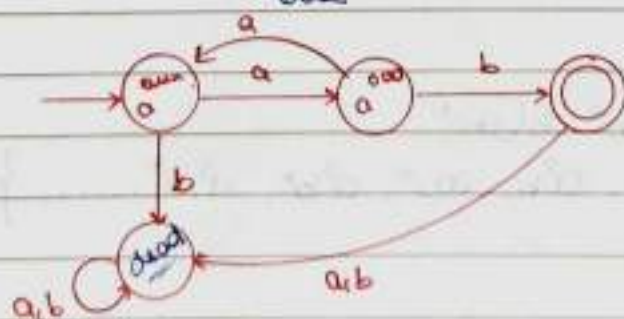
Probu - VIII [Multiple conditions, No. of symbols]

1. $\{w \mid w \in \{a, b\}^*, n_a(w) = 1, n_b(w) = 2\}$
2. $\{w \mid \text{" " } n_a(w) \geq 1, n_b(w) \geq 2\}$
3. $\text{" " } n_a(w) \leq 1, n_b(w) \leq 2\}$
4. $\text{" " } n_a(w) \leq 1, n_b(w) \leq 2\}$
5. $\text{" " } n_a(w) \geq 1, n_b(w) \leq 2\}$

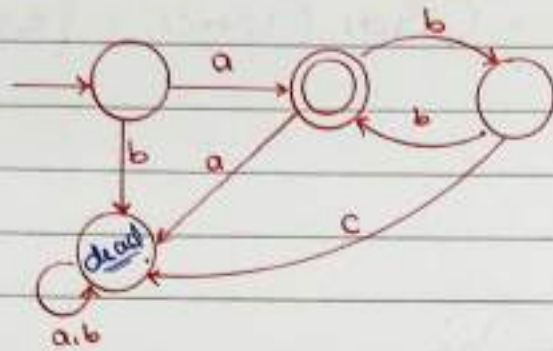
$$\textcircled{6} a^{\text{even}} b^1 \Rightarrow (aa)^* b = \{b, a^2b, a^4b, a^6b, \dots\}$$



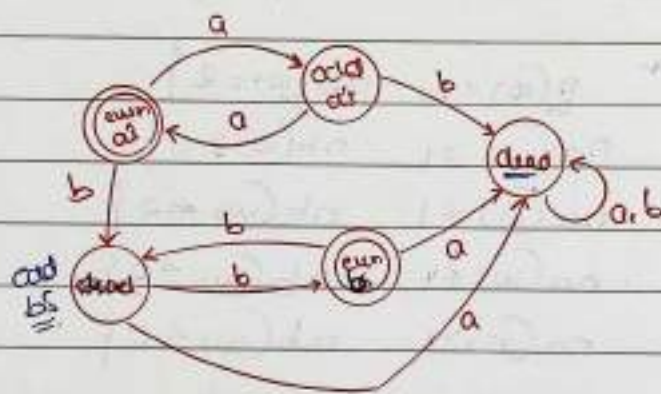
$$\textcircled{7} a^{\text{odd}} b^1 \Rightarrow \underline{a(aa)^*} b = \{ab, a^3b, a^5b, \dots\}$$



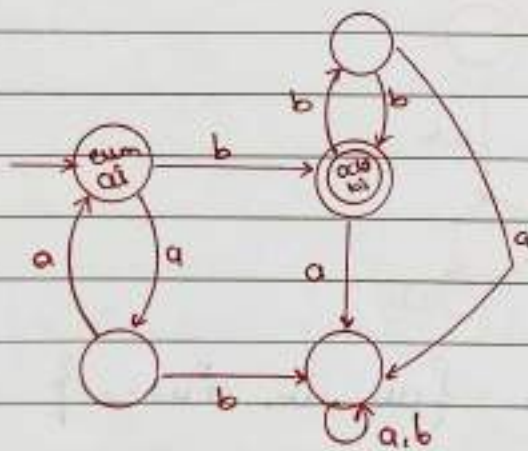
8. $a^1 b^{\text{even}} = a(bb)^+ \Rightarrow \{a, ab^2, ab^4, \dots\}$



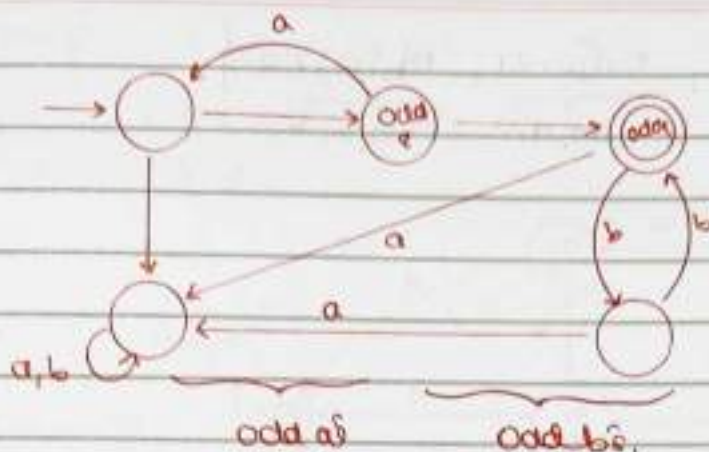
9. $a^{\text{even}} b^{\text{even}} = (aa)^+ (bb)^+.$
 $= \{ \epsilon, a^2, b^2, a^4, b^4, \dots \}$



10. $a^{\text{even}} b^{\text{odd}} = (aa)^+ b(bb)^+.$

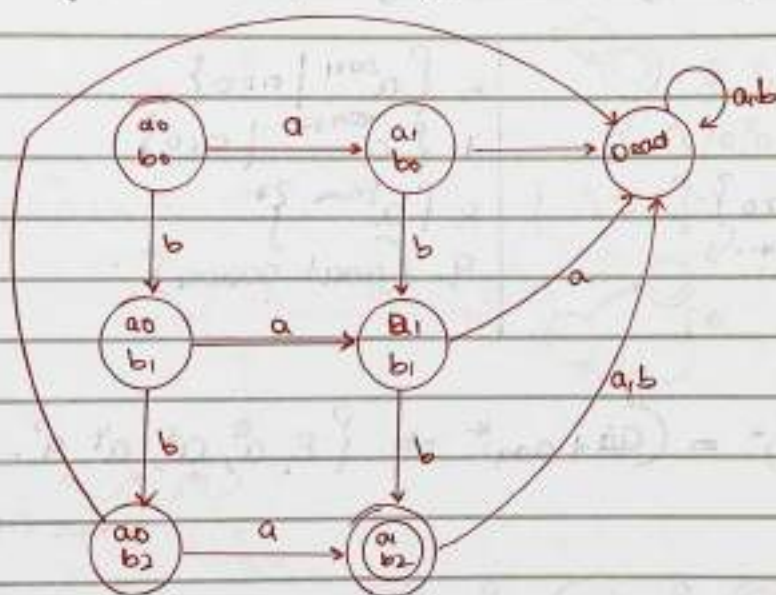


11. $a^{\text{odd}} b^{\text{odd}} = a(aa)^+ b(bb)^+.$
 $= \{ ab, a^3b, ab^3, a^3b^3, a^5b^3, \dots \}$

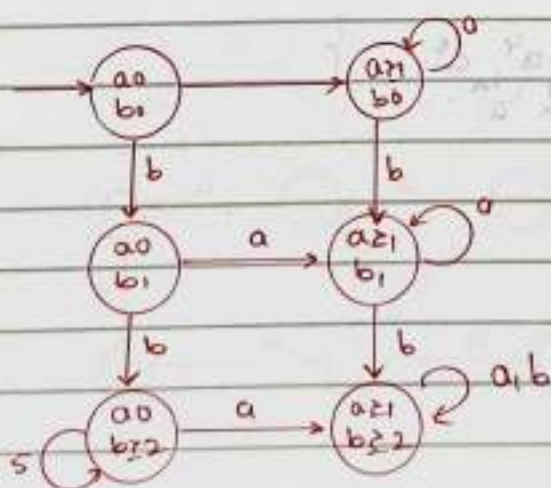


Model VIII (Solutions)

1. $\{w \mid w \in \{a,b\}^+, n_a(w)=1, n_b(w)=2\}$
 $= \{abb, bab, bba\}$



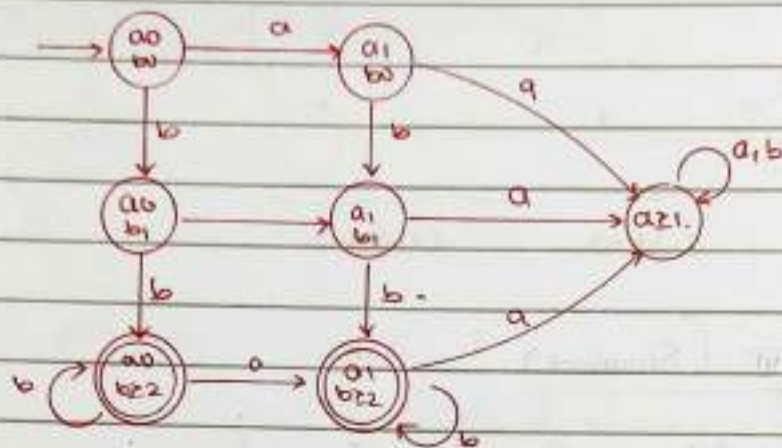
2. $\{w \mid w \in \{a,b\}^+, n_a(w) \geq 1, n_b(w) \geq 2\}$



$$a_1 \Rightarrow a_{\geq 1}$$

$$b_2 \Rightarrow b_{\geq 2}$$

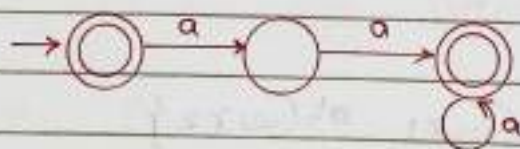
3. $\{w \mid w \in \{a,b\}^*, \underbrace{n_a(w) \leq 1}_{a_0, a_1}, \underbrace{n_b(w) \geq 2}_{b_0, b_1, b_2}\}$



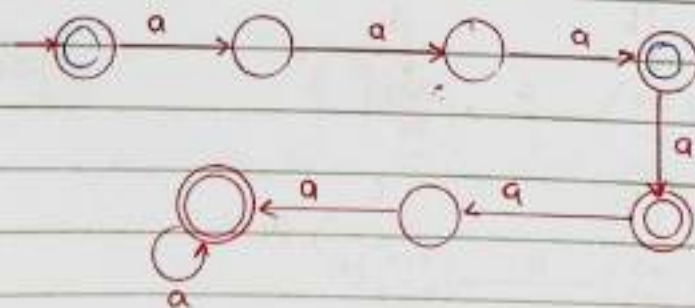
Model IV [Language over 1 Symbol]

- | | |
|---|-------------------------------------|
| 1. $a^+ = \{a, a^2, a^3, \dots\}$ | 6. $\{a^{3n+1} \mid n \geq 0\}$ |
| 2. $(aa)^+ = \{aa, a^2a^2, a^4, a^6, \dots\}$ | 7. $\{a^{100n+123} \mid n \geq 0\}$ |
| 3. $\{a^n a^{3n} \mid n \geq 0\} = \{\epsilon, a^3, a^6, \dots\}$ | 8. $\{a^{\text{Prime}}\}^+$ |
| 4. $(aa + aaa)^+$ | 9. $(aaat + aaacat)^+$ |
| 5. $(a^3 + a^4)^+$ | |

4. $(aa + aaa)^+ = (aa + aaa)^+ \Rightarrow \{\epsilon, a^2, a^3, a^4, a^5, \dots\}$
 forme AP.



5. $(a^3 + a^4)^+ = \{\epsilon, a^3, a^4, a^6, \dots\}$

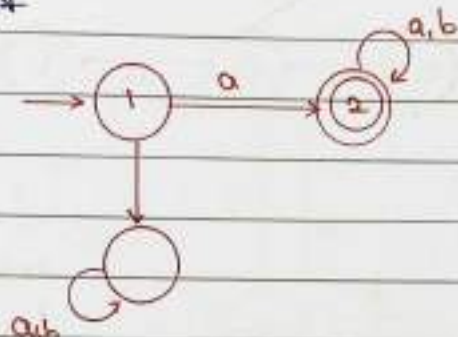


7
16
4.

$$\begin{aligned} \{a^{\text{Prime}}\}^+ &= \{a^2, a^3, a^5, a^7, \dots\}^+ \\ &= \{\epsilon, a^2, a^3, a^4, a^5, \dots\} \\ &= \{a^+\} \\ &= \{\epsilon + aaa^+\} \\ &= \{a\} = (a^2 + a^3)^+ \end{aligned}$$

Module 8

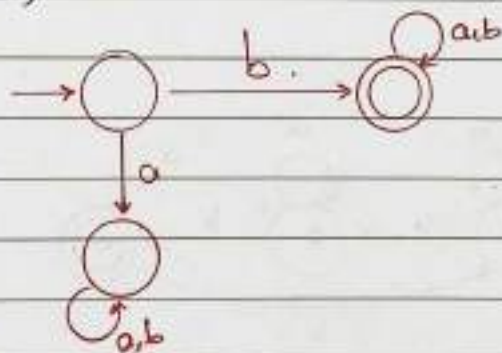
① $L = aCab^+$



$$\begin{aligned} \text{min} &= a \\ &= (|a| + 1) + 1 \\ &= 2 + 1 \\ &= 3 \end{aligned}$$

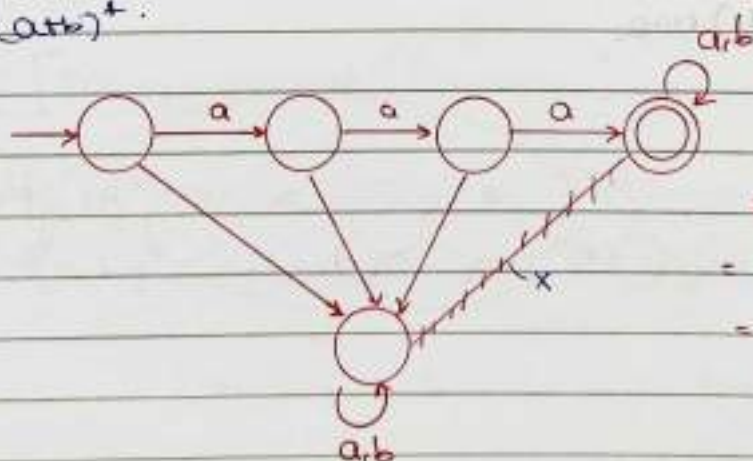
← dead

② $L = b(a+b)^+$



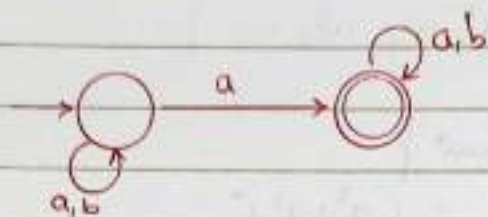
$$\begin{aligned} \text{min} &= b \\ &= |b| + 2 \\ &= 1 + 2 \\ &= 3 \end{aligned}$$

③ $L = aaa(Cab)^+$



$$\begin{aligned} \text{min} &= aaa \\ &= |aaa| + 2 \\ &= 3 + 2 \\ &= 5 \end{aligned}$$

4. $L = (a+ab)^* a (a+ab)^*$

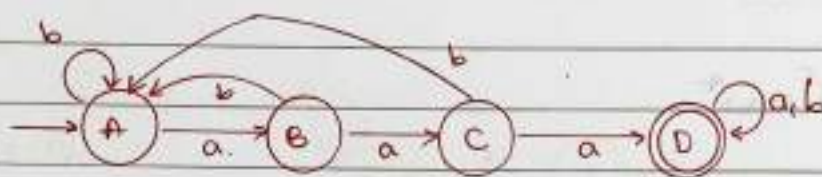


$$\text{min} = a$$

$$|a| + 1 = 1 + 1$$

$$= 2 \text{ states}$$

5. $L = (a+ab)^* aaa (a+ab)^*$



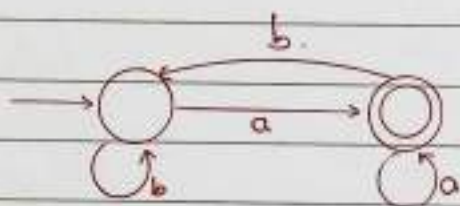
$$\text{min} = aaa$$

$$|aaa| + 1$$

$$= 3 + 1$$

$$= 4$$

6. $L = (a+ab)^*$

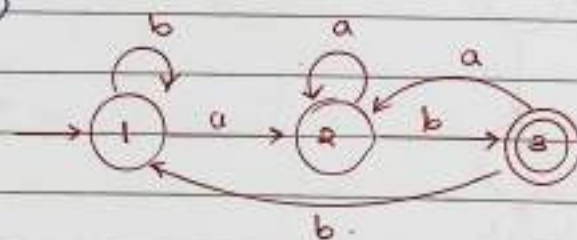


$$\text{min} = a$$

$$|a| + 1$$

$$= 2 \text{ states}$$

7. $L = (a+ab)^*$



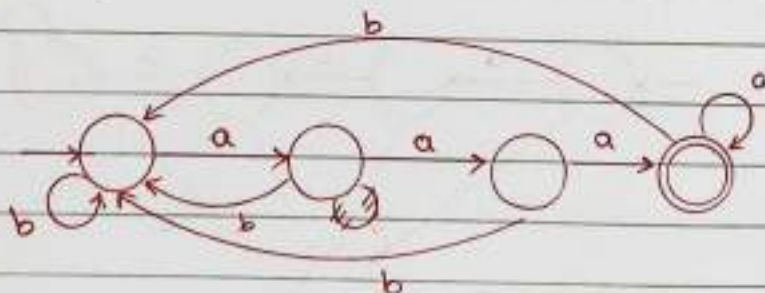
$$\text{min} = ab$$

$$|ab| = 1$$

$$2 + 1$$

$$= 3 \text{ states}$$

8. $L = (a+ab)^* aaa$

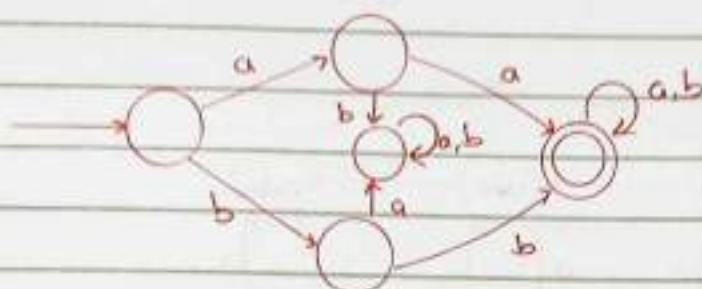


$$|aaa| + 1$$

$$= 3 + 1$$

$$= 4 \text{ states}$$

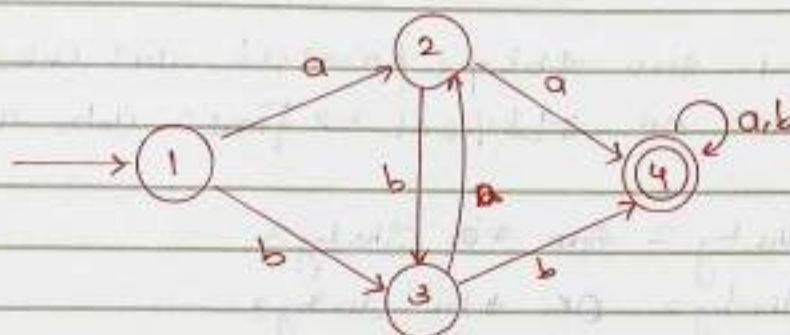
9. $L = (aatbb)(atb)^*$



min = aa, bb

= 5 states

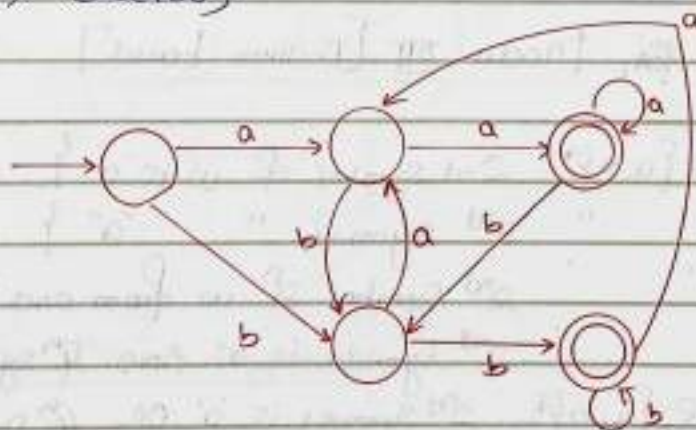
10. $L = (atb)^*(aatbb)(atb)^*$



min = aa, bb

= 4 states

11. $L = (atb)^*(aatbb)$

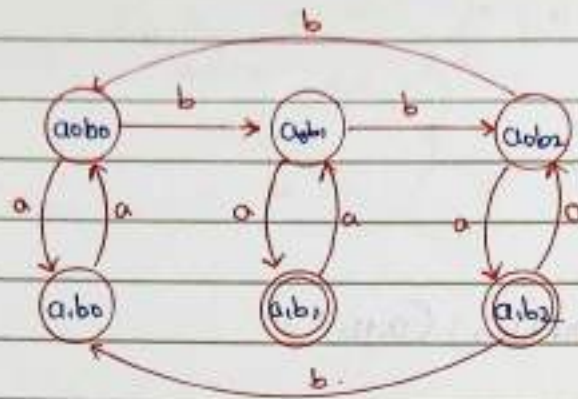


Modus - XI [Multiple conditions, Remainder]

1. $\{w \mid w \in \{a,b\}^*, na(w) \text{ is divisible by 2, } nb(w) \text{ is divisible by 2}\}$
2. $\{w \mid w \in \{a,b\}^*, na(w) \text{ is div. by 2 or } nb(w) \text{ is divisible by 2}\}$
3. $\{w \mid w \in \{a,b\}^*, na(w) \text{ is div. by 2 and } nb(w) \text{ is divisible by 2}\}$
4. $\{w \mid w \in \{a,b\}^*, na(w) \text{ is divisible by 3, And } nb(w) \text{ is div by 3}\}$

OR
BUT
NOT

Method 1.



II: $\#a_i \cdot 2 = 0$ or $\#b_i \cdot 3 = 1 \Rightarrow$ final c. $a_0 b_0, a_0 b_1, \dots$

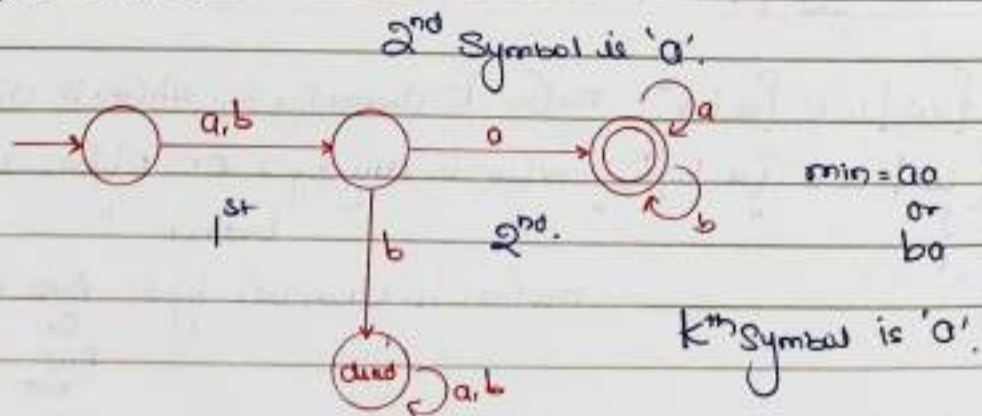
4. (ii) a_i din by 2 OK b_i din by 3

q. (ii), ~~that~~ dialyze 2 burner to 3 dialyze.

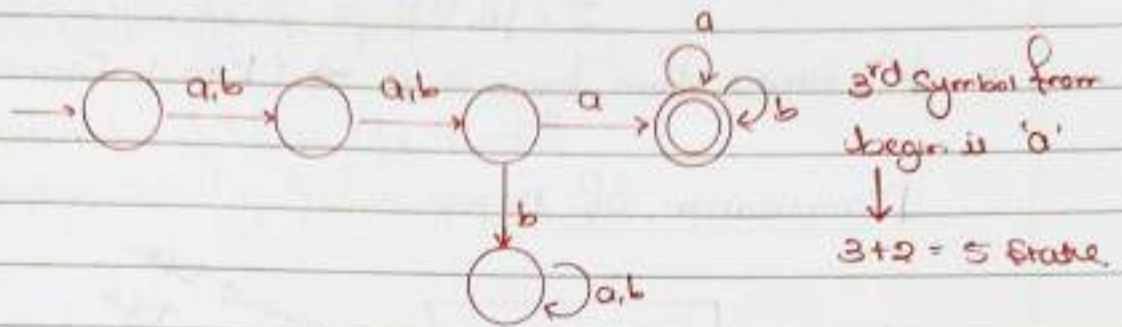
Exs. P_n trudu IT [Position based]

1. $L = \{ w \mid w \in \{a, b\}^+, \text{ 2nd symbol of } w \text{ is 'a'} \}$
2. $L = \{ w \mid w \in \{a, b\}^+, \text{ 3rd symbol of } w \text{ is 'a'} \}$
3. $L = \{ w \mid w \in \{a, b\}^+, \text{ 2nd symbol of } w \text{ from end is 'a'} \}$
4. $L = \{ w \mid w \in \{a, b\}^+, \text{ 2nd symbol is 'a' AND 4th symbol is 'b'} \}$
5. $L = \{ w \mid w \in \{a, b\}^+, \text{ 2nd symbol is 'a' OR 4th symbol is 'b'} \}$

1. $(a+b)a(a+b)^+$

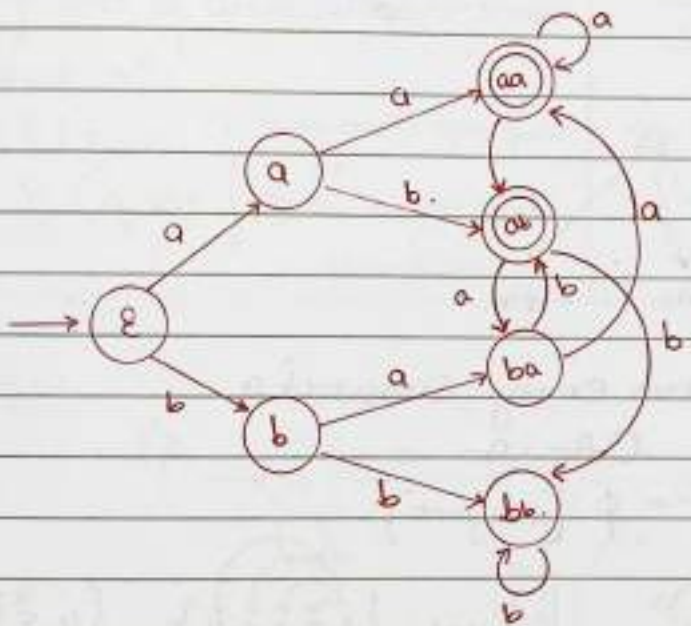


2. $L = (a+ b)(a+ b) a (a+ b)^+$



3. 2nd Symbol from end is 'a':

Method 1: Tree Approach:



$$aa \xrightarrow{a} aa$$

$$aa \xrightarrow{b} ab$$

$$ab \xrightarrow{a} ba$$

$$ab \xrightarrow{b} bb$$

$$ba \xrightarrow{a} aa$$

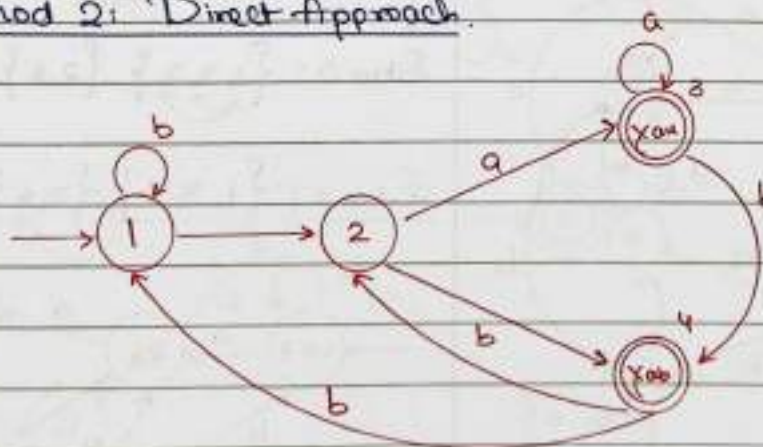
$$ba \xrightarrow{b} ab$$

$$bb \xrightarrow{a} ba$$

$$bb \xrightarrow{b} bb$$

Trace last
2 Symbols

Method 2: Direct Approach.



1: wait for a/b

2: wait for a/b

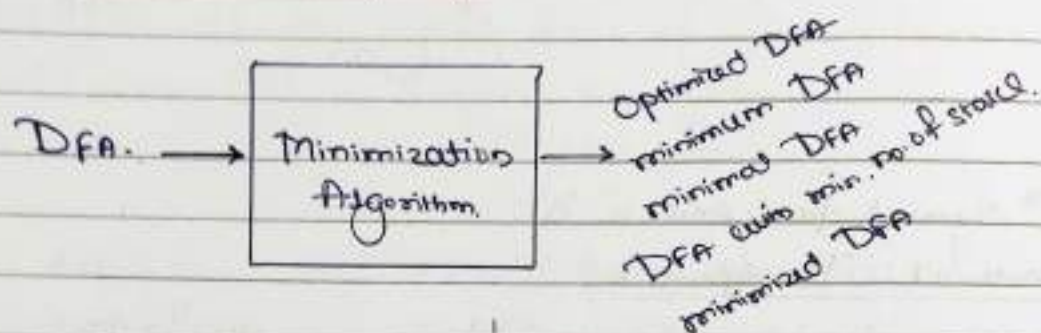
3: xaa

4: xab

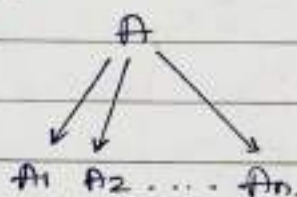
Note: I. K^n symbol from end is 'a' $\rightarrow 2^k$ states in DFA.
 $\Sigma = \{a, b\}$.

II. K^n State from begin is 'a' $\rightarrow (K+2)$ State.

Minimization of DFA:



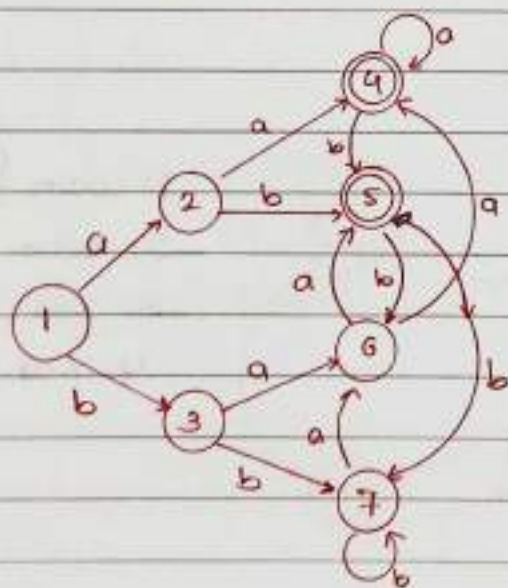
Partition on Set A.



I. Every A_i is non empty subset of A

II. $A_1 \cup A_2 \cup \dots \cup A_n = A$

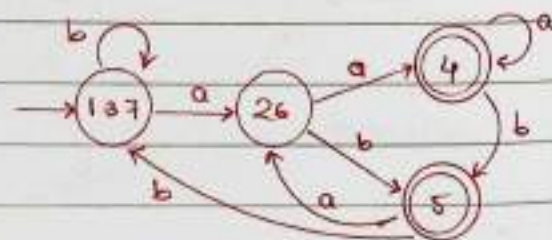
III. $\forall i, j, A_i \cap A_j = \emptyset$ [Disjoint]



Step 1: $\{1, 2, 3, 6, 7\}$ $\{4, 5\}$
 ↓ a, b All non finals All finals.

Step 2: $\{1, 3, 7\}$ $\{2, 6\}$ $\{4\}$ $\{5\}$.

Step 3: $\{1, 3, 7\}$ $\{2, 6\}$ $\{4\}$ $\{5\}$



Partitions on Set Q

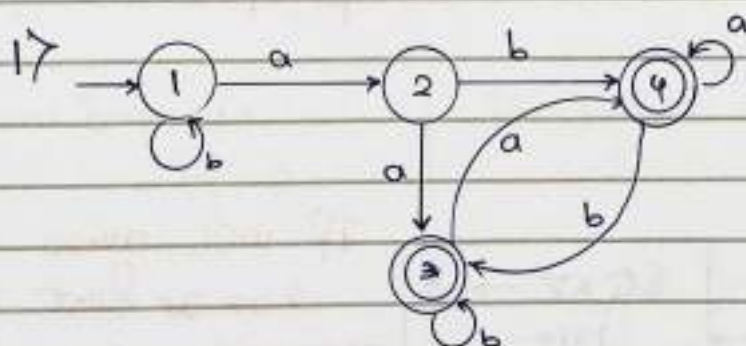
$$Q = \{q_1, q_2, q_3, q_4, q_5\}$$

Partition 1: $\{q_1, q_2, q_3, q_4, q_5\}$

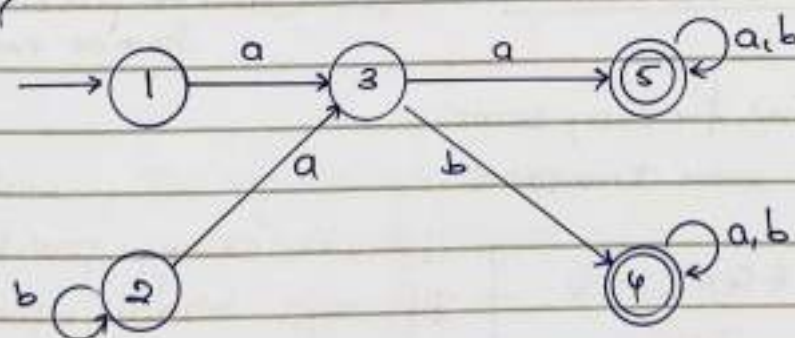
Partition 2: $\{q_1\} \{q_2, q_3, q_4, q_5\}$

Equivalent States, (non distinguishable)	Distinguishable States, (non equivalent)
p and q are equivalent states. $\forall w \in \Sigma^+$ $\hat{\delta}(p, w)$ } Both must go to final or $\hat{\delta}(q, w)$ } Both must go to non-final.	$\exists w \in \Sigma^+$ $p \xrightarrow{w} \text{final}$ or $p \xrightarrow{w} \text{non final}$ $q \xrightarrow{w} \text{final}$ or $q \xrightarrow{w} \text{non final}$

Ex



2>



Model - XIII [Special]

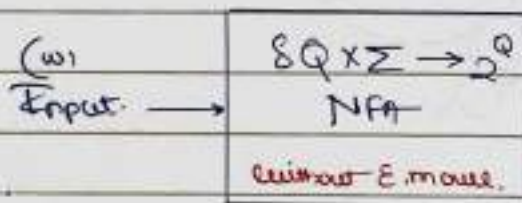
1. $\{w \mid w \in \{0,1\}^*, \text{Decimal}(w) \text{ is divisible by } 8\}$

$\xrightarrow{\text{Div by } 2^k}$
 $\downarrow$
 $k+1 \text{ states}$

 $(2^3 = 3+1 = 4 \text{ states})$
2. $\{w \mid w \in \{0,1\}^*, \#01(w) = \#10(w)\}$
 $= \{\epsilon, 0, 1, 00, 11, 000, \dots\}$
3. $\{w \mid w \in \{0,1\}^*, \text{every prefix } p \text{ of } w \text{ satisfies } |n_0(p) - n_1(p)| \leq 1\}$
 $\{\epsilon, 0, 1, \dots\}$

w	prefix	Condition
ϵ	ϵ	$\checkmark$
0	0	$ 1-0 =1 \checkmark$
1	1	$ 2-0 =2$

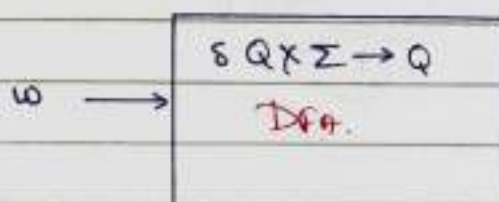
NFA



If $w \in L$, atleast 1 path that
ends at final.

If $w \notin L$, there is no path that
ends at final (either no path or every path
ends at non-final).

From every state, for every state,
Zero or more transitions.



If $w \in L$, exactly 1 path that ends at final.
If $w \notin L$, exactly 1 path that ends at
non-final.

For every state, for every $\forall$ Symbol, exactly 1 transition.

Note.

I. Every DFA is NFA

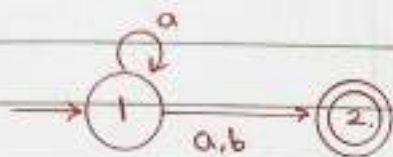
II. NFA may or may not be DFA

III. Min. DFA need not be min NFA

IV. For every regular language:

$$n(\text{min DFA}) \geq n(\text{min NFA})$$

eg:



	No. of paths	Valid/Invalid.
ϵ	1	Invalid
a	2	Valid
b	1	Valid
aa	2	Valid
ab	1	Valid
ba	0	Invalid

1. $L = \{ \} \text{ over } \Sigma = \{a, b\}$

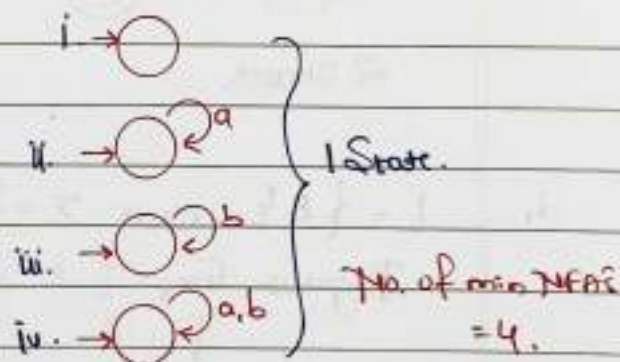
Regular Expression:

$$R = \phi$$

Min DFA:



Min. NFA:-



Note: For regular set,

i. No. of DFAs = Infinite

ii. No. of min DFAs = 1

iii. No. of NFAs = Infinite

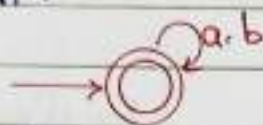
iv. No. of min NFAs = 1 or more.

2. $L = \Sigma^+$ over $\Sigma = \{a, b\}$

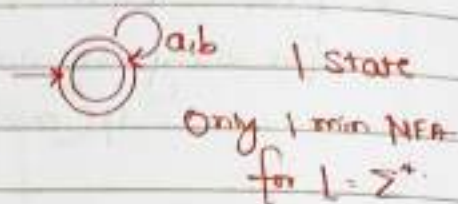
Regular Expressions:

$$R = (a+b)^+ \\ = (a^+b^+)^+ \cdot (b^+a^+)^+$$

min DFA:



min NFA:

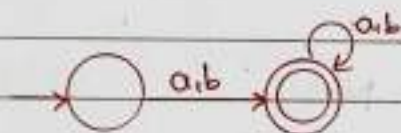


3. $L = \Sigma^+$ over $\Sigma = \{a, b\}$

Regular Expressions:

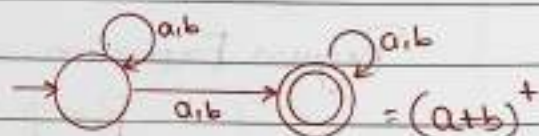
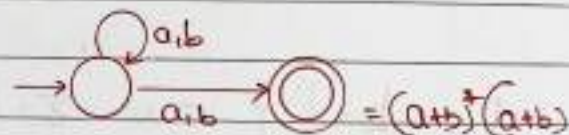
$$R = (a+b)^+ = (a+b) \cdot (a+b)^+ \\ = (a+b)^+ \cdot (a+b)$$

min DFA:



2 states.

min NFA: 2 states.

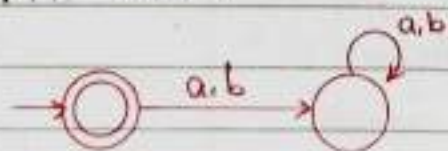


4. $L = \{\epsilon\}$ over $\Sigma = \{a, b\}$

Regular Exp:

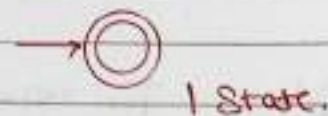
$$R = \epsilon \\ = \phi^+ = \epsilon^+ = \epsilon^+$$

min DFA:



2 states.

min NFA:

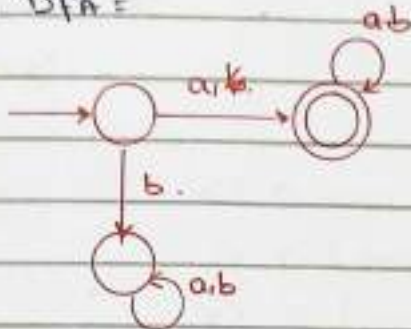


5. $L = \{w \mid w \in \{a,b\}^+, w \text{ starts with 'a'}\}$

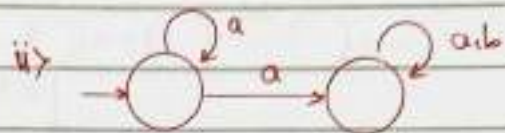
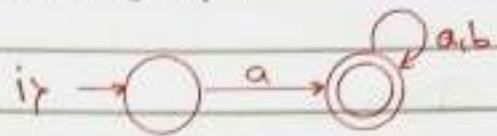
Regular Expression:

$$R = a(ab)^+ \\ = (ab^+)^+$$

Min DFA:



NFA (min):

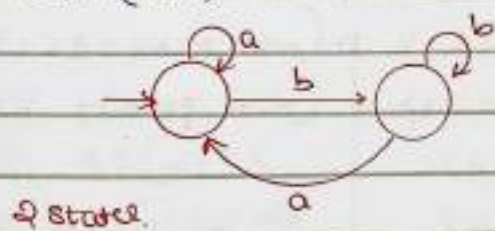


6. $L = \{w \mid w \in \{a,b\}^+, w \text{ ends with 'b'}\}$

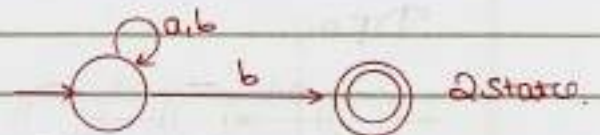
Regular Expression:

$$R = (a+b)^+ b \\ = (a^+b)^+$$

DFA (min):



NFA (min):

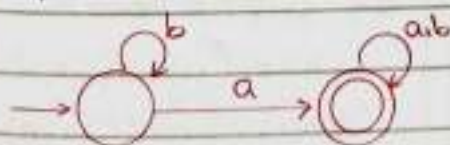


7. $L = \{w \mid w \in \{a,b\}^+, w \text{ contains 'a'}\}$

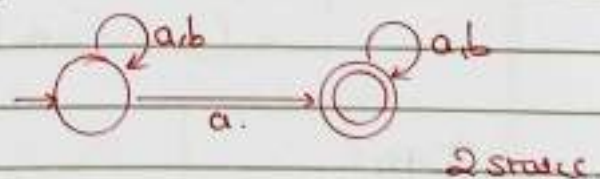
Regular Expression:

$$R = \Sigma^+ a \Sigma^+ \\ = b^+ a (a+b)^+$$

DFA:



NFA:

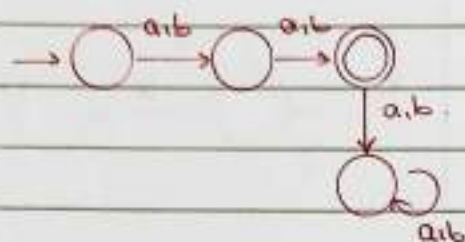


8. $L = \{w \mid w \in \{a,b\}^*, |w| \geq 2\}$

Regular Expression:

$$= (a+b)^2$$

DFA:



NFA:



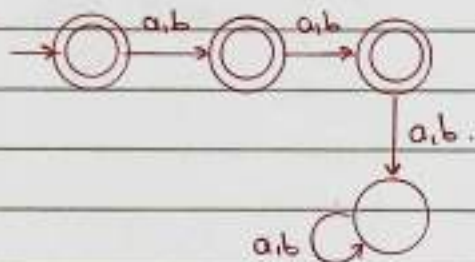
3 States

9. $L = \{w \mid w \in \{a,b\}^*, |w| \leq 2\}$

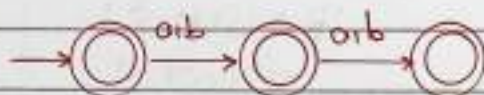
Regular Expression

$$= (\epsilon + a + b)^2$$

DFA:



NFA:



3 States

If $|w| \leq k$ then

i) $n(\text{min DFA}) = k+2$

ii) $n(\text{min NFA}) = k+1$

10. $L = \{w \mid w \in \{a,b\}^*, |w| \geq 2\}$

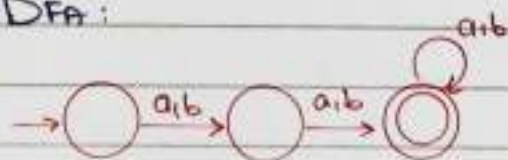
Regular Expression:

$$= (a+b)^2 (a+b)^*$$

$$= (a+b)^* (a+b)^2$$

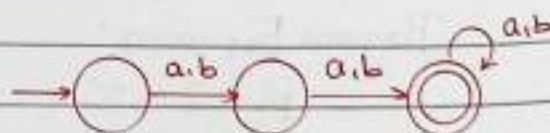
$$= (a+b) (a+b)^* (a+b)$$

DFA:



3 States

NFA:



3 States

If $|w| \geq k$ then

i) $n(\text{min DFA}) = k+1$

ii) $n(\text{min NFA}) = k+1$

		Min DFA	Min NFA
11.	$L = (a+b)^2 a (a+b)^*$	5 States	4 States
12.	$L = (a+b)^* a (a+b)^*$	8 States	4 States
13.	$b^* a b^* a b^*$	4 States	3 States
14.	$b^* (a+c) b^* (a+c) b^*$	4 States	3 States
15.	$\Sigma^* a \Sigma^* a \Sigma^*$	3 States	3 States

Note: I. K^{th} Symbol from begin 'a' $\Sigma = \{a, b\}$

i) $n(\text{min DFA}) = k+2$

ii) $n(\text{min NFA}) = k+1$

II. K^{th} Symbol from end is 'a'

i) $n(\text{min DFA}) = 2^k$

ii) $n(\text{min NFA}) = k+1$

$w \in \{a, b\}^*$	Min DFA	min NFA
I. $ w = k$	$k+2$	$k+1$
II. $ w \leq k \Rightarrow$	$k+2$	$k+1$
III. $ w \geq k \Rightarrow$	$k+1$	$k+1$
IV. $\#a(w) = k$	$k+2$	$k+1$
V. $\#a(w) \leq k \Rightarrow$	$k+2$	$k+1$
VI. $\#a(w) \geq k \Rightarrow$	$k+1$	$k+1$
VII. K^{th} Symbol is 'a'	$k+1$	$k+1$
VIII. K^{th} Symbol from end is 'a'	2^k	$k+1$

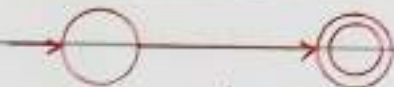
16.	$L = aaa (a+b)^*$	5	4
17.	$L = (a+b)^* aaa$	4	4
18.	$L = (a+b)^* aaa (a+b)^*$	4	4
19.	$L = (a+b)^2 a (a+b)^2 b (a+b)^*$	9	8

20.	$L = (abb)a(ab)^7a$ $(abb)^+$	12	11.
21.	$L = (ab)^+$	3 States	2 States.
22.	$L = (ba)^+$	3 States	2 States.
23.	$L = a(ba)^+$	3 States	2 States.

NFA

without ϵ move.

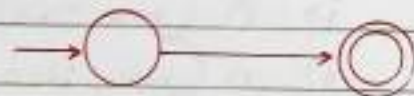
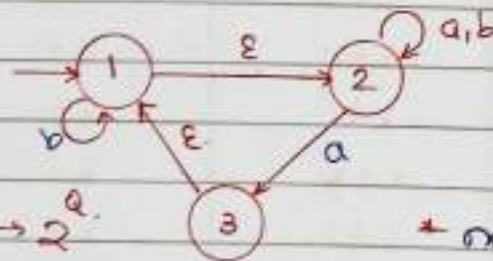
$$\delta: Q \times \Sigma \rightarrow 2^Q$$

not "NFA without
 ϵ moves.

NFA

with ϵ move.

$$\delta: Q \times \Sigma \cup \{\epsilon\} \rightarrow 2^Q$$

NFA with ϵ moves.

$$\delta: Q \times \Sigma \cup \{\epsilon\} \rightarrow 2^Q$$

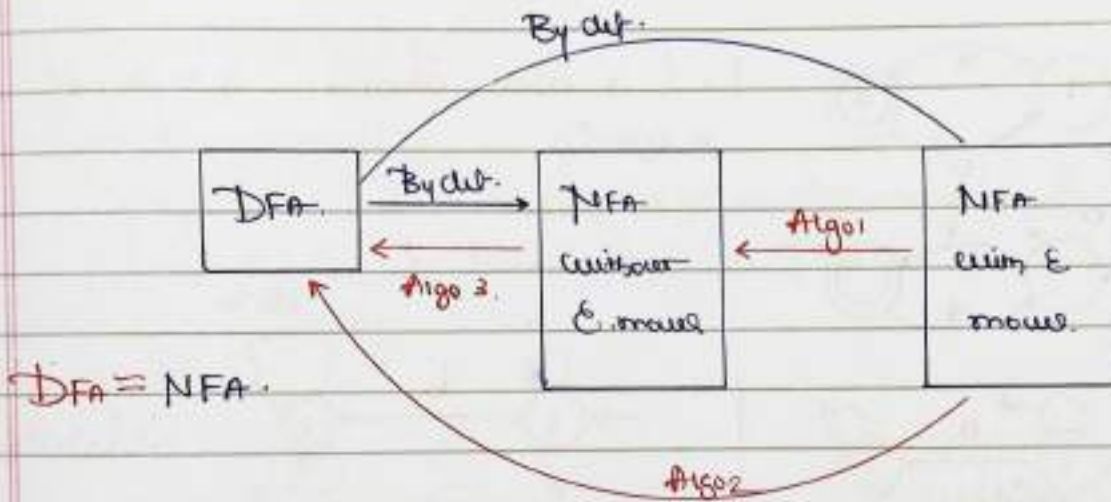
* not DFA

* not NFA without ϵ moves

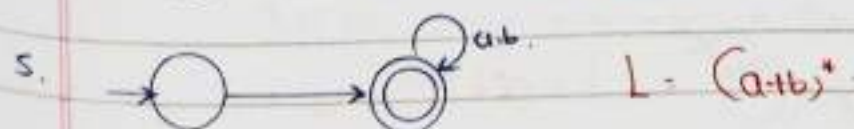
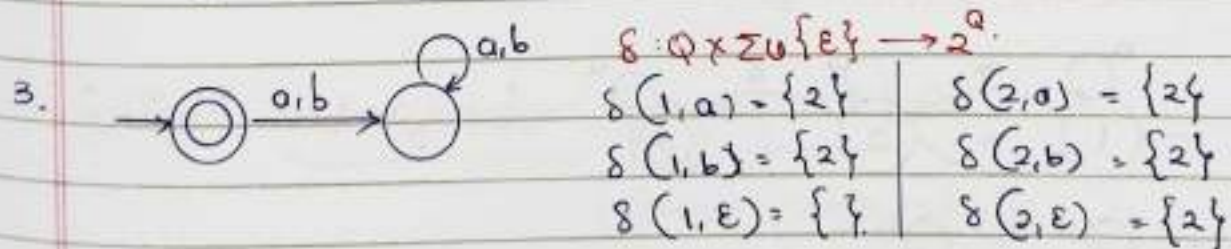
* It is NFA.

$(1, a)$	ϕ
$(1, b)$	$\{1\}$
$(1, \epsilon)$	$\{2\}$
$(2, a)$	$\{3\}$
$(2, b)$	$\{1, 2\}$
$(2, \epsilon)$	$\{1, 3\}$
$(3, a)$	$\{2, 3\}$
$(3, b)$	$\{1, 2, 3\}$
$(3, \epsilon)$	

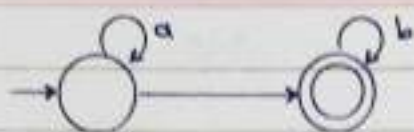
DFA	NFA
1. Every DFA is NFA	NFA may or may not be DFA
2. $\delta: Q \times \Sigma \rightarrow Q$	$\delta: Q \times \Sigma_{\epsilon} \rightarrow 2^Q$
3. $w \rightarrow$ DFA w/c, ex. on path. w/c, exactly one path.	$w \rightarrow$ NFA w/c at least 1 path. w/c 0 or more paths.
4. For every regular, unique min DFA exists.	For every regular, one or more min NFA's exist.



NFA with ϵ moves:

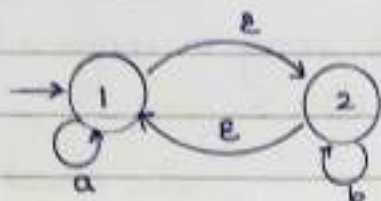


6.



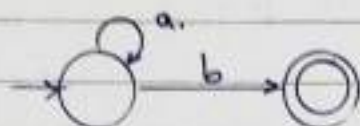
$$L = a^*b^*$$

7.



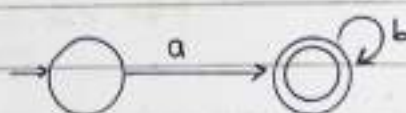
$$L = (a+ab)^* = (a^*b^*)^* \\ = (b^*a^*)^*$$

8.



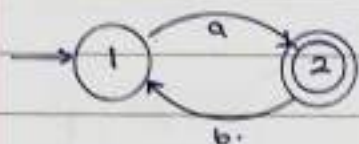
$$L = a^*b$$

9.



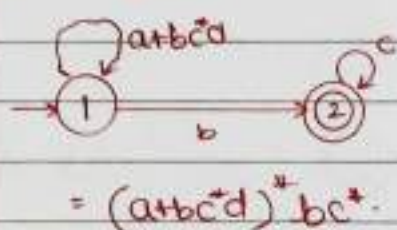
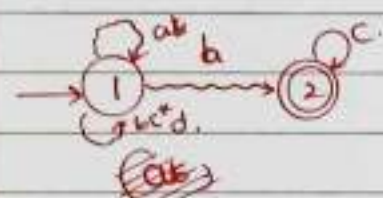
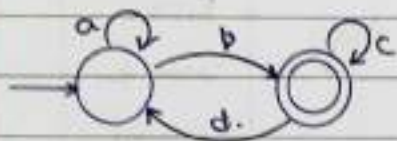
$$L = ab^*$$

10.

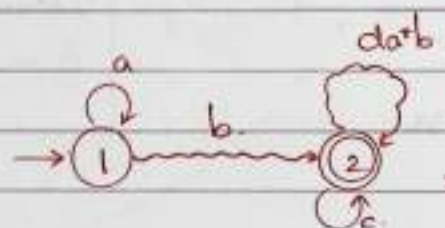


$$L = \{ \bar{a}, \bar{a}ba, \bar{a}ba\bar{b}ba, \dots \} = a(ba)^+ \\ = (ab)^+a$$

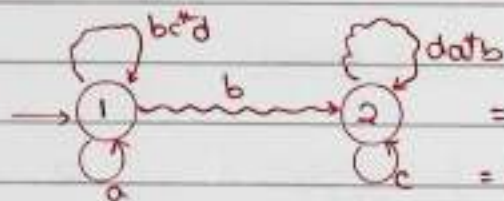
11.



$$= (atbca)^+bc^+$$

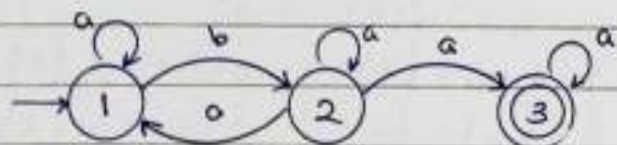


$$= ab(da^*b+c)^+$$



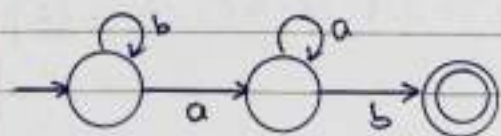
$$= ab(da^*b+c)^+ \\ = (atbca)^+bc^+$$

12.



$$L = (a+ba^*a)^+ba^*aa^*$$

13.



$$L = b^*a^*b \\ = b^+a^+b$$

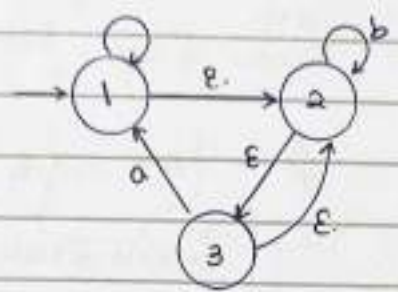
14.



$$L = a^*b^*c^*$$

NFA with ϵ moves:

ϵ -closure (state):



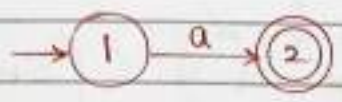
$\epsilon\text{-closure}(1) = \{1, 2, 3\}$

$\epsilon\text{-closure}(2) = \{2, 3\}$

$\epsilon\text{-closure}(3) = \{3, 2\}$

$\epsilon\text{-closure}(q) = \hat{\delta}(q, \epsilon)$
 = Set of all states reachable from state q without reading any string.

eg::



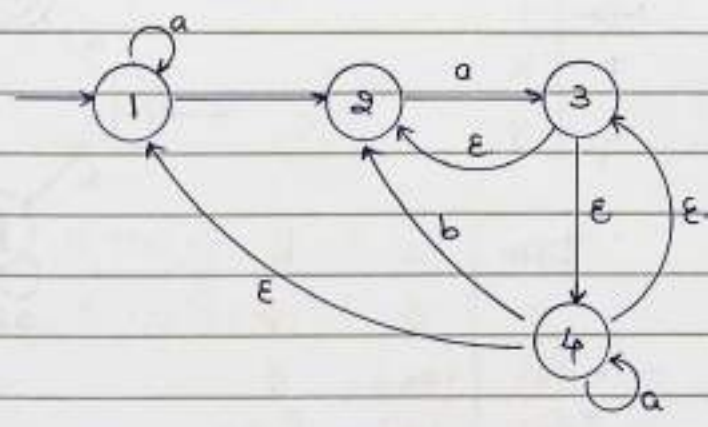
Transition Symbol/ ϵ

$\delta(1, \epsilon) = \{ \}$ From 1, there is no transition on ϵ .

$\hat{\delta}(1, \epsilon) = \{1\}$ From 1, there is path on ϵ .

Extended transition.

String



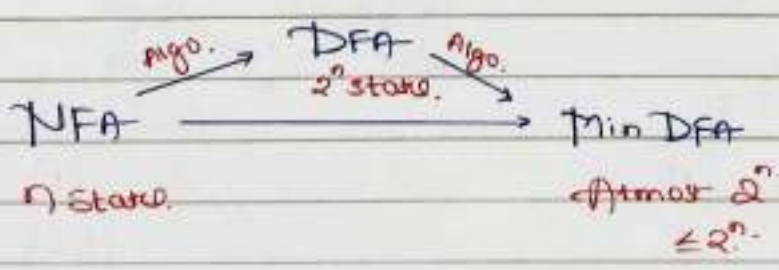
$\epsilon\text{-closure}(1) = \{1, 2\}$

$\epsilon\text{-closure}(2) = \{2\}$

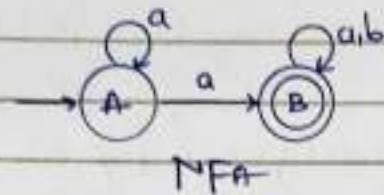
$\epsilon\text{-closure}(3) = \{3, 1, 2, 4\}$

$\epsilon\text{-closure}(4) = \{4, 1, 2, 3\}$

→ Algo: NFA without ϵ moves $\Rightarrow$ DFA
 (Subset Construction)



Given NFA $\longrightarrow$ DFA
 $(Q, \Sigma, \delta_{NFA}, q_0, F) \longrightarrow (2^Q, \Sigma, \delta_{DFA}, \{q_0\}, F)$



2 States

$$Q = \{A, B\}$$

Set of States of DFA = Set of all Subsets of Q.

$$: \phi, \{A\}, \{B\}, \{A, B\}$$

Initial States in DFA

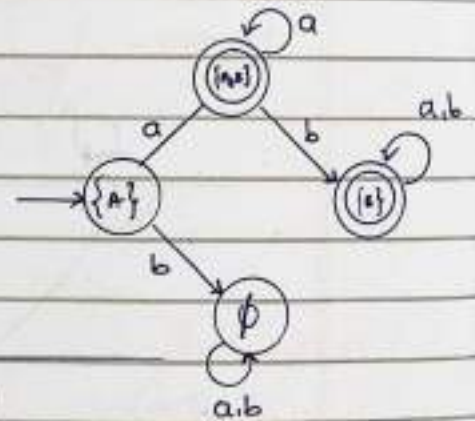
$$2^Q = \{ \phi, \{A\}, \{B\}, \{A, B\} \}$$

State of Q [State in DFA]
 Subsets

$$\delta_{DFA}(\{p, q\}, i) = \delta_{NFA}(p, i) \cup \delta_{NFA}(q, i)$$

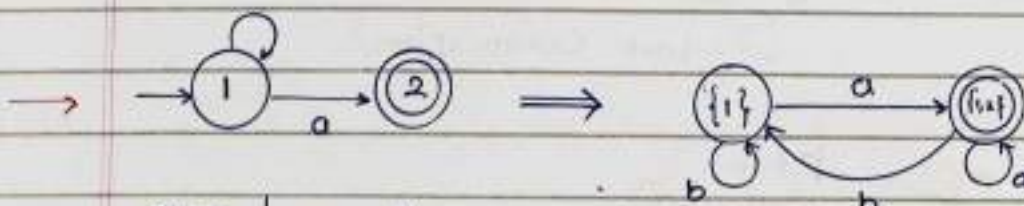
$$= X \cup Y$$

NFA	i
p	X
q	Y



NFA	a	b
$\rightarrow A$	$\{A, B\}$	ϕ
$* B$	$\{B\}$	$\{B\}$

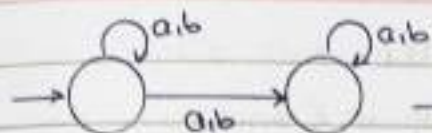
DFA	a	b
ϕ	ϕ	ϕ
$\rightarrow \{A\}$	$\{A, B\}$	ϕ
$* \{B\}$	$\{B\}$	$\{B\}$
$* \{A, B\}$	$\{A, B\}$	$\{B\}$



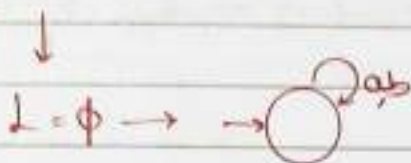
NFA	a	b
1	$\{1, 2\}$	$\{1\}$
2	ϕ	ϕ

	a	b
$\{1\}$	$\{1, 2\}$	$\{1\}$
$\{1, 2\}$	$\{1, 2\}$	$\{1\}$

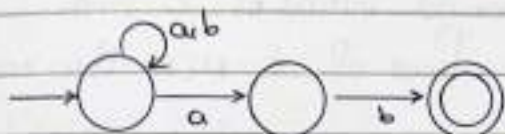
Q1.



No. of State in equivalent
Min DFA = 1.



Q2.



$L = (a+b)^* ab$

$\text{min} = ab$

$n(\text{min DFA}) = 3$

$\rightarrow$

NFA

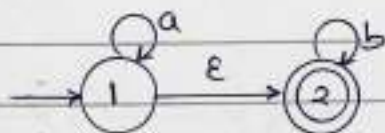
NFA

with ϵ moves

without ϵ moves

$(Q, \Sigma, \delta_{\text{NFA}}, q_0, F)$

$(Q, \Sigma, \delta_{\text{NFA}}, q_0, F')$



without ϵ moves

$\epsilon \cdot \text{clo}(1) = \{1, 2\}$

$\epsilon \cdot \text{clo}(2) = \{2\}$

$\rightarrow$

NFA $\xrightarrow{\text{Min}}$ DFA

If n state present in NFA then

$1 \leq$

no. of State in
min. DFA

$\leq 2^n$

Always important.

$\rightarrow$ Min NFA $\rightarrow$ Min DFA

$n \leq$

no. of State in
min. DFA

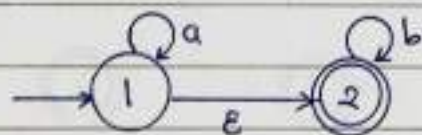
$\leq 2^n$

→ NFA $\xrightarrow{\text{with } \epsilon \text{ moves.}}$ NFA.
 $(Q, \Sigma, \delta_{E-NFA}, q_0, F)$ $\xrightarrow{\text{without } \epsilon \text{ moves.}}$ $(Q, \Sigma, \delta_{NFA}, q_0, F)$.

Note:

- I. Initial of NFA is same as initial of E-NFA.
 - II. If E-closure (q) contains final of E-NFA then make q as final in NFA.
 - III. $\delta_{NFA}(q, i) = \underbrace{\underbrace{\underbrace{\delta_{E-NFA} \left[\underbrace{E\text{-closure}(q), i \right]}_{1^{st}}}_{2^{nd}}}_{3^{rd}}}$
- $\forall q \in Q$
 $\forall i \in \Sigma$

NFA with ϵ moves $\xrightarrow{\quad} \text{DFA.}$
 $(Q, \Sigma, \delta_{E-NFA}, q_0, F) \xrightarrow{\quad} (Q^D, \Sigma, \delta_{DFA}, E\text{-clo}(q_0), F')$



$\delta(1, a) = \{1\}$	
$\delta(1, b) = \phi$	$\delta(2, a) = \phi$
$\delta(2, b) = \{2\}$	
$E\text{-clo}(1) = \{1, 2\}$	
$E\text{-clo}(2) = \{2\}$	

Note: I. Initial state of DFA = $E\text{-clo}(\text{Initial of NFA})$

II. $\delta_{DFA}(\{p, q\}, i) = E\text{-clo} \left[\delta_{E-NFA}(\{p, q\}, i) \right]$

III. Final of DFA: if any subset contains final of E-NFA of q .

NFA with ϵ -moves
or

NFA without ϵ -moves.

} $\Rightarrow$ No. of states in min.
DFA is at most 2^n .

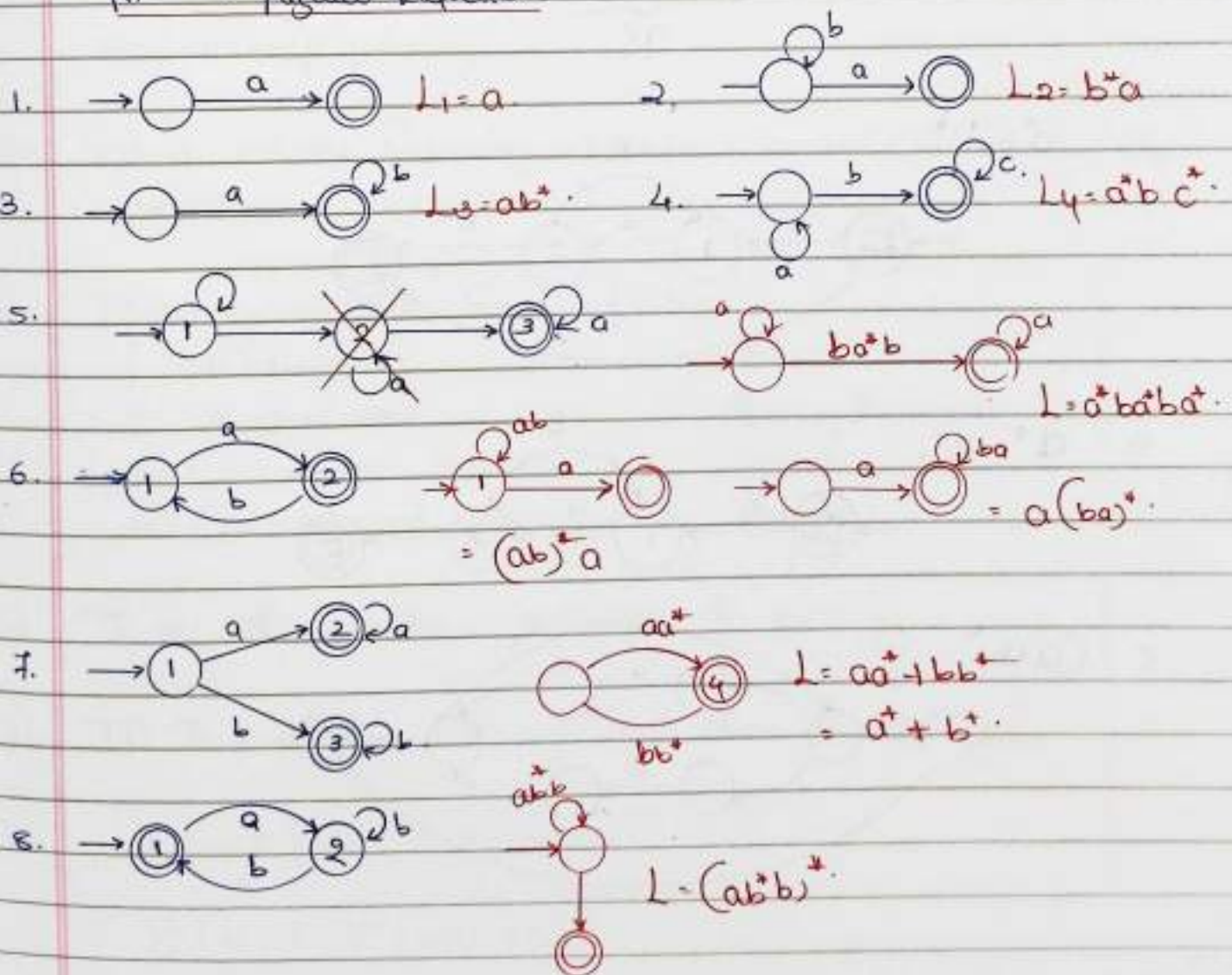
n States

Max no. of states in min. DFA
is 2^n

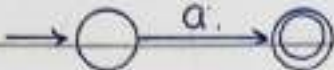
Regular Exp. $\equiv$ FA:

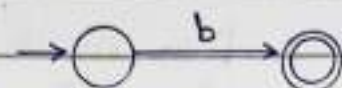
- I. FA $\Rightarrow$ Regular Expression [using state elimination]
- II. Regular Expression $\Rightarrow$ FA [Induction method].

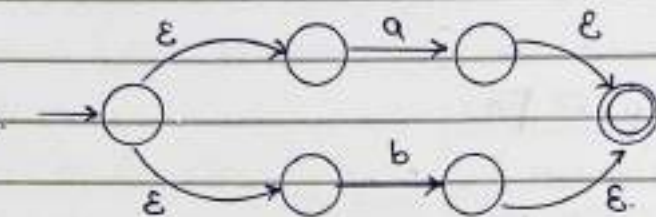
FA $\rightarrow$ Regular Expression

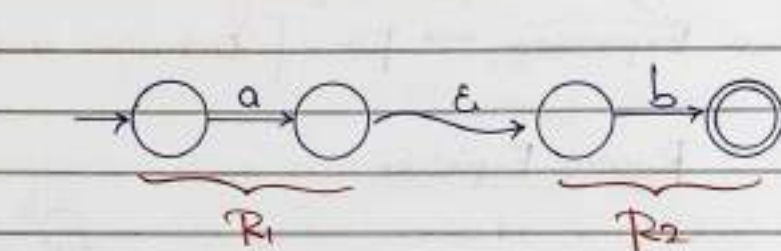


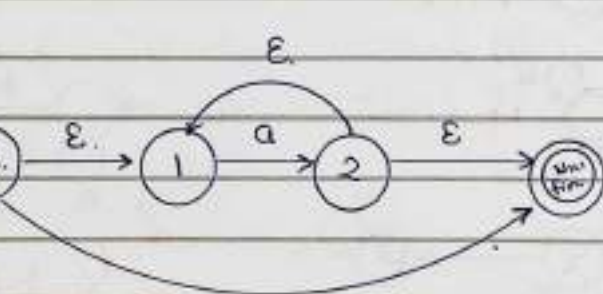
Regular Expression $\rightarrow$ F.A.

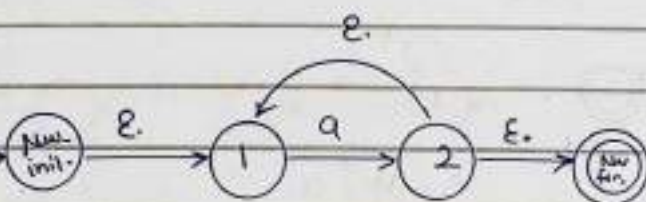
1. $R = a \rightarrow$ 

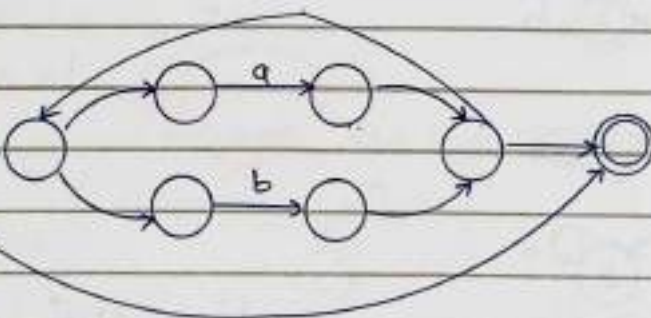
2. $R = b \rightarrow$ 

3. $R = a + b = R_1 + R_2 \rightarrow$ 

4. $R = a \cdot b = R_1 \cdot R_2 \rightarrow$ 

5. $a^+ = R^+$ 

6. $d^+ = R^+$ 

7. $(a+b)^+$ 

Q1. Number of prefixes of "n" length string is $n+1$

Q2. $((ab)^* \phi)$ is equivalent to ϕ

Q3. $(\phi^* \cup (bb^*)) = \underline{b^*}$

Q4. Which of the following is true?
 $(ab)^* a = a (ba)^*$

Q5. OR operator in regular expressions satisfies.
Associative and Commutative

Q6. Concatenation Operator in regular expressions satisfies.
Associative property

Q7. Which of the following distribution is valid in regular exp?
Concatenation over OR

Q8. Match groups over Sigma = $\{a, b\}$.

- | | | |
|----------------------------|---|------------------|
| 1. OR identity | → | Epsilon |
| 2. OR dominator | → | Empty Expression |
| 3. Concatenation identity | → | $(ab)^*$ |
| 4. Concatenation dominator | → | $(aa)^*$ |

Q9. If $R_1 = a^*$, and $R_2 = (aa)^*$ then $R_1 R_2 = \underline{R_1}$

Q10. If $R_1 = a(aa)^*$, and $R_2 = (aa)^*$ then $R_1 + R_2 = \underline{R_1}$

Q11. $L = \{w \in \{a, b\}^* : \#_a(w) \leq 3\}$

$b^* (aue) b^* (aue) b^* (aue) b^*$

Homework:

Find the min. string in the following expressions:

1. $(ab)^+aaa(b+ab) \rightarrow \text{min. } abaaab$

2. $(ab)^+aaa(b+\epsilon) \rightarrow aaa$

3. $(a+ab)^+(bb+aaa)^+(ab)^+ \rightarrow abbb$

4. $(ab)^+ + \epsilon \rightarrow \epsilon$

5. $(ab)(a+b)^+aaa+ba \rightarrow ba$

6. $[(ab^+ + a^+ + ba)ab]^+(a+b)^+ \rightarrow aab$

7. $[(ab)^+a)^+aaa]^+ \rightarrow aaa$

8. $[(aba)^+aba)^+ab]^+ \rightarrow \epsilon$

9. $(a^+)^+ \rightarrow \epsilon$

10. $((ab)^+a^+)^+bb^+aa^+ \rightarrow abbaa^+$

11. $[(ab)^+aba]^+aaa \rightarrow ababaaaa$

12. $(aaa^+ab^+)^+ \rightarrow aaab$

13. $(a+aaa+aa)^+ \rightarrow a$

14. $a^+b^+ \rightarrow ab$

15. $a^+b^+ \rightarrow \epsilon$

16. $a^+b^+c^+d^+ \rightarrow c$

17. $(a+b^+c^+)^+ \rightarrow \epsilon$

18. $(a+b+c)^+ \rightarrow \epsilon$

Q12. $L = \{w \in \{a,b\}^+ : \#a(w) \geq 3\}$
 $(aub)^+ a (aub)^+ a (aub)^+ a (aub)^+$

Q13. $L_1 = a^+b^+ \quad L_2 = a^+b^+$
 Find $L_2 - L_1 \rightarrow \text{None}$

Q14. $L_1 = a^+ + b^+ \quad \text{and} \quad L_2 = a^+b^+$
 Which of the following is true? $\rightarrow L_1^+ = L_2^+$

Q15. $L = a^+ + b^+ \quad \text{and} \quad L_2 = a^+b^+$
 Which of the following is true?
 L_1 is subset of L_2 .

Q16. $L_1 = a^+b^+$ and $L_2 = a^+b^+$

Which of the following is FALSE? $L_1^+ = L_2^+$

Q17. $L_1 = a^+ + b^+$ and $L_2 = a^+ + b^+$

Which of the following is TRUE? $L_1 \cup L_2 = L_2$

Q18. $L_1 = a^+$ and $L_2 = a^+$

Which of the following is TRUE? $L_1^+ = L_2^+$

Q19. $A = a^+$ and $B = b^+$

$AB = ? \quad \{a^m b^n \mid m, n \geq 0\}$

Q20. $A = aa^+$ and $B = bb^+$ $(A \cup B)^+ = ? \quad (a+bb)^+$

Q21. Given the language $L = \{ab, aba, baa\}$, which of the following strings are not in L^+ ?

baaaaaabaaaaab

Q22. The length of the shortest string NOT in the language (over $\Sigma = \{a, b\}$) of the following regular expression is $a^+(ba)^+a^+$ is 1

Q23. $L = a(ab)^+$ is equivalent to $(ab)^+$, $(a^+b^+)^+$, $a^+(ab^+)^+$

Q24. $L = (a+bb)^+b$ is equivalent to $b^+(ab^+)^+b$

Q25. $(b+ba)(b+ab)^+(ab+ab)$ $b(ab)^+b$

Q26. $\{w \in \{a, b\}^+ : \#a(w) = 3\}$
 $(b^+ab^+ab^+a)^+b^+$

Q27. $(a+ab)^+(a+ab)^+b^+ \rightarrow (a+ab)^+$

Q28. $L = \{w \in \{a,b\}^* \mid w \text{ has } bba \text{ as a substring}\}$
Which of the following describes L ?
 $(a+ b)^* bba (a+ b)^*$

Q29. $L = \{w \in \{a,b\}^*\}$
How many of below are equivalent to given L ?
 $(a+ b)^*$ $(a^* b^*)^*$
 $(a+ b+ \epsilon)^*$ $(b^* a^*)^*$
 $\epsilon + (a+ b)^*$ $\therefore$ 5 are equivalent.

GATE PYQs

1. Which two of following regular expressions are valid?

Ans. $(00)^* (\epsilon + 0) = 0^* \rightarrow 0^* \therefore (i) \text{ and } (iii)$

2. If the regular set A is represented by $A = (01+1)^*$ and regular set B is represented by $B = ((01)^* 1)^*$, which of the following is true?

Ans. $01 = x$ $A = (x+y)^*$
 $1 = y$ $B = (x^* y^*) \therefore A = B.$

3. The string 1101 does not belong to the set represented by

Ans. c. $(10)^* (01)^* (00+11)^*$
 d. $(00+ (11)^* 0)^*$

4. Let S and T be languages over $\Sigma = \{a,b\}$ represented by the regular expressions $(a+ b)^*$ and $(a^* + b)^*$ respectively. Which of the following is true?

Ans. $S \subseteq T$ $(a+ b)^* = (a^* + b)^*$
 $S \supseteq T$ $(a^* + b)^* = (a+ b)^* \therefore S = T$
 $T \subseteq S$

HW

5. Consider the set Σ^* of all strings over the alphabet $\Sigma = \{0, 1\}$. Σ^* with the Concatenation operator for strings.

6. The regular expressions $0^*(10^*)^+$ denote the same set as $(1^+0)^+1^+ = (0+1)^+$.

7. Which of the following languages over alphabet $\{0, 1\}$ is described by regular expression $(0+1)^+0(0+1)^+0(0+1)^+$
The set of all strings containing at least two 0's.

8. Consider the languages $L_1 = \phi$ and $L_2 = \{a\}$. Which one of the following represents $L_1 L_2^+ \cup L_1^+$?
 $\{\epsilon\}$

9. The length of the shortest string not in the language (over $\Sigma = \{a, b\}$) of the following regular expressions $a^+b^+(ba)^+a^+$.
3

10. Which of the following regular expressions represents the language: the set of all binary strings having two consecutive 0's and two consecutive 1's?
 $(0+1)^+(00(0+1)^+ \cup 11(0+1)^+00)(0+1)^+$

11. Let $r = 1(1+0)^*$, $s = 11^+0$ and $t = 1^+0$ be three regular expressions. Which one of the following is true?
 $L(s) \subseteq L(r)$ and $L(s) \subseteq L(t)$
 $L(s) \subseteq L(t)$ and $L(s) \subseteq L(r)$
 $S = \{1(1+0)^+\}$
 $S \subseteq \{11^+0\}$
 $S \subseteq \{1^+0\}$
 $t = 1^+0 = 0+1^+0$

12. Which of the following regular expressions identified are true?
 $(r^+s^+)^+ = (r+s)^+$

13. Which one of the following regular expressions represents the set of all binary strings with an odd number of 1's?

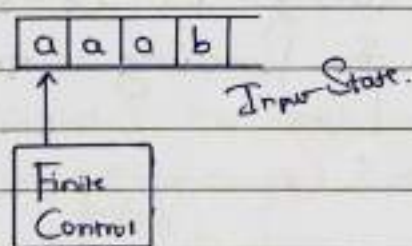
Ans. $0^*(0^*10^*10^*)^*0^*10^*$

14. Which one of the following regular expressions over $\{0,1\}$ denotes the set of all strings not containing 100 as a substring?

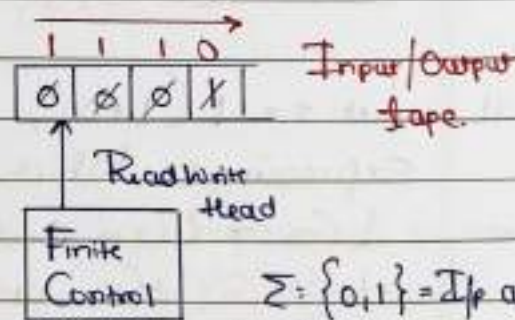
Ans. $0^*(10+1)^*$

Acceptor	Transducer
Input. $\rightarrow$ FA Accepted/Rejected.	Input. $\rightarrow$ FA $\rightarrow$ Output
FA without o/p. - NFA - DFA.	FA with o/p. - Moore t/c - Mealy t/c
Applications: Token recogniser, Spell checker.	Applications: Sequence detectors, Generators, Inc/dec, Addition... etc.

FA Configuration.

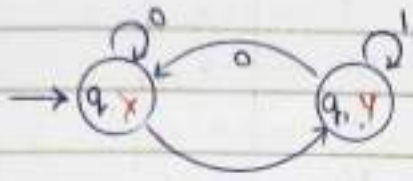
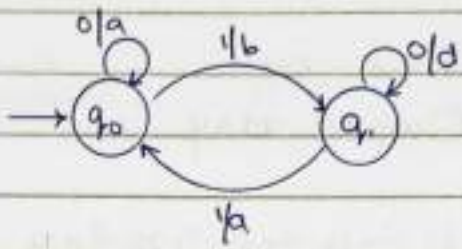


$FA = (Q, \Sigma, \delta, q_0, F)$
 $\delta DFA \quad Q \times \Sigma \rightarrow Q$
 $\delta NFA \quad Q \times \Sigma \rightarrow 2^Q$

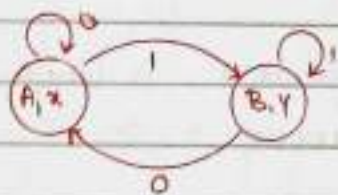


$\Sigma = \{0,1\} = \text{I/p alphabet}$
 $\Delta = \{0,1\} = \text{O/p alphabet}$

$FA = (Q, \Sigma, \delta, q_0, \Delta, \lambda)$
 $\delta Q \times \Sigma \rightarrow Q$
 $\lambda \quad \text{O/p function}$
 $\Delta \quad \text{O/p alphabet}$

Moore M/c	Mealy M/c.
1. $\lambda: Q \rightarrow \Delta$	$\lambda: Q \times \Sigma \rightarrow \Delta$
2. Output is associated with State.	Output is associated with transition.
3. 	
$\Delta = \{x, y\}$ $\lambda(q_0) = x$ $\lambda(q_1) = y$	$\lambda(q_0, 0) = a$ $\lambda(q_0, 1) = b$ $\lambda(q_1, 0) = d$ $\lambda(q_1, 1) = a$

Moore Machine



$\epsilon: A$
 x

$0: A \xrightarrow{0} A$
 $x \quad x$

$A \rightarrow B \rightarrow B$
 $x \quad y \quad y$

By default.

o/p associated with

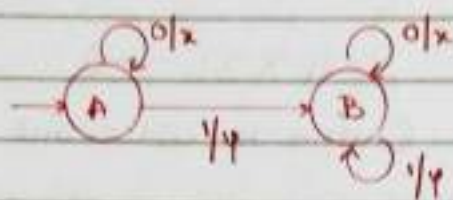
initial state will be produced.

Input.	Output
ϵ	x
0	xx
1	xy
00	xxx
01	$xxxy$
10	$xyxx$
11	$xyyy$

I. If n length input is given then output length for given input will be $n+1$

II. If n length input is given and every state is associated with 3 length output then output for the given i/p: 3^{n+1}

Mealy Machine:



Input: 0001

Output: xxxy

$A \xrightarrow{0/x} A \xrightarrow{0/x} A \xrightarrow{0/x} A \xrightarrow{1/y} B$

I/p	O/p
ϵ	nothing
0	x
1	y
00	xx
01	xy
10	yx
11	yy

I. If n length input given then Output length for the given input = n

II. If every transition is associated 3 length Output then Output length for n length input = $3n$

Moore M/c

Mealy M/c

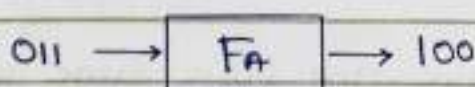
Common points
between both

$\delta: Q \times Z \rightarrow Q$

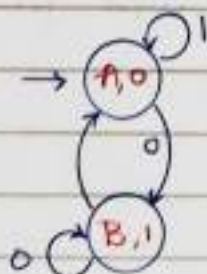
Both are DFA's

Final state are not there.

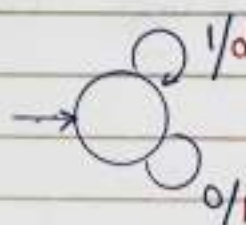
① 1's Complement



Moore M/c

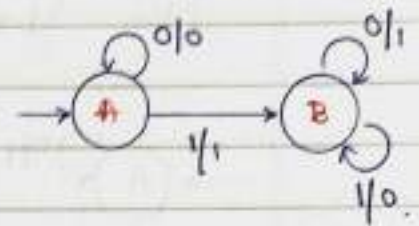
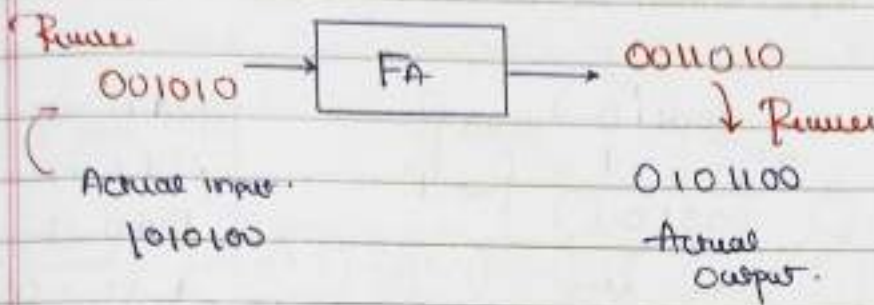


Mealy M/c

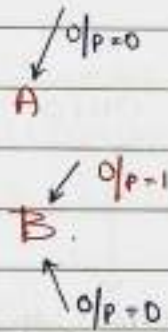
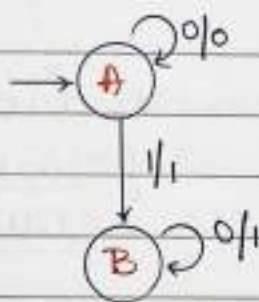


2) 2's Complement.

Mealy m/c



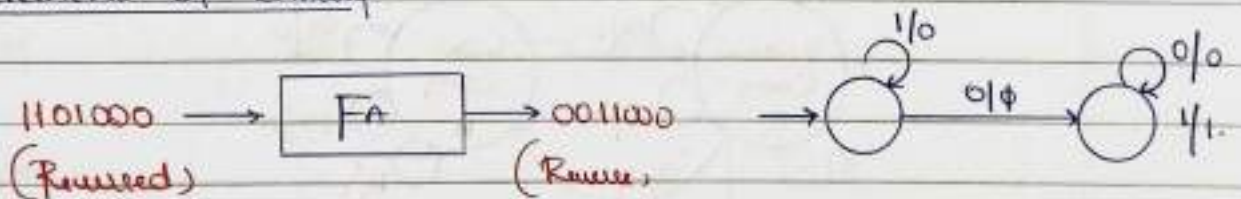
Mealy m/c → Moore m/c.



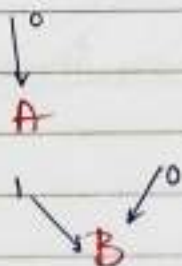
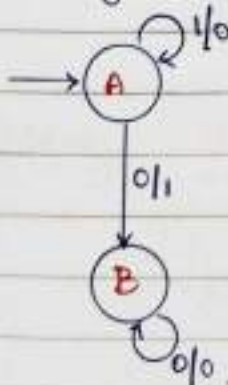
Find outputs which are incoming towards every state.



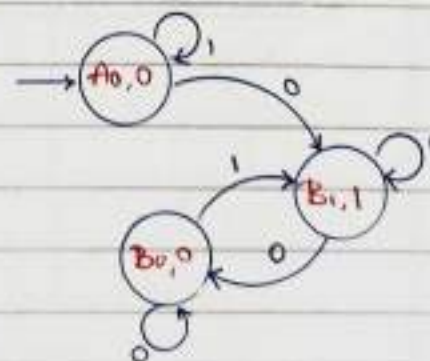
3) Increment of Binary



Mealy:

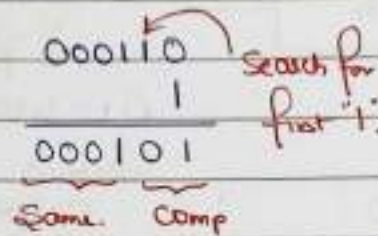
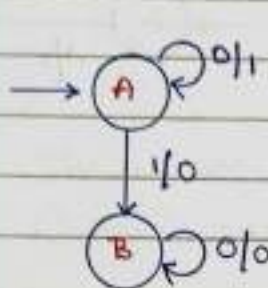


Moore m/c



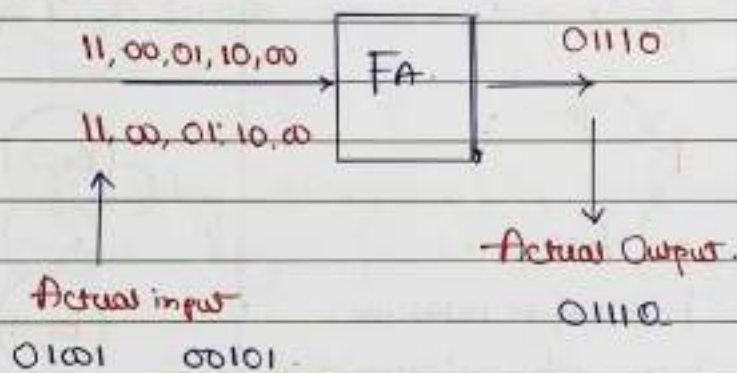
4. Decrement of Binary Number $f(x) = x - 1$

Note: Given input should be in reverse order.



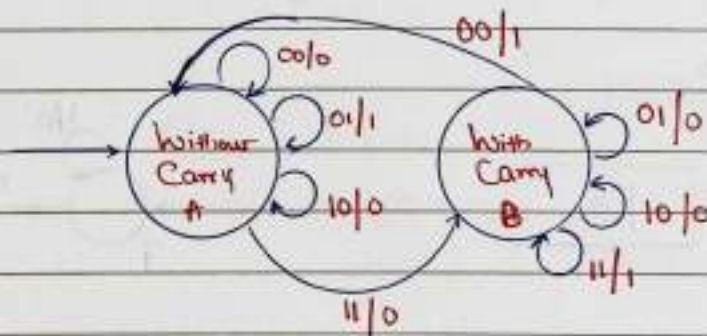
$$\begin{array}{l} 0 + 0 = 0 \\ 0 + 1 = 1 \\ 1 + 0 = 1 \\ 1 + 1 = 0 \text{ (carry)} \end{array}$$

5. Addition of 2 Binary Inputs



$$\begin{array}{r} x = 01001 \\ y = 00101 \\ \hline 01110 \end{array}$$

5th bit, 1st bit, carry



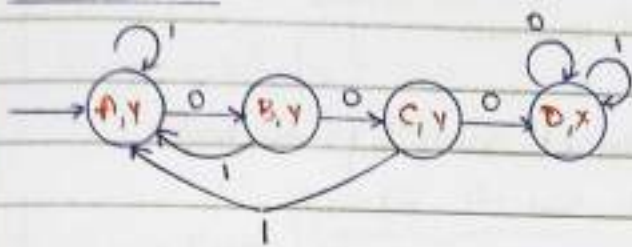
Two

6. Subtraction of two Binary Inputs

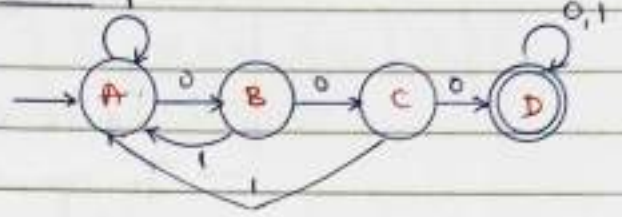
7. Sequence Detector

Detect '000' Sequence in the given binary input.

Moore M/C



DFA



Note: Containing '000' as Substring $\rightarrow$ DFA

8. If '000' present, produce x. Otherwise produce y.

I/p: 00100010011001

O/p: yyyyyxxxxxxx

8. Find the number of occurrences of '000' in the binary input.

I/p: 01001000010001110

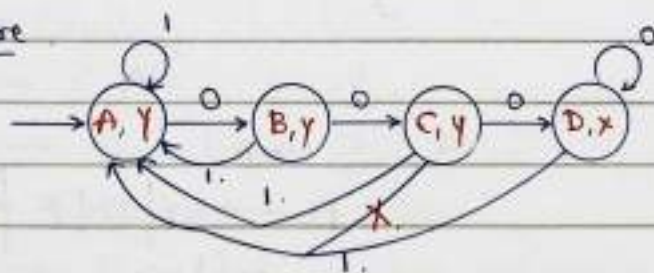
O/p: yyyyyyyxxxyyyxyyyy

3 times

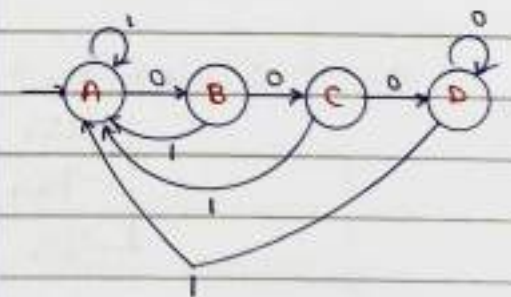
If '000' occurs produce 'x' otherwise produce 'y'.

Note: Ending with '000' $\rightarrow$ DFA

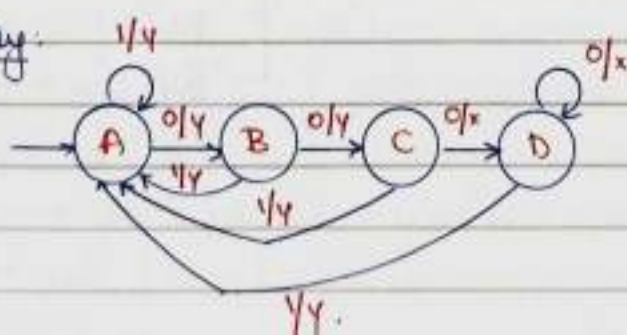
Moore



DFA

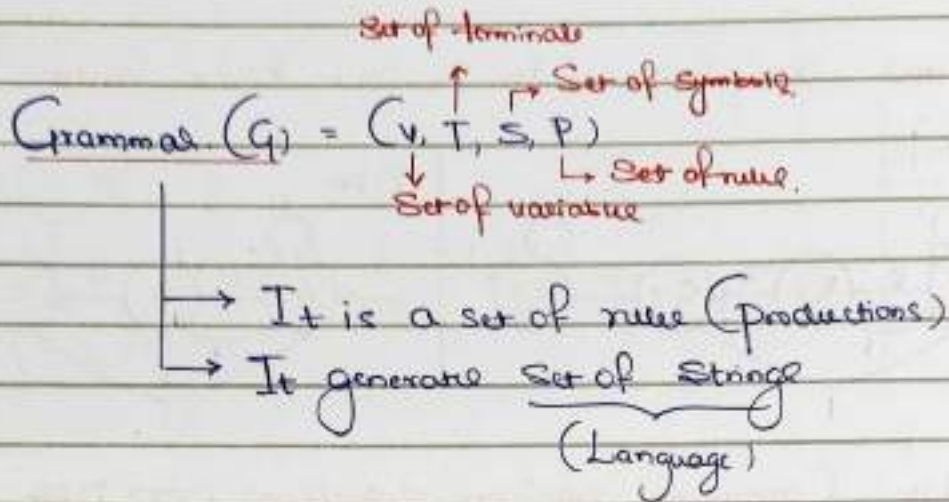


Mealy



HW

9. Produce the sum of present bit and previous bit of input.



Regular Grammar (RG)

It is left linear grammar (LLG) or Right Linear grammar (RLG)

LLG	RLG
$V \rightarrow VT^+ T$	$V \rightarrow T^+V T$
Example: $S \rightarrow Sab \epsilon ab$	Example: $S \rightarrow \epsilon a ab aabS$

Identify RG.

1. $\{ S \rightarrow \epsilon \}$

- LLG
- RLG
- RG

3. $\{ S \rightarrow A, A \rightarrow a | S | \epsilon \}$

- LLG
- RLG
- RG

2. $\{ S \rightarrow a | bc | \epsilon \}$

- LLG
- RG
- RLG

4. $\begin{cases} S \rightarrow Sa \\ S \rightarrow bs \\ S \rightarrow a \end{cases}$ neither LL or RL

5. $\{S \rightarrow Sa | b\}$
LL ✓
RL ✗
RG ✓

6. $\{S \rightarrow abS | a\}$
RL and RG

7. $\{S \rightarrow aSb | ab | \epsilon\}$
not RL

8. $\{S \rightarrow AaB | \epsilon\}$
not RL

9. $\{S \rightarrow AaB | \epsilon\}$
not RL

Find Regular Language from Following RLs.

1. $S \rightarrow \epsilon$ $L = \{\epsilon\}$

2. $S \rightarrow a | bb | \epsilon$ $L = \{\epsilon, a, b, b\}$

3. $S \rightarrow \epsilon | a | aa | aaa$ $L = \{\epsilon + a + aa + aaa\} = \{a^n | 0 \leq n \leq 3\}$

4. $S \rightarrow Aa$ $L = \{\} = \phi$

5. $S \rightarrow Aa | b$ $L = \{b\}$

6. $S \rightarrow Aa$ $A \rightarrow \epsilon$ $L = \{a\}$

7. $S \rightarrow Aa$ $A \rightarrow a | b | \epsilon$ $L = (\epsilon + a + b)a = a + aa + ba$

8. $S \rightarrow Aa | Bb$ $A \rightarrow c$ $B \rightarrow d$ $L = \{ca, db\}$

9. $S \rightarrow Aa | Bb$ $A \rightarrow \epsilon | a$ $B \rightarrow \epsilon | b$ $L = \{aa, a, bb, b\}$

10. $S \rightarrow Sa | b$ $L = ba^*$

11. $S \rightarrow Sa | \epsilon$ $L = a^*$

12. $S \rightarrow Saa | \epsilon$ $L = (aa)^* = \{a^{2n} | n \geq 0\} = \{a^n | n = \text{even}\}$

13. $S \rightarrow Sab | \epsilon$ $L = (ab)^*$

14. $S \rightarrow Sab | c$ $L = c(ab)^*$

15. $S \rightarrow aS | b$ $L = a^*b$

16. $S \rightarrow aS | a$ $L = a^*a = aa^* = a^+$

17. $S \rightarrow aaS | \epsilon$ $L = (aa)^*$

18. $S \rightarrow aaS | a$ $L = (aa)^*a = a(aa)^*$

19. $S \rightarrow abS | \epsilon$ $L = (ab)^*$

20. $S \rightarrow Sa \quad L = \phi$.
21. $S \rightarrow Sa | b | c \quad L = (b+c)a^* = ba^* + ca^*$.
22. $S \rightarrow abS | c | de | fgh \quad L = (ab)^*(c+de+fgh)$.
23. $S \rightarrow Sa | b | c \quad L = (b+c)a^*$.
24. $S \rightarrow Sa | \epsilon | b \quad L = (\epsilon+b)a^*$.
25. $S \rightarrow Sa | Sb | c \quad L = c(ab)^*$.
26. $S \rightarrow Sa | Sb | \epsilon \quad L = \epsilon(ab)^* = (ab)^*$.
27. $S \rightarrow aS | bS | \epsilon \quad L = (a+b)^*\epsilon = (a+b)^*$.
28. $S \rightarrow Sa | Sb | a \quad L = a(ab)^*$.
29. $S \rightarrow aS | bS | b \quad L = (a+b)^*b$.
30. $S \rightarrow aA \quad A \rightarrow Aa | Ab | \epsilon \quad L = (a+b)^*a$.
31. $S \rightarrow aA \quad A \rightarrow ab | bA | \epsilon \quad L = a(ab)^*$.
32. $S \rightarrow Sa | Sb | a \quad L = a(ab)^*$.
33. $S \rightarrow aS | bS | a \quad L = (a+b)^*a$.
34. $S \rightarrow Sa | Sb | A \quad A \rightarrow aa | ba \quad L = S(ab)^* = (a+ba)(ab)^*$.
35. $S \rightarrow Aa | Ab \quad A \rightarrow Ba \quad B \rightarrow Ba | Bb | \epsilon$
 $S \cdot L = A(ab)^*$
 $= (a+b)^*a(ab)^*$
36. $S \rightarrow aS | bS | cS | \epsilon \quad L = (a+b+c)^*$.
37. $S \rightarrow Aa | Ab | \epsilon \quad A \rightarrow a | b | \epsilon$
 $L = (a+b+\epsilon)a + (a+b+\epsilon)b + \epsilon$
 $= \{ \epsilon, a, b, aa, ab, ba, bb \}$
38. $S \rightarrow Sa | Sb | A \quad A \rightarrow Bab \quad B \rightarrow Ba | Bb | \epsilon$
 $L = S \cdot (ab)^* = A \cdot (ab)^* = (ab)^*ab(ab)^*$
39. $S \rightarrow aS | bS | A \quad A \rightarrow abB \quad B \rightarrow aB | bB | \epsilon$
 $L = (a+b)^*A = (a+b)^*ab(a+b)^*$
40. $S \rightarrow aA | bA \quad A \rightarrow aB | bB \quad B \rightarrow aB | bB | \epsilon$
 $S = (a+b)^*A = (a+b)^2(ab)^*$
41. $S \rightarrow Aa | Ab \quad A \rightarrow Ba | Bb \quad B \rightarrow Ba | Bb | \epsilon$
 $S = A(ab)^* = (a+b)^*(ab)^2 = (a+b)^2(ab)^*$
42. $S \rightarrow Sa | Sb | \epsilon | A \quad A \rightarrow Aa | Ab | \epsilon$
 $S = S(ab)^* = (\epsilon+a)(ab)^* = (a+b)^*(ab)^* = (a+b)^*$

43. $S \rightarrow Sa | Ab | c \quad A \rightarrow Sb | d$

$L = S(A+bb)^* = (db+cb)(a+bb)^*$

44. $S \rightarrow Saa | Ab \quad A \rightarrow Sb | Ac | e$

Step 1: $A \rightarrow Ac | Sb | e$

$A = (Sb+e)C^* = Sbc^* + ec^*$

Step 2: $S \rightarrow Saaa | Ab$

$S \rightarrow Saa | Sbc^*b | ec^*b \quad L = ec^*b(aa+bc^*b)^*$

45. $S \rightarrow Sa | Saa | e \quad L = a^* = (a+aa)^*$

46. $S \rightarrow Sa | Sb | a | b \quad L = (a+bb)^*$

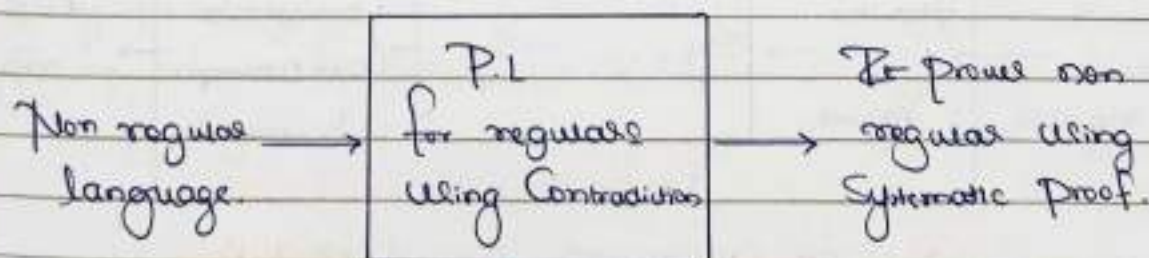
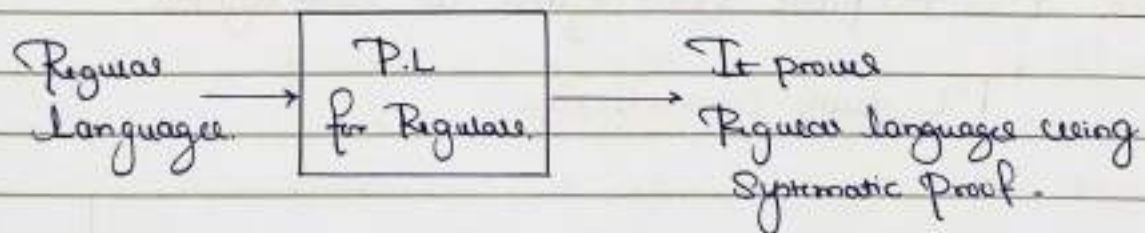
47. $S \rightarrow aS | bS | a | b | e \quad L = (a+bb)^*S = (a+bb)^*(a+bb+e)$

48. $S \rightarrow aS | bS | abS | e \quad L = (a+bb+ab)^* = (a+bb)^*$

49. $S \rightarrow Sa | Ab | a \quad A \rightarrow Ab \quad B \rightarrow Ba | Bb | e \quad L = S = a^+$

Pumping Lemma for Regular Languages:

- * It is a Lemma (proof).
 - * It satisfies regular languages.
 - * It can also be used to prove non regulars using Contradiction.
- Note: This pumping lemma can't be used to check unknown language is regular or not.



P.L for regular language.

P.L for non regular using contradiction.

Pumping Lemma for Non-regular using P.L for regular with Contradiction.

Step 1: Assume L is regular

Step 2: Choose P

Step 3: Select $w \in L$, $|w| \geq P$.

Step 4: $w = xyz$, $|x| \neq \emptyset$, $|xy| \leq P$

Step 5: $\forall i \ x y^i z \in L$ if L is regular

But if false, some, $xy^2 \notin L$

Step 6: Contradiction. So L is non-regular.

Assumption

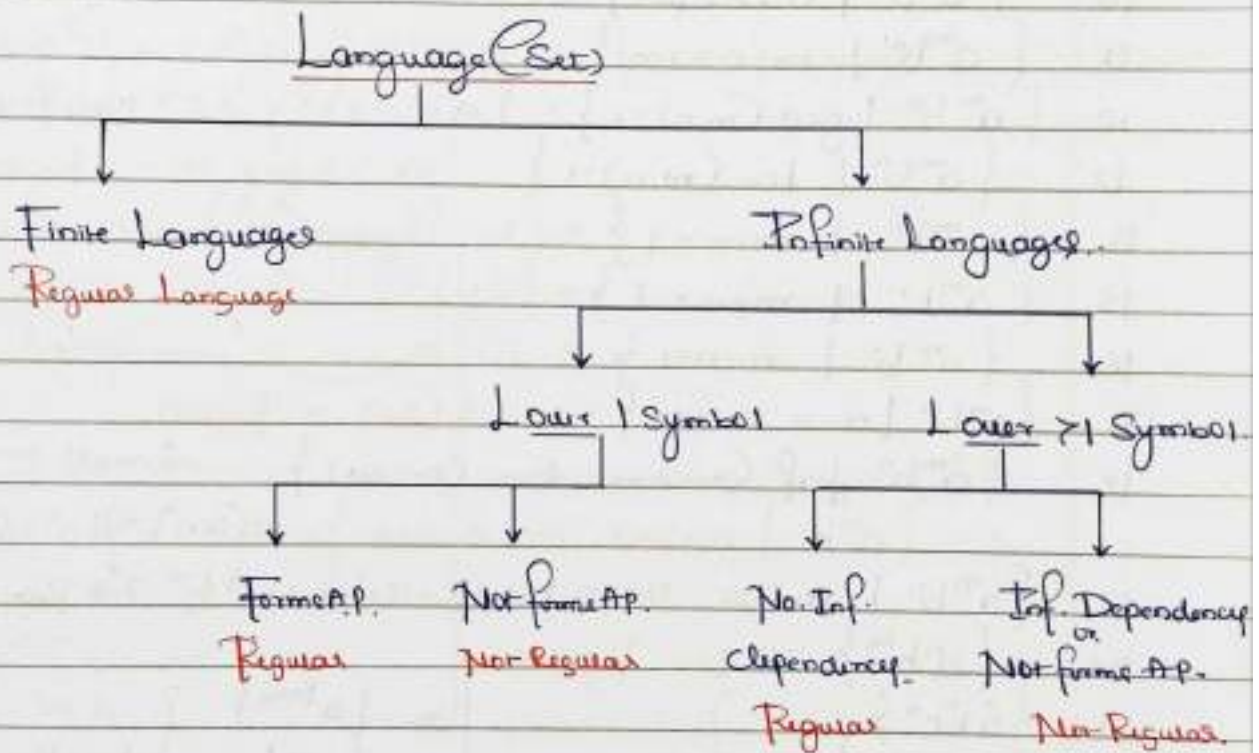
Choose P

$\exists$ String w

$\&$ Divide

Regular

Contradiction.



* Every finite language is Regular [TRUE]

* Every infinite language is Regular [FALSE]

Some infinite languages are regular and some are non-regular.

a^+
Regular

a^{Prime}
Not regular

Identify Regular and not Regular:

1. $\{a^m b^n\} = a^+ b^+ \rightarrow$ Regular.
2. $\{a^m b^n \mid m > n\} \rightarrow$ Not Regular language
3. $\{a^m b^n \mid m < n\} \rightarrow$ Not Regular language
4. $\{a^m b^n \mid m = \text{even}, n = \text{even}\} = (aa)^+ (bb)^+ \rightarrow$ Regular
5. $\{a^m b^n \mid m = n = \text{even}\} = \{a^{2n} b^{2n}\} \rightarrow$ Not Regular
6. $\{a^n b^n \mid n \geq 0\} = \{\epsilon, ab, aabb, aaabbb, \dots\} \rightarrow$ Not Regular
7. $\{a^n b^{n+1}\} \rightarrow$ Not Regular
8. $\{a^{n+2}, b^{n+100}\} \rightarrow$ Not Regular
9. $\{a^m b^n \mid m = n \text{ or } m \neq n\} = a^+ b^+ \rightarrow$ Regular
10. $\{a^m b^n \mid m < n < 100\} \rightarrow$ Finite Languages $\rightarrow$ Regular
11. $\{a^m b^n \mid m > n > 100\} \rightarrow$ Not Regular
12. $\{a^m b^n \mid \gcd(m, n) = 1\} = \{a^2 b^3, a^3 b^2, \dots\} \rightarrow$ Not Regular
13. $\{a^m b^n \mid \text{lcm}(m, n) = 1\} = \{a^1 b^1\} = \text{finite set} \rightarrow$ Regular.
14. $\{a^m b^n \mid m+n = 1\} = \{b, a\}$
15. $\{a^m b^n \mid m+n = 1\} = \{a^1 b^1\}$
16. $\{a^m b^n \mid m+n = 1\} = \{a, b\}$
17. $\{a^m b^n \mid m = 2, n = \text{odd}\} = aa b(bb)^+ =$ Regular
18. $\{a^m b^n \mid \text{if } (m = \text{even}) \text{ then } (n = \text{odd})\}$
 $= \{a^m b^n \mid m = \text{odd or } n = \text{odd}\} = a(aa)^+ b^+ + a^+ b(bb)^+ \rightarrow$ Regular
19. $\{a^m b^n \mid m < 100, n > 100\} = (\epsilon + a + a^2 + \dots) b^{101} b^+ \rightarrow$ Regular
20. $\{a^n b^{2n}\} \rightarrow$ Not Regular
21. $\{a^n b^n c^n\}$
22. $\{a^n b^{2n} c^{3n}\}$
23. $\{a^n b^{n^2} c^{n^3}\}$
24. $\{a^n b^n c^+\}$
25. $\{a^n b^+ c^n\}$
26. $\{a^n b^{n^2} c^{2n}\}$
27. $\{a^n \mid n \geq 0\} = a^+ \rightarrow$ Regular.
28. $\{a^{2n}\} = (aa)^+ \rightarrow$ Regular.
29. $\{a^{2n+100}\}$
30. $\{a^{\text{Prime}}\}$
31. $\{a^{n!}\}$
32. $\{a^{2^n}\}$
33. $\{a^{n^2}\}$
34. $\{a^{100n}\}$
35. $\{a^{n^{45}}\}$
36. $\{a^{100n}\}$
37. $\{a^{2024n^{11}}\}$
38. $\{a^{100n}\} \cup \{b^{100n}\} \rightarrow$ Regular.

- | | |
|---|---|
| <p>39. $\{a^{2n}\}^+ \}$ Regular</p> <p>40. $\{a^{Prime}\}^+$</p> <p>41. $\{a^n\} = \{a, a^2, \dots\} \rightarrow$ Not Regular</p> <p>42. $\{a^{m^n}\} = \{\epsilon, a, a^4, \dots\} \rightarrow$ Regular</p> <p>43. $\{a^{2^n}\}^+ = a^+$</p> <p>44. $\{a^{n^2}\}^+ = a^+$ } Equal.</p> <p>45. $\{a^{n^n}\}^+ = a^+$ } Regular.</p> <p>46. $\{a^{m^n}\}^+ = a^+$</p> | <p>47. $\{a^+ \cdot b^{Prime}\} \rightarrow$ Not Regular</p> <p>48. $\{a^{2m} \cdot b^{3m}\} \rightarrow$ Regular</p> <p>49. $\{a^{2n} \cdot b^{m^2}\}$</p> <p>50. $\{a^{n!} \cdot b^{Prime}\}$ } Not Regular</p> <p>51. $\{a^{n^2} \cdot b^{2m}\}$</p> <p>52. $\{a^n \cdot b^{n^2}\}$ } Not Regular</p> <p>53. $\{a^{n^2} \cdot b^{2^n}\}$</p> <p>54. $\{a^n \cdot b^{n!}\}$</p> |
|---|---|
-
- | | |
|--|---|
| <p>56. $\{ww \mid w \in a^+\} = a^{2n} \rightarrow$ Regular</p> <p>57. $\{w \# w \mid w \in a^+\} = a^+ \# a^+ \rightarrow$ Not R.</p> <p>58. $\{ww^R \mid w \in a^+\} =$ 56, Regular.</p> <p>59. $\{w \# w^R \mid w \in a^+\} =$ 57, Non-R.</p> <p>60. $\{ww \mid w \in \{ab\}^+\} \rightarrow$ Non Reg.</p> | <p>61. $\{w \# w \mid w \in \{a, b\}^+\}$ } Not</p> <p>62. $\{ww^R \mid w \in \{a, b\}^+\}$ } Regular</p> <p>63. $\{w \# w^R \mid w \in \{a, b\}^+\}$</p> <p>64. $\{w_1 w_2 \mid w_1, w_2 \in a^+\} = a^+ \rightarrow$ Regular.</p> |
|--|---|

What is closure property?

(D, o) is closed.

↳ operation

↳ domain.

if.

$$\forall x_1, x_2 \in D \exists x_1, o x_2 \in D$$

even 2 elements in D.

(D, o) is not closed.

if.

$$\exists x_1, x_2 \in D \exists x_1, o x_2 \notin D$$

Some 2 elements in D.

For Finite Languages, which of the following are closed?

I) Union $\rightarrow$ Closed

II) Intersection

Finite, $\cup$ Finite $\rightarrow$ Finite

Finite, $\cap$ Finite $\rightarrow$ Finite

Domain: Set of finite languages.

	Union	Intersection	Complement
For Finite Language.	<u>closed</u>	<u>closed</u>	<u>not closed</u>
	$F_1 \cup F_2 = \text{Finite}$	$F_1 \cap F_2 = \text{Finite}$	$\overline{F_2} = \text{Infinite}$
For Infinite Language.	<u>closed</u>	<u>not closed</u>	<u>not closed</u>
	$\text{Inf}_1 \cup \text{Inf}_2 = \text{Inf.}$	$\text{Inf}_1 \cap \text{Inf}_2 = \text{either inf or finite.}$	$\overline{I} = \text{either finite or infinite.}$

→ $\text{Inf}_1 \cap \text{Inf}_2 = \text{either finite or infinite}$
(need not be infinite)

Example 1: $a_{\text{inf}}^* \cap a_{\text{inf}}^* = a_{\text{inf}}^*$

Example 2: $a_{\text{inf}}^* \cap b_{\text{inf}}^* = \{\epsilon\}$ finite lang
 $\{ \epsilon, \dots \} \quad \{ \epsilon, \dots \}$

1. Complement of infinite language is either finite or infinite
2. Complement of finite language is always infinite
3. Complement for infinite language is not closed
4. Complement for infinite language is not closed

Types of Domains:

- Set of finite languages
- Set of infinite languages
- " " regular "
- " " not regular "
- " " languages.

Operation	finite languages	infinite languages
Union	✓	✓
Intersection	✓	✗
Compliment	✗	✗
Difference	✓	✗
Concatenation	✓	✓
Reversal	✓	✓
Kleene star	✗	✓
Kleene plus	✗	✓
Subset	✓	✗
Symmetric Difference	✓	✗
$Dual(L) = \overline{L}$ ^{R/L}	✗	✓

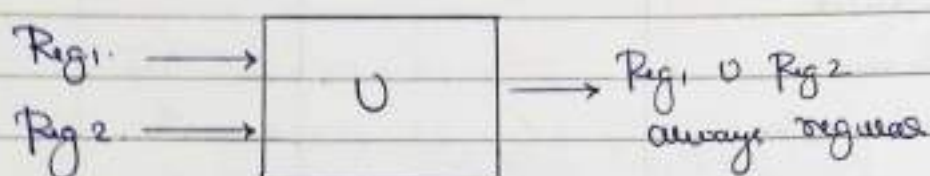
Closure properties for regular languages:

1. Union	16. ϵ -free homomorphism	
2. Intersection	17. Reverse homomorphism	
3. Compliment	18. Quotient	
4. Difference	19. $\frac{1}{2}(L) = \text{half}(L)$	30. Infinite $\cup$
5. Concatenation	20. Second half (L)	31. Infinite $\cap$
6. Reversal	21. One-third $(L) = \frac{1}{3}(L)$	32. Infinite $-$
7. Kleene plus	22. Tiddle $\frac{1}{3}(L)$	33. Infinite $\cdot$
8. Kleene star	23. Last $\frac{1}{3}(L)$	34. Infinite $\subseteq$
9. Symmetric diff.	24. Finite $\cup$	35. Infinite $\cap$
10. Subset. ✗	25. Finite $\cap$	
11. Prefix	26. Finite $-$	
12. Suffix	27. Finite $\cdot$	
13. Substring	28. Finite $\subseteq$	
14. Substitution	29. Finite f .	
15. Homomorphism.		

① Union for regular language.

└ Closed.

$\text{Reg}_1 \cup \text{Reg}_2 \rightarrow \text{Always Regular}$



Proof:

$\text{Reg. Lang} \cup \text{Reg. Lang} \rightarrow \text{Reg. Lang}$

Reg exp 1.

Reg exp 2.

Add '+' in between

Reg exp 1 + Reg exp 2.

"new Reg exp."

eg.

$$L_1 = \phi, L_2 = \Sigma^+ \rightarrow L_1 \cup L_2 = \Sigma^+$$

$$L_1 = a^*, L_2 = (aa)^+ \rightarrow L_1 \cup L_2 = a^+$$

$$L_1 = \text{Any}, L_2 = \phi \rightarrow L_1 \cup L_2 = L_1$$

$$L_1 = \Sigma^+, L_2 = \text{Any} \rightarrow L_1 \cup L_2 = \Sigma^+$$

$$L_1 = a^+b^+, L_2 = a^+ \rightarrow L_1 \cup L_2 = L_1$$

$$L_1 = a(atb)^+, L_2 = a^+b^+ \rightarrow L_1 \cup L_2 = L_1$$

$\text{Reg} \cup \text{Non Regular} \rightarrow \text{Either Reg or non-regular}$

Case 1: $\Sigma^+ \cup \text{Any non-regular} \rightarrow \text{Regular}$

Case 2: $\phi \cup \text{Any non-regular} \rightarrow \text{Non regular}$

$\text{Non Regular} \cup \text{Non Regular} \rightarrow \text{Either regular or non-regular}$

Case 1: $a^+b^+ \cup \overline{a^+b^+} \rightarrow (a^+b^+)$

Regular.

Case 2: $a^i b^j \cup a^i b^j \rightarrow a^i b^j$

Not regular.

111

1. Reg. Language $\cup$ Reg. Language $\rightarrow$ Regular
2. Regular Language $\cup$ Non Regular Language $\rightarrow$ either reg / non regular
3. Non regular Language $\cup$ Non regular Language $\rightarrow$ either reg / non regular
4. If $L_1 \cup L_2$ is Regular then L_1 is Either reg or non regular.

Reg $\cup L_2 \rightarrow$ Regular possible

Non regular $\cup L_2 \rightarrow$ Regular possible.

5. If $L_1 \cup L_2$ is Not Regular then L_1 is Either regular / not regular.

Reg $\cup L_1 \rightarrow$ Not regular possible

Not Regular $\cup L_2 \rightarrow$ Not regular possible

② Intersection for regular language

$\rightarrow$ closed.

Reg₁ $\cap$ Reg₂ $\rightarrow$ Always Regular.

- (i) $\phi \cap L \rightarrow \{\} \cap L \rightarrow \{\} \rightarrow \phi$
- (ii) $\Sigma^* \cap L \rightarrow$ Set of all strings $\cap L \rightarrow L$
- (iii) $a^+ \cap b^+ \rightarrow \{\epsilon, a, a^2, \dots\} \cap \{\epsilon, b, b^2, \dots\} \rightarrow \{\epsilon\}$
- (iv) $a^+ \cap b^+ \rightarrow \phi$
- (v) $a^+ b^+ \cap a^+ \rightarrow a^+$

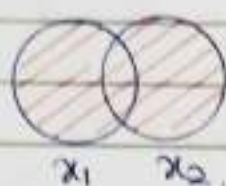
I. If L_1 is Regular and L_2 is Regular then $L_1 \cap L_2$ is Regular

II. If L_1 is Regular and L_2 is not regular then $L_1 \cap L_2$ is Either regular or non regular.

III. If L_1 is not regular and L_2 is not regular then $L_1 \cap L_2$ is Either regular or not regular.

IV. If $L_1 \cap L_2$ is regular then L_1 is Either regular or not regular.

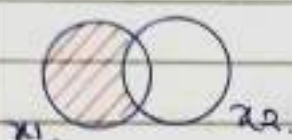
V. If $L_1 \cap L_2$ is not regular then L_2 is Either reg or non regular.



$$x_1 \cup x_2 = \{x \mid x \in x_1, \text{ or } x \in x_2\}$$



$$x_1 \cap x_2 = \{x \mid x \in x_1 \text{ and } x \in x_2\}$$



$$x_1 - x_2 = \{x \mid x \in x_1 \text{ and } x \notin x_2\}$$

x is only in x_1 .

If $x_1 \cap x_2 = \phi$ then x_1 and x_2 are Disjoint sets.

③ Complement for Regular language

→ Closed

Regular → Always Regular

- I. L is Regular iff $\bar{L}$ is Regular
- II. L is not regular if $\bar{L}$ is not regular.
- III. Not regular → Not regular

- i. $L = \phi \rightarrow \bar{L} = \Sigma^+$
- ii. $L = \Sigma^+ \rightarrow \bar{L} = \phi$
- iii. $L = a\Sigma^+ \rightarrow \bar{L} = \epsilon + b\Sigma^+$
- iv. $L = b\Sigma^+ \rightarrow \bar{L} = \epsilon + a\Sigma^+$
- v. $L = \Sigma^+a \rightarrow \bar{L} = \epsilon + \Sigma^+b$
- vi. $L = \Sigma^+a\Sigma^+ \rightarrow \bar{L} = b^+$
- vii. $L = \Sigma^+b\Sigma^+ \rightarrow \bar{L} = a^+ + \epsilon$

Proof:

L is regular.



Design DFA



Modified DFA



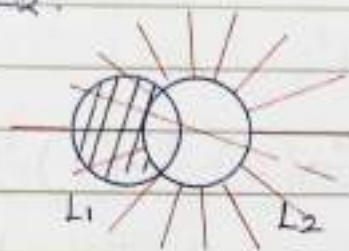
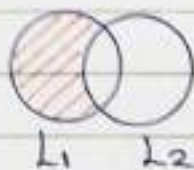
$\bar{L}$ is regular.

④ Difference for regular language.

→ Closed.

$\text{Reg}_1 - \text{Reg}_2 \rightarrow \text{Always Regular.}$

Def I $L_1 - L_2 = L_1 \cap \bar{L}_2$



$$a^+ \cdot b^+ \rightarrow a^+$$

Def II $L_1 - L_2 = L_1 - (L_1 \cap L_2)$

i. $L_1 = \phi, L_2 = \text{Any} \rightarrow L_1 - L_2 = \phi, L_2 - L_1 = L_2$

ii. $L_1 = \Sigma^+, L_2 = \text{Any} \rightarrow L_1 - L_2 = \Sigma^+ - L_2 = \bar{L}_2$

$$L_2 - L_1 = L_2 - \Sigma^+ = \phi$$

iii. $L_1 = a^+, L_2 = b^+ \rightarrow L_1 - L_2 = a^+ - b^+ \rightarrow a^+$

$$L_2 - L_1 = b^+ - a^+ \rightarrow b^+$$

iv. $L_1 = a^+, L_2 = (aa)^+ \rightarrow L_1 - L_2 = a(aa)^+$

$$L_2 - L_1 = \phi$$

⑤ Concatenation for regular languages.

→ Closed.

$L_1 \cdot L_2 \rightarrow \text{Always Regular.}$

reg. reg.

(i) $L_1 = \phi, L_2 = \text{Any} \rightarrow L_1 \cdot L_2 = \phi, L_2 \cdot L_1 = \phi$

(ii) $L_1 = \Sigma^+, L_2 = \{a\} \rightarrow L_1 \cdot L_2 = \Sigma^+ a, L_2 \cdot L_1 = a \Sigma^+$

(iii) $L_1 = a^+, L_2 = b^+ \rightarrow L_1 \cdot L_2 = a^+ b^+, L_2 \cdot L_1 = b^+ a^+$

(iv) $L_1 = a^+, L_2 = (aa)^+ \rightarrow L_1 \cdot L_2 = L_1, L_2 \cdot L_1 = L_1$

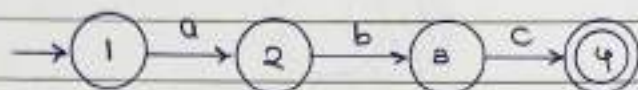
(v) $L_1 = \Sigma^+, L_2 = a \Sigma^+ \rightarrow L_1 \cdot L_2 = \Sigma^+ a \Sigma^+, L_2 \cdot L_1 = L_2$

⑥ Regular Language Reversal / Reversal for regular language

IF L is Regular, $L^{Rev} = \text{Regular}$

(Reg)^{Reverse} = Regular

eg: $L = \{abc\}$



$L^{Rev} = \{cba\}$

Step 1: make 1 as final

Step 2: Initial $\leftrightarrow$ Final

Step 3: Reverse all transitions



i. $L = \phi \rightarrow L^{Rev} = \phi = L$

ii. $L = \Sigma^+ \rightarrow L^{Rev} = \Sigma^+ = L$

iii. $L = a\Sigma^+ \rightarrow L^{Rev} = \Sigma^+a$

iv. $L = \Sigma^+b \rightarrow L^{Rev} = b\Sigma^+$

v. $L = ab^+ \rightarrow L^{Rev} = b^+a$

vi. $L = \Sigma^+a\Sigma^+ \rightarrow L^{Rev} = \Sigma^+a\Sigma^+$

vii. $L = a^+b^+ \rightarrow L^{Rev} = b^+a^+$

viii. $(L^{Rev})^{Rev} = L$

ix. $(\overline{L})^{Rev} = \overline{L^{Rev}}$

I. Reversal of regular language is Regular

II. Reversal of non-regular language is non-regular

Note: * L is Regular if L is regular

* L is Regular if L^{Rev} is regular

⑦ Kleene Star (Kleene closure)

IF L is Reg. L^+ is Regular

Note: * IF L^+ is regular then L is either reg/non regular

* IF L^+ is regular then L is either reg/non regular.

⑧ Kleene Plus (positive closure)

IF L is Reg. L^+ is Regular

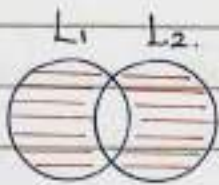
$\left\{ \begin{array}{l} \{a^{Prime}\}^+ \text{ is Regular} \\ \text{but } \{a^{Prime}\} \text{ is not regular} \end{array} \right.$

- | | |
|---|--|
| i. $L = \phi \rightarrow L^+ = \{\epsilon\}$
$L^+ = \phi - 1$ | v. $L = a^+b^+ \rightarrow L^+ = (a+b)^+ \cdot L^+ \cdot (a+b)^+$ |
| ii. $L = \Sigma^+ \rightarrow L^+ = \Sigma^+ - 1$
$L^+ = \Sigma^+ - 1$ | vi. $L = b^+a^+ \rightarrow L^+ = (a+b)^+$
$L^+ = (a+b)^+$ |
| iii. $L = \{a, b\} \rightarrow L^+ = (a+b)^+$
$L^+ = (a+b)^+$ | vii. $L = a^+ + b^+ \rightarrow L^+ = (a+b)^+$
$L^+ = (a+b)^+$ |
| iv. $L = a^+ + b^+ \rightarrow L^+ = (a+b)^+$
$L^+ = (a+b)^+$ | viii. $L = a^+b^+ \rightarrow L^+ = (a^+b^+)^+ \neq (a+b)^+$
$L^+ = (a^+b^+)^+$ |
| | ix. $L = a + b^+ \rightarrow L^+ = (a+b)^+ \cdot L^+ \cdot (a+b)^+$ |

⑨ Symmetric Difference for regulars.
→ closed.

$$L_1 \oplus L_2 = L_1 \Delta L_2 = (L_1 - L_2) \cup (L_2 - L_1)$$

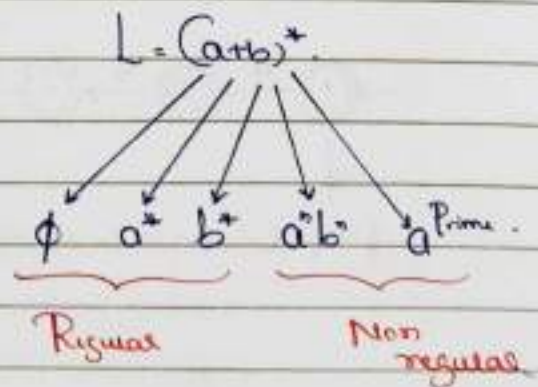
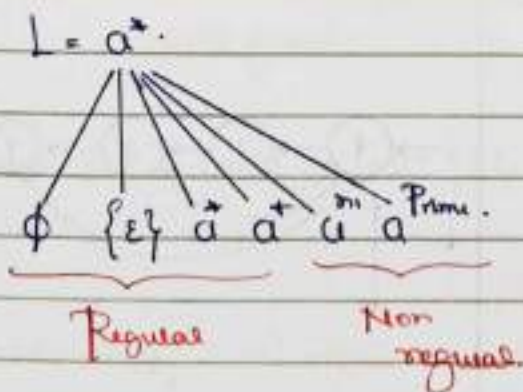
$$= (L_1 \cup L_2) - (L_1 \cap L_2)$$



eg: $L_1 = a^+ \left\{ \begin{array}{l} L_1 \Delta L_2 = a^+ + b^+ \\ L_2 = b^+ \end{array} \right. \quad \text{or} \quad L_1 = a\Sigma^+ \left\{ \begin{array}{l} L_1 \Delta L_2 = a\Sigma^+ + \\ L_2 = b\Sigma^+ \end{array} \right. b\Sigma^+$

⑩ Subset for Regulars.
→ Not closed.

Every Subset of a regular language is Either regular or not regular.



- classmate
Date _____
Page _____
- I. Every Subset of finite language is Finite Language (Regular)
 - II. Every Subset of infinite language is either finite or not infinite
 - III. Every Subset of regular language is either regular or not regular
 - IV. Every Subset of non regular language is either regular or non regular

For Language:

$$\text{prefix}(L) = \{u \mid w \in L, w = uv\} = \{u \mid uv \in L\}$$

$$\text{Suffix}(L) = \{v \mid uv \in L\}$$

$$\text{SubString}(L) = \{y \mid xy_2 \in L\}$$

For String:

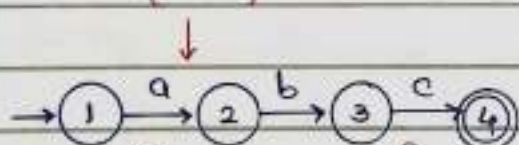
$$\text{prefix}(w) = \{u \mid uv = w\} \quad \text{prefix}(abc) = \{\epsilon, a, ab\}$$

$$\text{Suffix}(w) = \{v \mid uv = w\} \quad \text{Suffix}(abc) = \{\epsilon, b, \overset{ab}{bc}\}$$

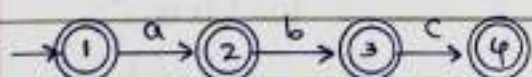
$$\text{SubString}(w) = \{y \mid xy_2 = w\} \quad \text{SubString}(abc) = \{\epsilon, a, b, abc\}$$

⑪ Prefix (Reg Language) is Regular

$$L = \{abc\}$$



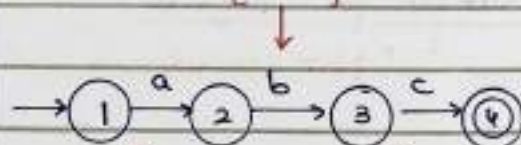
↓ from initial to final state path, make every state final.



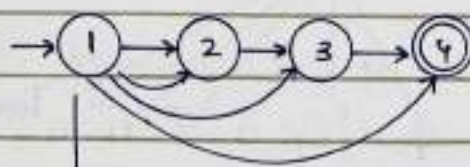
$$\text{prefix}(L) = \{\epsilon, a, ab, abc\}$$

⑫ Suffix (Regular) is Regular

$$L = \{abc\}$$



↓ from initial to final state path, add ϵ move, from initial to every state



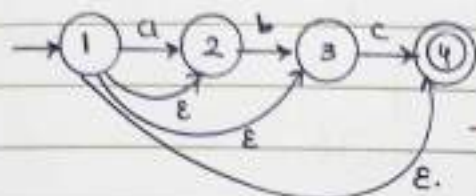
$$\text{suffix}(L) = \{\epsilon, c, abc, abc\}$$

(13) Substring (Reg) is Regular.

$L = \{abc\}$



Combine both prefix algorithm and suffix algorithm.

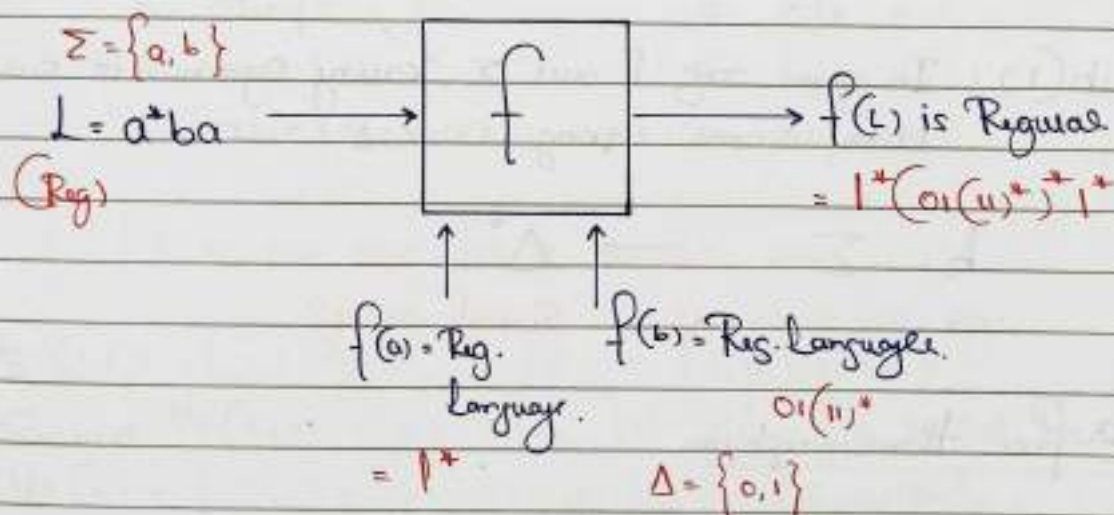


$\rightarrow \text{Substring}(L) = \{\epsilon, a, b, c, ab, bc, abc\}$

(14) Substitution for Regular Language.

$\rightarrow$ closed

Substitution of a regular language is Regular Language.
(Regular Substitution).



1. $L = (ab)^*$ and $f(a) = \{\epsilon\}$, $f(b) = 0^*$.

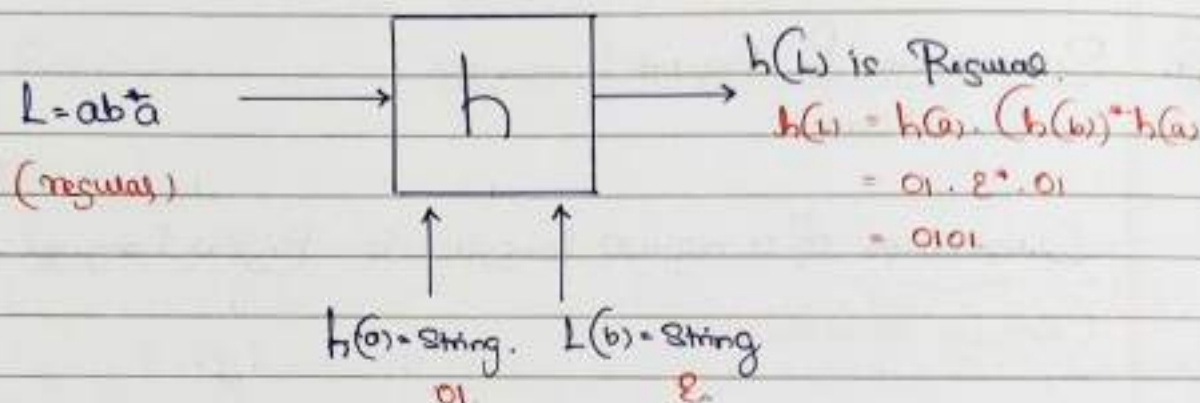
$$\begin{aligned} f(L) &= (f(a) \cdot f(b))^* \\ &= (\epsilon \cdot 0^*)^* \\ &= 0^* \end{aligned}$$

2. $L = a(ab)^*$, $f(a) = 0^*$, $f(b) = 1^*$.

$$\begin{aligned} f(w) &= f(a(ab)^*) \\ &= f(a) \cdot (f(a) + f(b))^* \\ &= 0^* (0^* + 1^*)^* \\ &= (0+1)^* \end{aligned}$$

Σ & Δ may or may not be same.

(15) Homomorphism
[String Substitution]



$h(L)$: In given reg L over Σ , every symbol is substituted with some string over Δ .

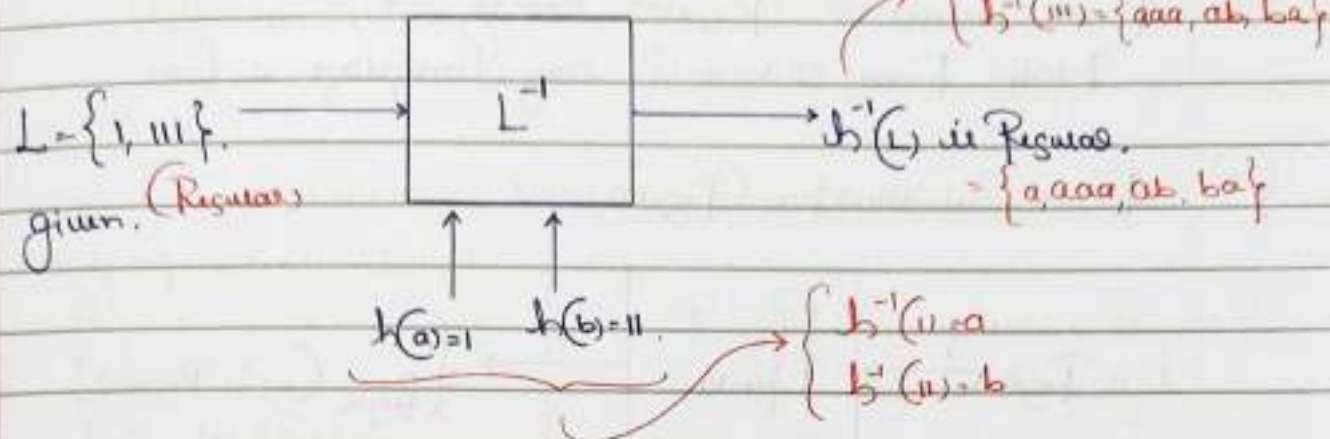
$$h: \underbrace{\Sigma}_{\text{Set of symbols}} \rightarrow \underbrace{\Delta^+}_{\text{Set of strings}}$$

(16) ϵ -free Homomorphism

$$h: \Sigma \rightarrow \Delta^+$$

In given reg L over Σ , every symbol is substituted with some non empty string (other than ϵ) over Δ .

17) Inverse Homomorphism for Regular Language $L \rightarrow \text{closed}$



18) Quotient (1)

$$L_1 / L_2 = \{u \mid uv \in L_1, v \in L_2\}$$

eg:: 1. $abc / \epsilon = abc$

2. $abc / a = \emptyset$

3. $abc / c = ab$

4. $abc / ab = \emptyset$

5. $abc / bc = a$

6. $abc / abc = \epsilon$

7. $\epsilon / abc = \emptyset$

8. $\epsilon / \epsilon = \epsilon$

9. $L / \epsilon = L$

10. $a / a = \epsilon$

$$L = \{\epsilon, a, ab, abc, abcd, abcda, abbbcd\}$$

eg:: 19) $\frac{1}{2}(L) = \{x \mid xy \in L, |x| = |y|\} = \{\epsilon, a, ab, abb\}$

20) Second $\frac{1}{2}(L) = \{y \mid xy \in L, |x| = |y|\} = \{\epsilon, b, cd, bcd\}$

21) $\frac{1}{3}(L) = \{x \mid xyz \in L, |x| = |y| = |z|\} = \{\epsilon, a, ab\}$

22) modulo $\frac{1}{3}(L) = \{y \mid \text{" " " "}\} = \{\epsilon, b, bb\}$

23) Last $\frac{1}{3}(L) = \{z \mid \text{" " " "}\} = \{\epsilon, c, cd\}$

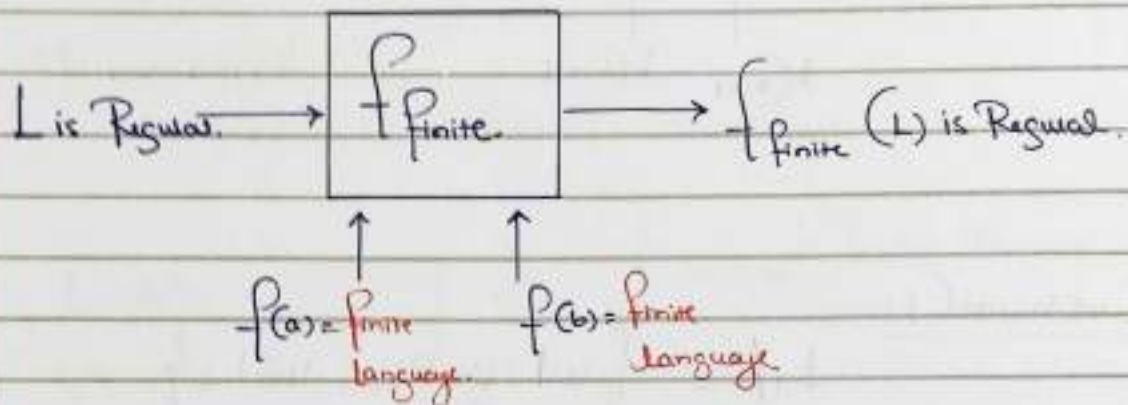
24) Finite union is closed for regular languages.

$$L_1 \cup L_2 \cup L_3 \dots \dots \dots \cup L_n \rightarrow \text{Regular}$$

25) Finite intersection is closed for regulars.

$$L_1 \cap L_2 \cap L_3 \dots \dots \dots \cap L_n \rightarrow \text{Regular}$$

- (26) $L_1 \cup L_2 \cup L_3 \dots \cup L_k \rightarrow \text{Regular}$
- (27) $L_1 \cdot L_2 \cdot L_3 \dots \cdot L_k \rightarrow \text{Regular}$
- (28) Finite Subset of Regular set is finite set (Regular)
 Note: Finite subset of any language is finite
- (29) Finite Substitution (Regular) $\rightarrow$ Regular.



- (30) $R_1 \cup R_2 \cup R_3 \dots \rightarrow \text{Either regular or non regular}$
- (31) $R_1 \cap R_2 \cap R_3 \dots \rightarrow "$
- (32) $R_1 - R_2 - R_3 \dots \rightarrow "$
- (33) $R_1 \cdot R_2 \cdot R_3 \dots \rightarrow "$
- (34) Infinite Subset of regular language $\rightarrow \text{Either regular or non regular}$
- (35) Infinite Subset of regular language $\rightarrow "$
 Substitution

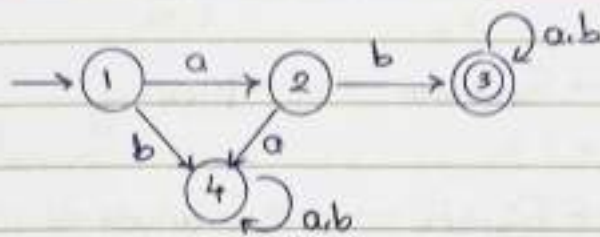
Equivalence Classes [Myhill - Nerode Equivalence Classes]

I. No. of state in min DFA for Reg = no. of equivalence classes for regular.

$\rightarrow$ Find no. of equivalence class for $L = ab^*(abb)^*$
Regular Language

$\rightarrow$ Step 1: Construct min DFA

Push Down Automata



→ Step 2: Each state represents one equivalence class.

= 4 equivalence classes

[1] = ϵ

[2] = a

[3] = ab (a+b)*

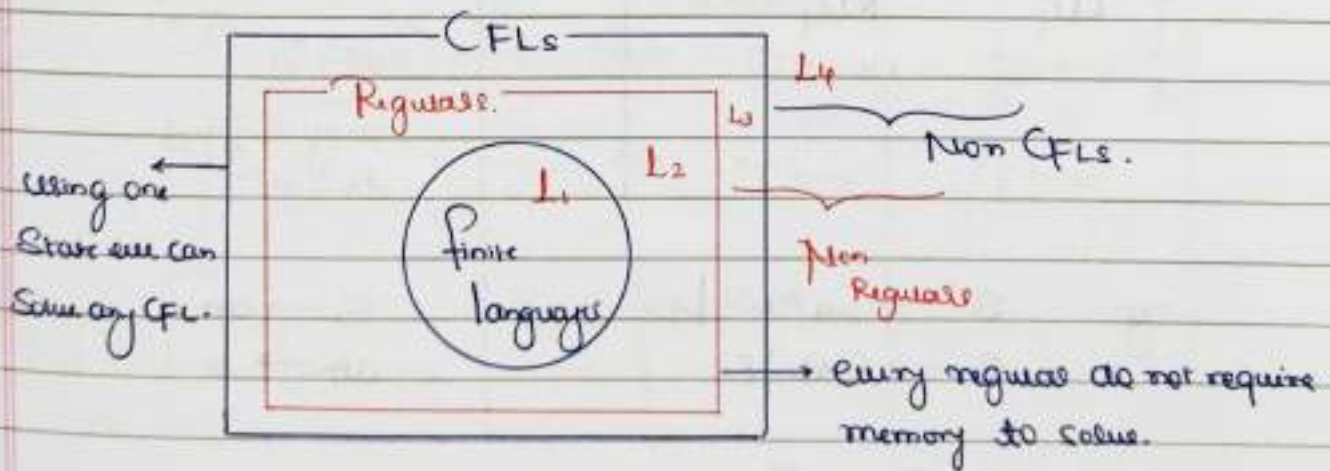
[4] = (b+aa)(a+b)*

II. No. of equivalence classes for non-regular languages.

Regular a^+b^+ → finite no. of equivalent classes.

Non-Regular $a^n b^n$ → Infinite no. of equivalent classes.

Context Free Languages



- I. L_1 is finite language (Reg. CFL)
- II. L_2 is regular but not finite (Infinite Regular)
- III. L_3 is CFL but not regular (CFL and not regular)
- IV. L_4 is not CFL (not regular and not finite)

Regular Language, CFL, CSL

Words $\rightarrow$ Structure $\rightarrow$ Meaning

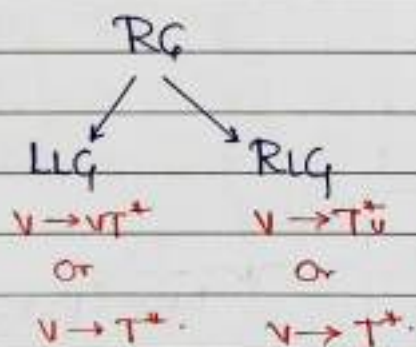
Tokens $\rightarrow$ Syntax $\rightarrow$ Semantic

Words $\rightarrow$ Sentences $\rightarrow$ English language meaning

CFL Applications:

1. Every regular application
2. Messages
3. Call history
4. Web pages history
5. DFS
6. Balanced parenthesis

$$G = (V, T, P, S)$$



CFG.

It represents CFL.

$$V \rightarrow \underbrace{(V \cup T)^+}_{\text{any sequence.}}$$

eg:: $S \rightarrow SaAbS \mid a$ } CFG ✓

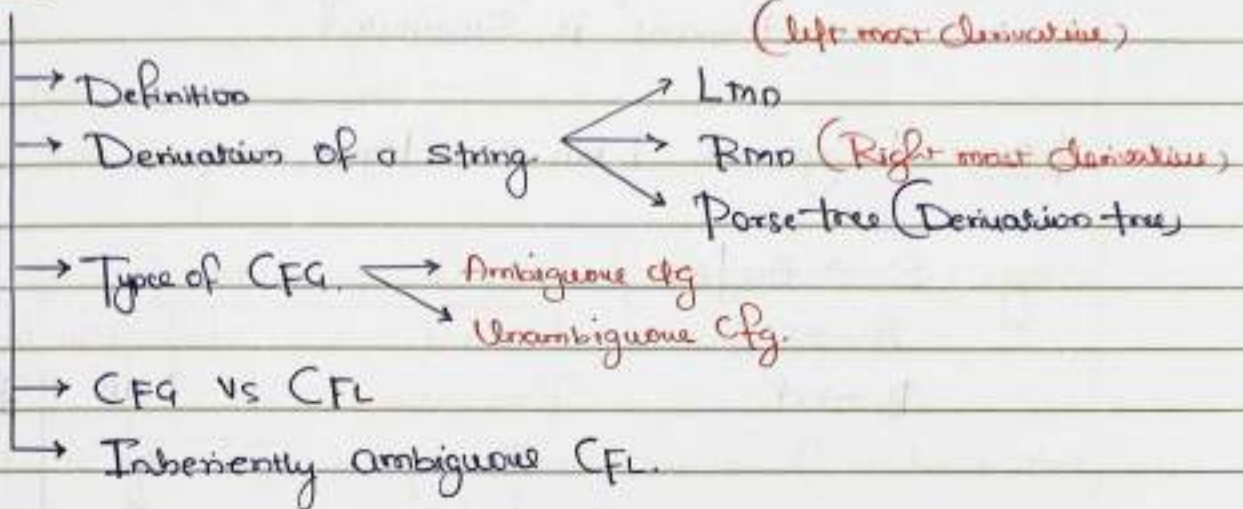
$A \rightarrow Sab \mid \epsilon$

$S \rightarrow aA$ } CFG ✗

$aA \rightarrow b$

- Note:
- (i) Every RG is CFG
 - (ii) CFG need not be RG
 - (iii) Every Regular Language is CFL
 - (iv) CFL is may or may not be regular.

CFG.

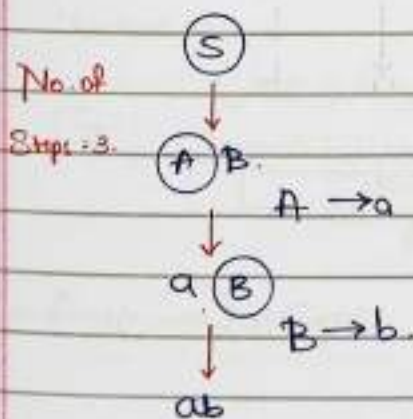


eg.:

CFG	$S \rightarrow AB \dots$	String ab.
	$A \rightarrow a \dots$	
	$B \rightarrow b \dots$	

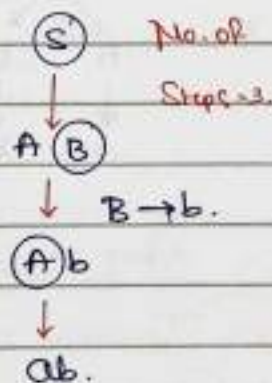
Derivation:

① LMD



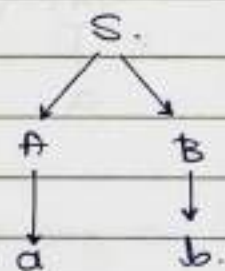
Left Sequential form: aA ,
 ab , aB .

② RMD



Right sequential form:
 aA , aB , ab .

③ Parse Tree



LMD: To derive a string, in every sequential form "left most non terminal" is substituted.

aAB . Q1. left most symbol = 'a'. Q2. left most terminal = 'a'.
Q3. left most non terminal = 'A'.

Rmd: To derive a string, in every sequential form, "right-most non-terminal" is substituted.

Note: In general, Lmd and Rmd need not be same.

eg: $S \rightarrow Aa \mid AB$

$A \rightarrow c$

$B \rightarrow d$

$w = ca$

Lmd and

Rmd are same!

S

↓

Aa

↓

ca

Lmd

Rmd

$w = cd$

Lmd, Rmd are different.

LMD

Rmd

S

↓

AB

↓

CB

↓

cd

S

↓

AB

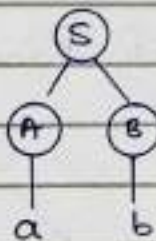
↓

Ad

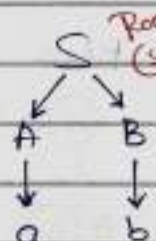
↓

cd

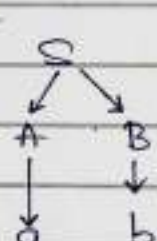
Parse Tree (Derivation Tree)



OR



OR



x no correct!

collect all leaf-nodes = ab

Leaf node = Terminal or ϵ

Non leaf node (Intermediate node) = Variable (non-terminal)

For a given string: (If only one derivation exists)

No. of derivation Steps = No. of Steps in Lmd

= No. of Steps in Rmd

= No. of non-leaf nodes in parse tree.

No. of Lrmd
No. of Rmd
No. of para-tree.

No. of derivations $\rightarrow$ How many ways string can be derived.

$\neq$

No. of derivation Steps: for a particular derivation, how many steps?

$\rightarrow$ It is possible to answer if we have only one derivation.

$\rightarrow$ No. of Substitutions in Rmd.

" " " in Lmd

" " " non leaf node " Para-tree.

Type of CFGs:

[I.] Ambiguous CFG

$\rightarrow$ Some string has more than 1 derivation. (you have to find one string which has > 1 Para-string).
at least one

(not ambiguous).

[II.] Unambiguous CFG

$\rightarrow$ every string derived from CFG has only one derivation.

CFG vs CFL

1. $S \rightarrow AB$
 $A \rightarrow \epsilon$
 $B \rightarrow b$
 $L = \{b\}$

2. $S \rightarrow SS | \epsilon$
 $L = \{\epsilon\}$

3. $S \rightarrow Sa | \epsilon$
 $L = a^*$

4. $S \rightarrow aS | \epsilon$
 $L = a^*$

5. $S \rightarrow Sa | a$
 $L = a^+$

6. $S \rightarrow aS | a$
 $L = a^+$

7. $S \rightarrow as | bs | \epsilon$

$L = (a+b)^*$

11. $S \rightarrow aaS | \epsilon$

$L = (aa)^*$

15. $S \rightarrow AB$

$A \rightarrow aA | \epsilon$

$B \rightarrow Bb | \epsilon$

 $B = b^*$
 $A = a^*$

8. $S \rightarrow as | bs | a | b$

$L = (a+b)^+$

12. $S \rightarrow as | bs | cs | \epsilon$

$L = (a+b+c)^*$

It is not regular grammar but it generates a regular language.

9. $S \rightarrow Sa | Sb | \epsilon$

$L = (a+b)^*$

13. $S \rightarrow abS | aab | \epsilon$

$L = (ab+aa)^*$

$L = a^*b^*$

10. $S \rightarrow Sa | Sb | a | b$

$L = (a+b)^+$

14. $S \rightarrow as | bs | b$

$L = (a+b)^+b$

16. $S \rightarrow aS | Sb | \epsilon$

$\rightarrow a^*Sb^* = a^*b^*$

17. $S \rightarrow bS | Sa | \epsilon \rightarrow b^*a^*$

18. $S \rightarrow aS | Sb | a \rightarrow L = a^*ab^* = a^+b^*$

19. $S \rightarrow aS | Sb | b \rightarrow L = a^*bb^* = a^*b^+$

20. $S \rightarrow aS | Sb | a | b \rightarrow L = a^*(a+b)b^* = a^+b^* + a^*b^+$

21. $S \rightarrow aS | Sb | a | b \rightarrow L = a^*(a+b)b^* = a^+b^* + a^*b^+$

21. $S \rightarrow aS | Sb | ab \rightarrow L = a^*abb^* = a^+b^*$

22. $S \rightarrow aSb | \epsilon \rightarrow L = a^nSb^n = a^n b^n$

$= \{a^n b^n \mid n \geq 0\} = \{a^i b^j \mid \#a = \#b\}$

23. $S \rightarrow bSa | \epsilon$ $L = \{b^n a^n \mid n \geq 0\}$

24. $S \rightarrow aSb | a$ $L = \{a^n a b^n \mid n \geq 0\} = \{a^{n+1} b^n\}$

25. $S \rightarrow aSb | b$ $L = \{a^n b^{n+1} \mid n \geq 0\}$

26. $S \rightarrow aaSbb | \epsilon$ $L = \{a^{2n} b^{2n}\}$

27. $S \rightarrow aaaSbbb | \epsilon$ $L = \{a^{3n} b^{3n}\}$

28. $S \rightarrow aSbb | \epsilon$ $L = \{a^n b^{2n} \mid n \geq 0\}$

29. $S \rightarrow aaSb | \epsilon$ $L = \{a^{2n} b^n\} = \{a^m b^n \mid m = 2n\}$

30. $S \rightarrow aaSbbb | \epsilon$ $L = \{a^{2n} b^{3n} \mid n \geq 0\}$
 $= \{a^i b^j \mid i = 2n, j = 3n\}$

31. $S \rightarrow aSb \mid ab$

$$L = \{ab, a^2b^2, a^3b^3, \dots\} \neq a^+b^+ \\ \left\{ \begin{array}{l} L = \{a^n ab B \mid n \geq 0\} \\ = \{a^m, b^m \mid n \geq 0\} \\ = \{a^n, b^n \mid n \geq 1\} \\ = \{a^n b^n \mid n \geq 2\} \end{array} \right.$$

Note: $L_1 = \{a^n \mid n \geq 0\} = a^+$

$L_2 = \{b^n \mid n \geq 0\} = b^+$

$$\begin{aligned} L_1 \cdot L_2 &= \{a^n\} \cdot \{b^n\} \\ &= \{\epsilon, a, a^2, \dots\} \cdot \{\epsilon, b, b^2, \dots\} \\ &= \{a^m b^n \mid m, n \geq 0\} \\ &= \{a^n b^{n_2} \mid n_1, n_2 \geq 0\} \end{aligned}$$

32. $S \rightarrow aSb \mid A$

$A \rightarrow aA \mid \epsilon$

$L = \{a^n a^+ b^n\} = \{a^i b^j \mid i \geq j\}$

33. $S \rightarrow aSb \mid A$

$A \rightarrow bA \mid \epsilon \rightarrow b^+$

$L = \{a^n b^{m+n}\} = \{a^i b^j \mid i \leq j\}$

34. $S \rightarrow aSb \mid A \quad \left\{ \begin{array}{l} S \rightarrow aSb \mid \epsilon \\ A \rightarrow bB \mid \epsilon \end{array} \right. \quad L = a^n b^n$

35. $S \rightarrow SA \mid \epsilon$

$A \rightarrow a \quad L = A^+ = a^+$

36. $S \rightarrow SA \mid \epsilon$

$A \rightarrow aa \quad L = A^+ = (aa)^+$

37. $S \rightarrow SA \mid \epsilon$

$A \rightarrow a \mid b \quad L = A^+ = (a+b)^+$

38. $S \rightarrow SA \mid \epsilon$

$A \rightarrow aAb \mid \epsilon$

$L = \{a^n b^n\}^+ = \{a^n b^n \mid n \geq 0\}$

39. $S \rightarrow SA \mid \epsilon$

$A \rightarrow bAa \mid \epsilon$

$L = \{b^n a^n\}^+$

40. $S \rightarrow SA \mid \epsilon$

$A \rightarrow ab \quad L = (ab)^+$

41. $S \rightarrow aSa | bSb | \epsilon \rightarrow L = \{ww^R \mid w \in \{a,b\}^+\}$
 - Set of all even length Palindromes
 - $\{x \mid x \in \{a,b\}^+, x = x^R, |x| = \text{even}\}$
42. $S \rightarrow aSa | bSb | a | b \rightarrow L = \text{length of all odd length Palindromes}$
43. $S \rightarrow aSa | bSb | a | b | \epsilon \rightarrow L = \text{set of all palindromes}$
 - $\{x \mid x = x^R, x \in \{a,b\}^+\}$
 - $\{ww^R \mid w \in \{a,b\}^+, x \in \{a,b,\epsilon\}\}$
44. $S \rightarrow aSa | bSb | \#$
 $L = \{w\#w^R \mid w \in \{a,b\}^+\}$
45. $S \rightarrow aSb | a$
 $A \rightarrow aAc | \epsilon \rightarrow L = a^k c d^k b^k = \{a^k c^1 b^k\}$
46. $S \rightarrow Ab$
 $A \rightarrow aAb | \epsilon$ $B = c^1 d^1$ $L = \{a^k b^1 c^1 d^1\}$
 $B \rightarrow cBd | \epsilon$ $A = a^k b^k$
47. $S \rightarrow aSb | A$ } $A = c^1 b^1$
 $A \rightarrow cAb | \epsilon$ } $L = a^k c^1 b^1 b^k = \{a^k c^1 b^{k+1}\}$
48. $S \rightarrow Aec$
 $A \rightarrow aA | \epsilon$ - a^+
 $B \rightarrow bB | \epsilon$ - b^+
 $C \rightarrow cC | \epsilon$ - c^+ } $L = a^+ b^+ c^+$
49. $S \rightarrow Ss | (S) | \epsilon$
 $L = \text{Set of all balanced Parentheses}$
50. $E \rightarrow E+E | E * E | (E) | a$
 $L = \text{Set of all arithmetic expressions using } +, *$ } $\left. \begin{array}{l} \text{functionality} \\ \text{(meaning)} \end{array} \right\}$
51. $S \rightarrow aSb | bSa | SS | \epsilon$
 $L = \{w \mid w \in \{a,b\}^+, n_a(w) = n_b(w)\}$
52. $S \rightarrow aSb | bSa | SS | aS | \epsilon$
 $L = \{w \mid w \in \{a,b\}^+, n_a(w) \geq n_b(w)\}$
53. $S \rightarrow aSb | bSa | SS | bS | \epsilon$
 $L = \{w \mid w \in \{a,b\}^+, n_a(w) \leq n_b(w)\}$
54. $S \rightarrow aSb | bSa | SS | aS | a$
 $n_a(w) > n_b(w)$

55. $S \rightarrow SaSbS \mid SbSaS \mid \epsilon$

$L = \{w \mid w \in \{a,b\}^*, n_a(w) = n_b(w)\}$

→ Every Regular Language is CFL.

→ Some non regular languages are CFL.

$\{a^n b^n\}$ ← not reg
CFL

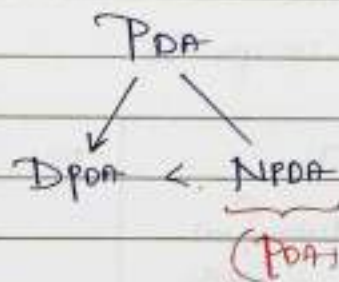
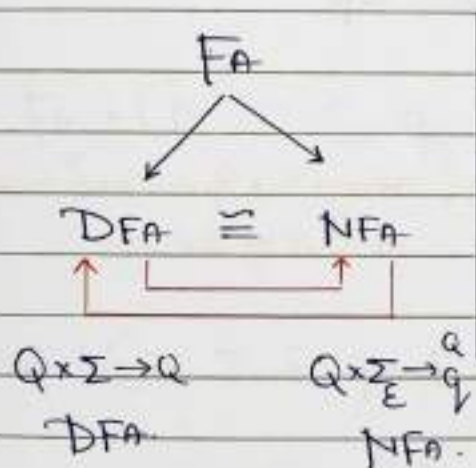
$\{ww^R \mid w \in \{a,b\}^*\}$

$\{w \# w^R\}$

$\{a^n b^{2n}\}$

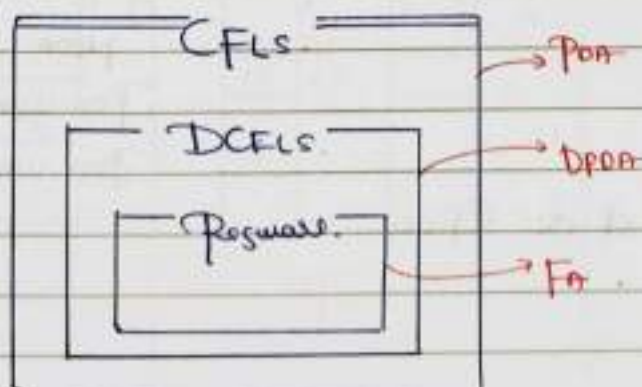
$\{a^n b^n\}$

Push Down Automata



DPDA	PDA
$\cong$	$\cong$
DCFL	CFL

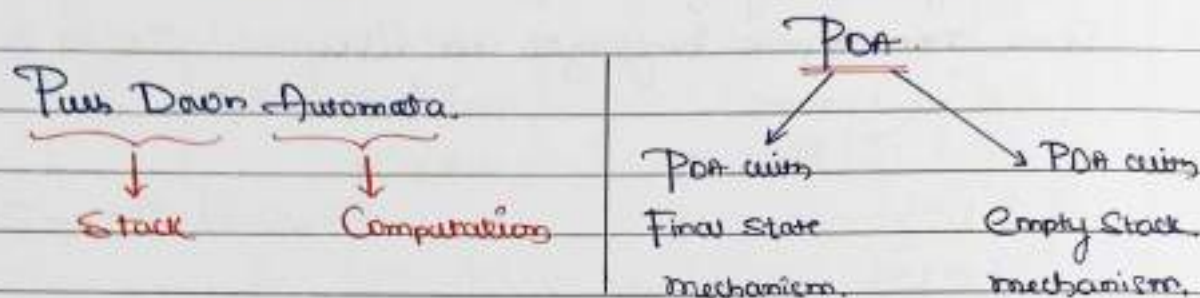
- * Every DCFL is CFL
- * CFL need not be DCFL
- * Every DPDA is PDA
- * PDA need not be DPDA



$FA < DPDA < PDA$

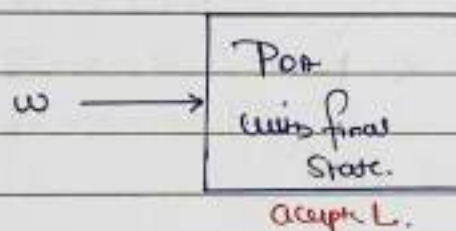
→ Which of the following statements is "True?"

1. Regular Language is Subset of CFL [False]
2. Set of all regular is Subset of Set of all CFLs [True]

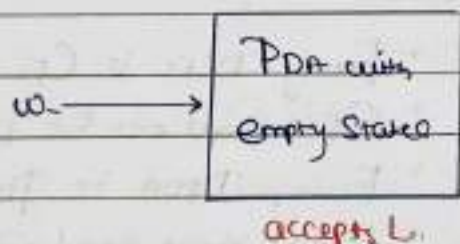


PDA (NPDAs)

→ by default non-deterministic m/c
→ it accepts CFL.



If $w \in L$, at least one path exist that halt at final state.



If $w \in L$, at least one path exist that makes empty stack

→ $L_1 =$ Set of all language represented by PDA

→ $L_2 =$ " " " "

→ $L_3 =$ " " " "

→ $L_4 =$ " " " "

→ $L_5 =$ Set of all CFL.

NPDA

PDA with Final State

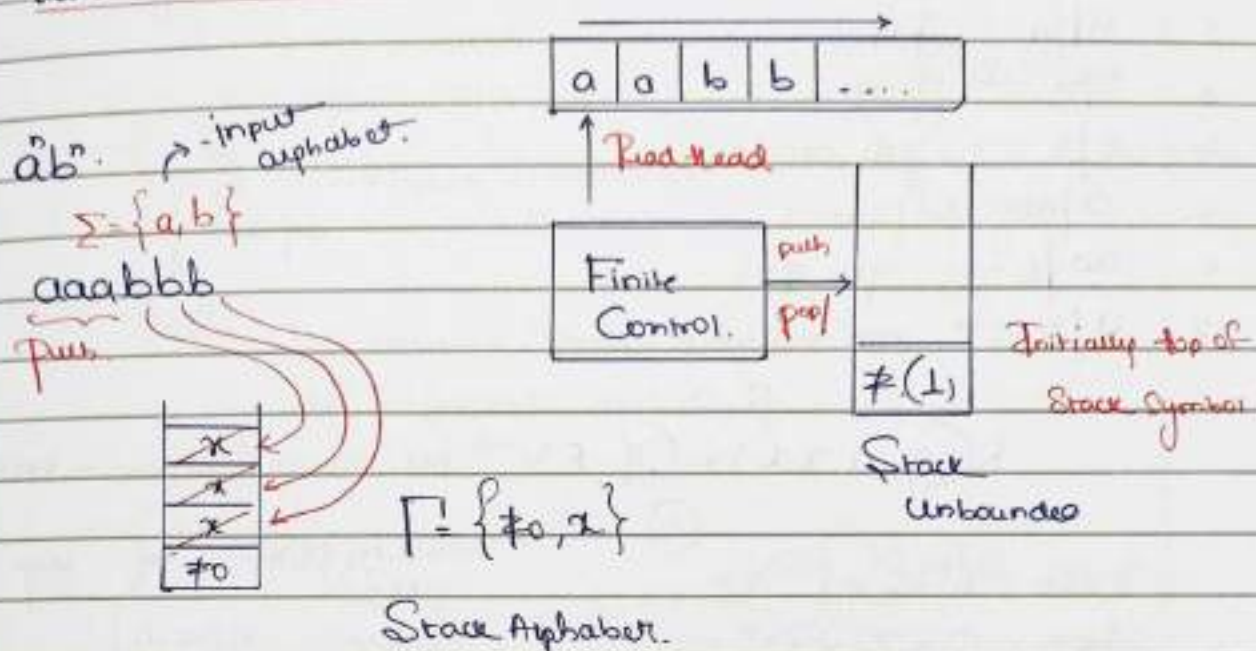
PDA with Empty State

$L_1 = L_2$

$= L_3 = L_4$

$= L_5$

all are same

Push Down Automata.

$$FA = (Q, \Sigma, \delta, q_0, F)$$

$$\delta_{NFA} : Q \times \Sigma \rightarrow 2^Q$$

$$\delta_{DFA} : Q \times \Sigma \rightarrow Q$$

$$POA = FA + \text{Stack}$$

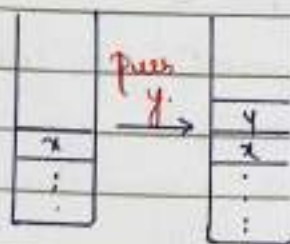
$$POA = (Q, \Sigma, \delta, q_0, F, z_0, \Gamma)$$

$$\delta_{POA} : Q \times \Sigma \times \Gamma^+ \rightarrow 2^{Q \times \Gamma^+}$$

$$\delta_{DPOA} : Q \times \Sigma \times \Gamma^+ \rightarrow Q \times \Gamma^+$$

Stack alphabet
(Set of Stack Symbols)

initial top of stack symbol
(bottom of Stack Symbols).

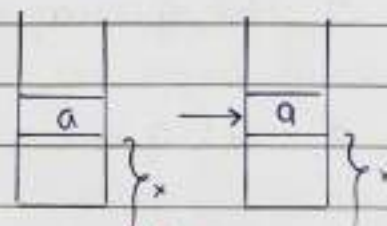
Push

$$x \rightarrow yx$$

$$x/yx$$

Pop

$$x/e$$

No Operations

$$a/a$$

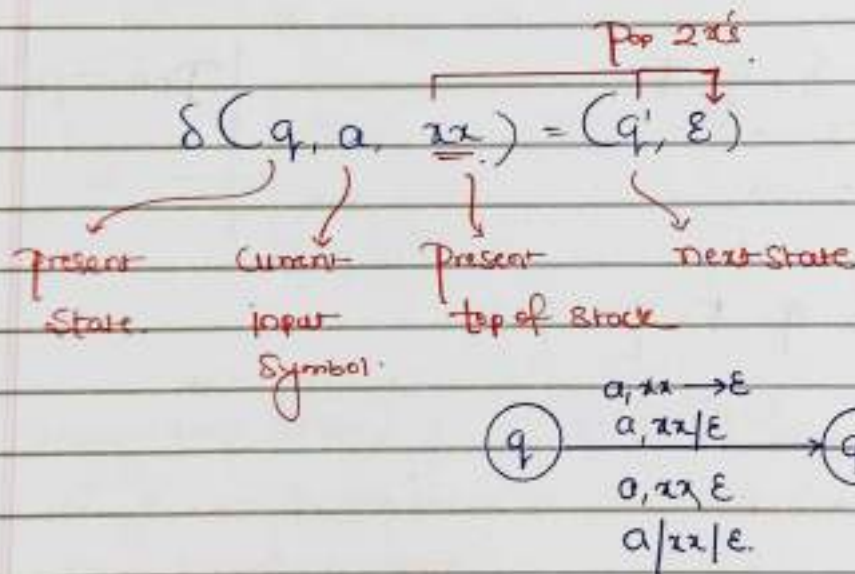
$$\epsilon/e$$

1. $a/a \rightarrow$ no operation performed on stack
2. $a/\epsilon \rightarrow$ pop a
3. $\epsilon/a \rightarrow$ push a
4. $\epsilon/\epsilon \rightarrow$ no operations " "
5. $a/aa \rightarrow$ push a
6. $aa/\epsilon \rightarrow$ pop aa
7. $a/b \rightarrow$ not valid.

$\delta(q, a, \underline{xx}) = (q', \epsilon) \rightarrow$ not valid transition in DPDA
 $\rightarrow$ valid in PDA.

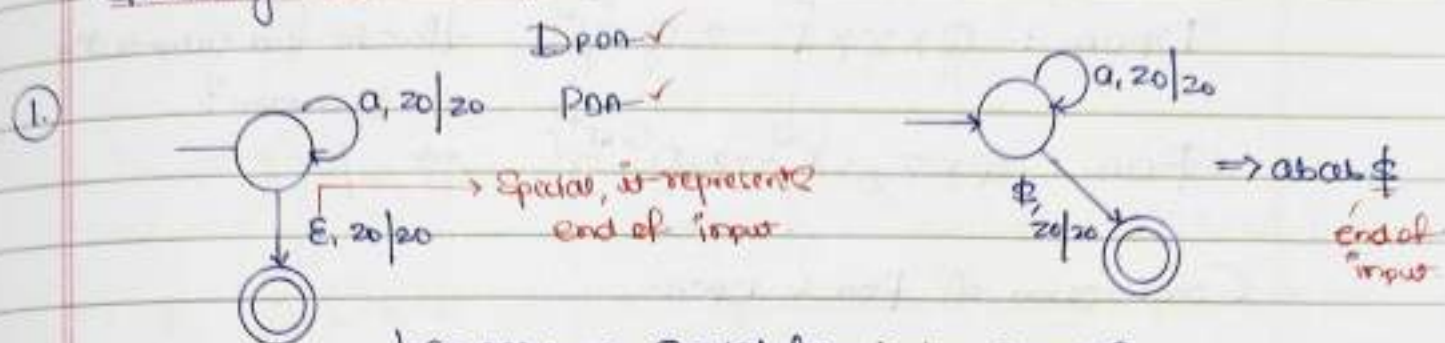
DPDA = $Q \times \Sigma \times \Gamma \rightarrow$

PDA = $Q \times \Sigma \times \Gamma^+ \rightarrow$

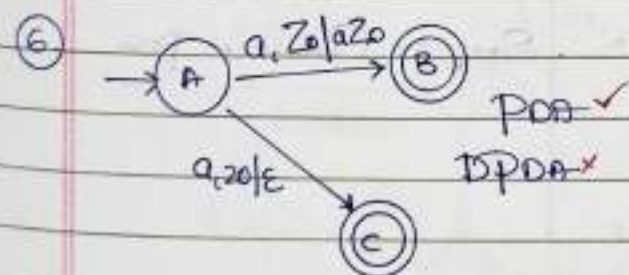
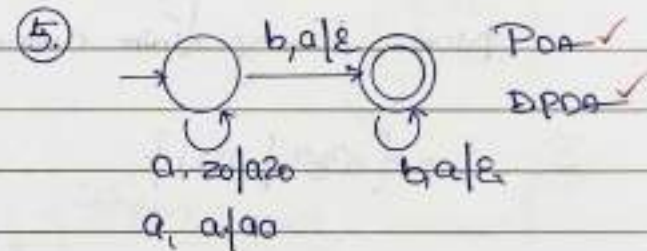
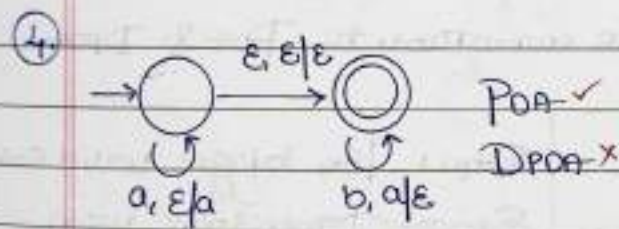
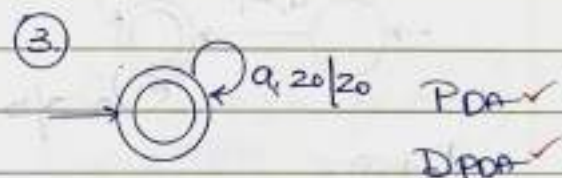
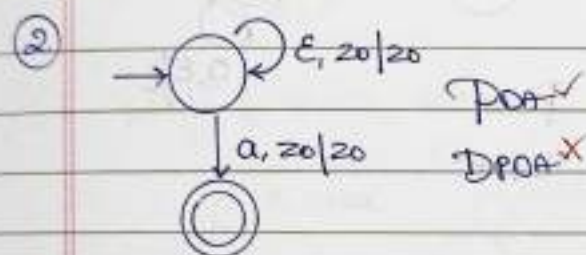


1. $\delta(q, \epsilon, \epsilon) = (q', a)$ $\begin{matrix} \text{PDA} \checkmark \\ \text{DPDA} \times \end{matrix}$ $\rightarrow$ Without reading input, without looking at tos, push 'A'.
2. $\delta(q, \epsilon, a) = (q', a)$ $\begin{matrix} \text{DPDA} \times \\ \text{PDA} \checkmark \end{matrix}$ $\rightarrow$ Without reading input, when tos = a no operations performed on stack.
3. $\delta(q, a, \epsilon) = (q', a)$ $\begin{matrix} \text{PDA} \checkmark \\ \text{DPDA} \times \end{matrix}$ $\rightarrow$ When i/p = a, without looking at stack push 'a'.
4. $\delta(q, a, b) = (q', ab)$ $\begin{matrix} \text{PDA} \checkmark \\ \text{DPDA} \checkmark \end{matrix}$ $\rightarrow$ When i/p = a, tos = b, push a.
5. $\delta(q, a, b) = (q', aab)$ $\begin{matrix} \text{PDA} \checkmark \\ \text{DPDA} \checkmark \end{matrix}$ $\rightarrow$ When i/p = a, tos = b, push aab.

Identify DPDA or PDA



We assume special transitions required to represent end of i/p.



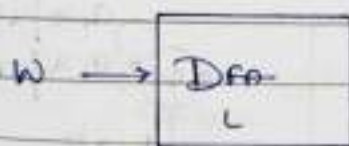
$$Q \times \Sigma \times \Gamma \rightarrow Q \times \Gamma^+$$

Fail

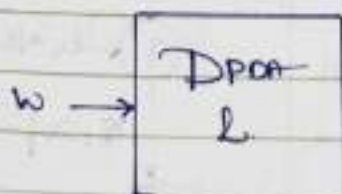
$$Q \times \Sigma_E \times \Gamma^+ \rightarrow Q \times \Gamma^+$$

$$\delta(A, a, z_0) = (B, a z_0)$$

or
(C, ε)



If $w \in L$, exactly 1 path leads to final state
If $w \notin L$, exactly 1 path leads to non final state

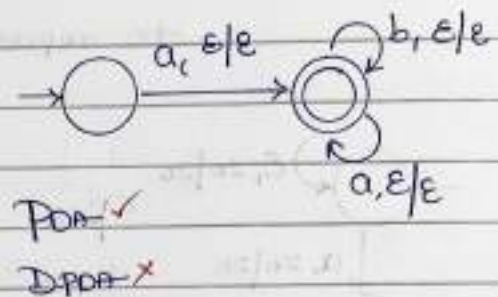
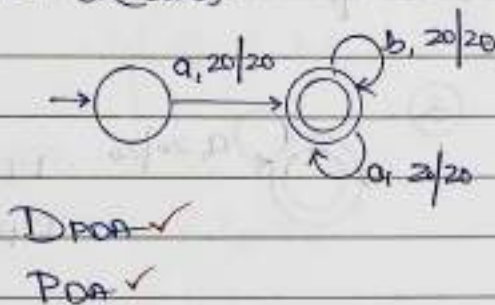


If $w \in L$, exactly 1 path leads to final state
If $w \notin L$, exactly 1 path leads to non final state
or
Zero paths

Operation
 $DPOA : Q \times \Sigma \times \Gamma \rightarrow Q \times \Gamma^+$ How to find cutar to the operation?
 $POA : Q \times \Sigma \times \Gamma \rightarrow Q \times \Gamma^+ \rightarrow 3/5$

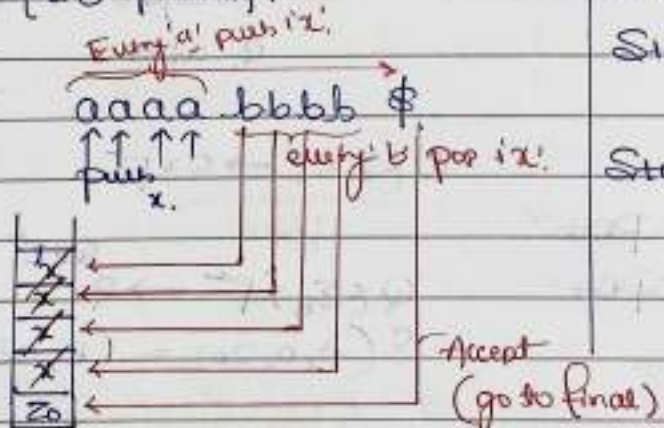
Construction of POA & DPOA

① $L = a(Cab)^+$



Note: Every finite automata is convertible to POA & DPOA.

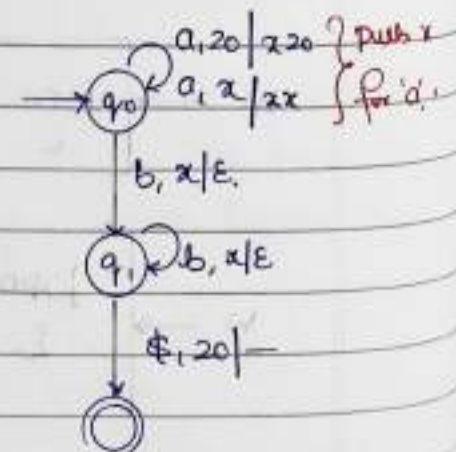
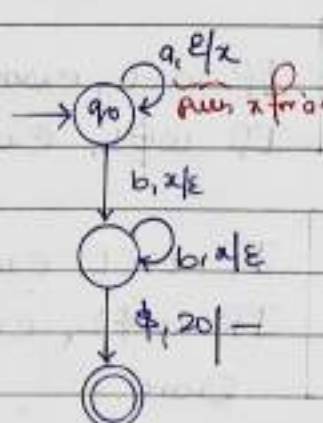
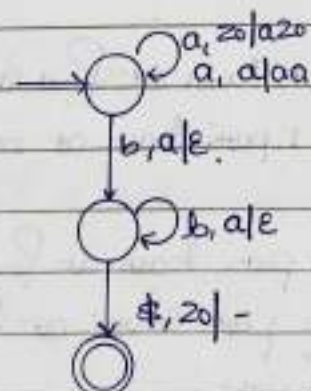
② $L = \{a^n b^n \mid n \geq 1\}$



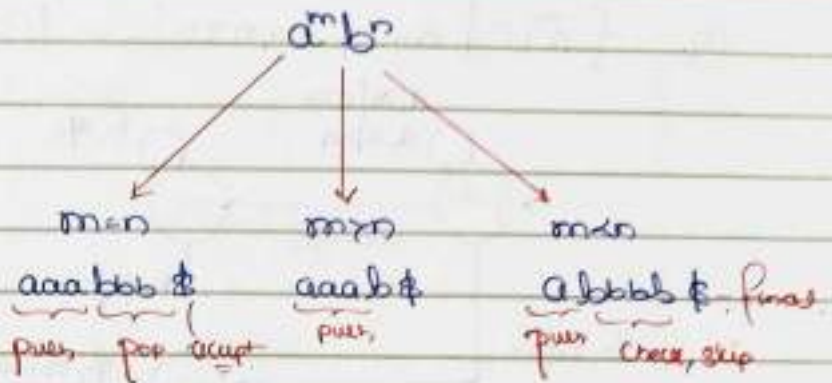
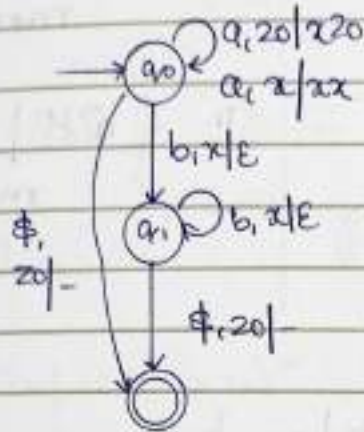
Step 1: Take bigger valid string

Step 2: Take logic using Stack

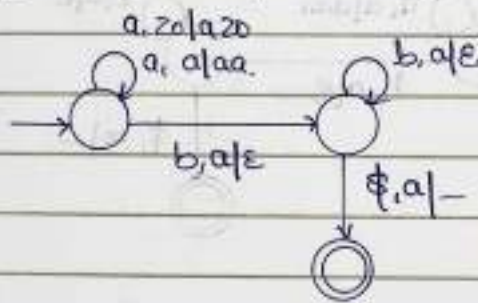
Step 3: Construct POA.



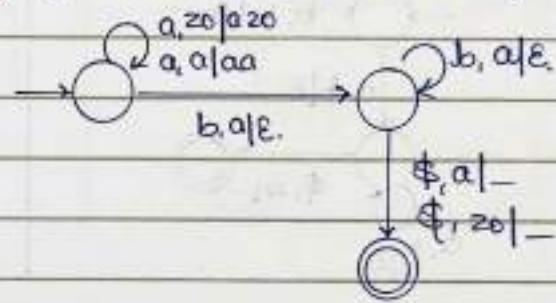
③ $L = \{a^n b^n \mid n \geq 0\}$
 $= \emptyset \cup \{ \epsilon \}$



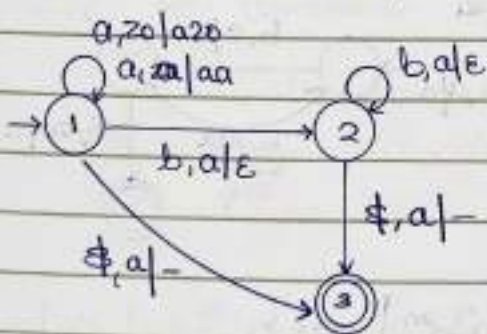
④ $\{a^m b^n \mid m > n, m, n \geq 1\}$



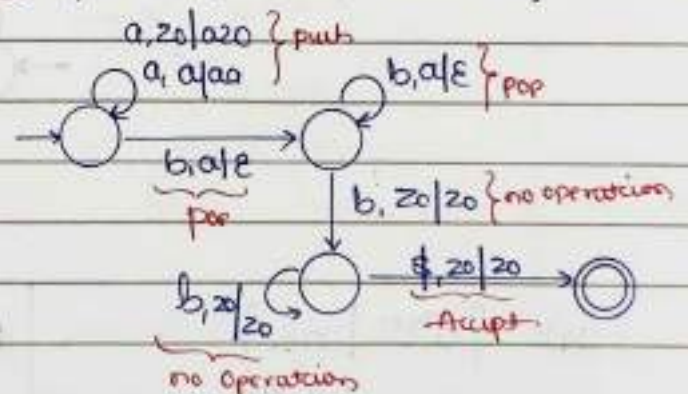
⑤ $\{a^m b^n \mid m \geq n, m, n \geq 1\}$



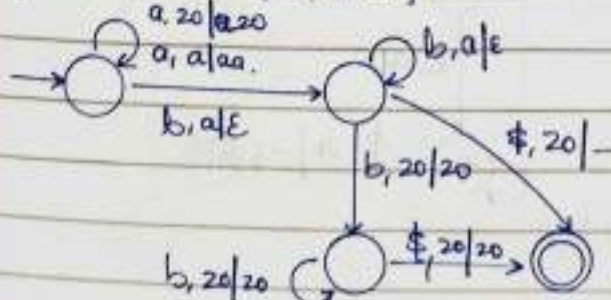
⑥ $\{a^m b^n \mid m > n, m, n \geq 0\}$



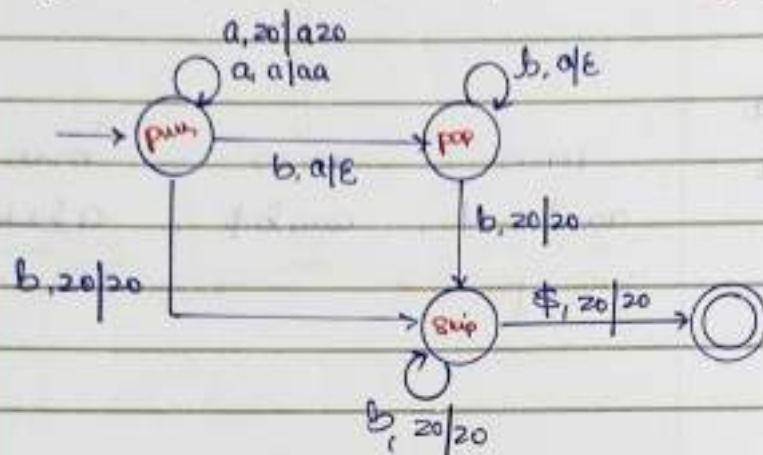
⑦ $\{a^m b^n \mid m \leq n, m, n \geq 1\}$



⑧ $\{a^m b^n \mid m \leq n, m, n \geq 1\}$



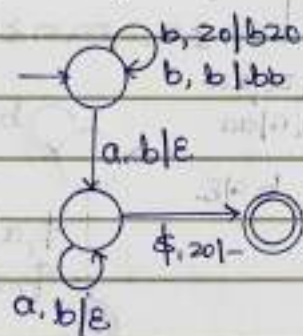
9. $\{a^m b^n \mid m, n \geq 0\}$. 7. $u \in \{b^+\}$



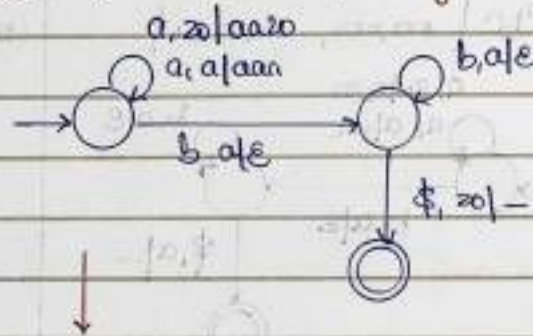
10. $\{a^m b^n \mid m, n \geq 1, m \neq n\}$

11. $\{a^m b^n \mid m, n \geq 0, m \neq n\}$

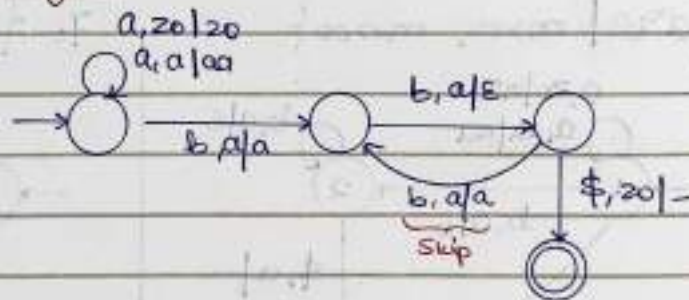
12. $\{b^m a^n \mid n \geq 1\}$



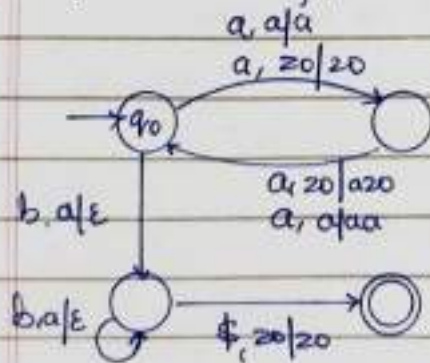
13. $\{a^m b^{2m} \mid n \geq 1\}$ Logic 1:



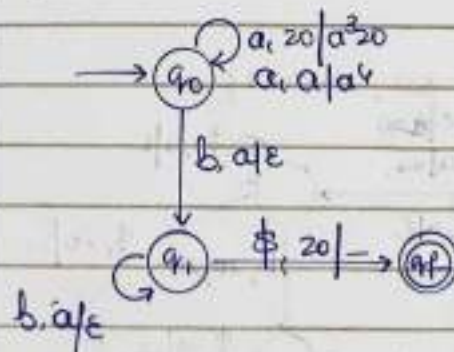
Logic 2:



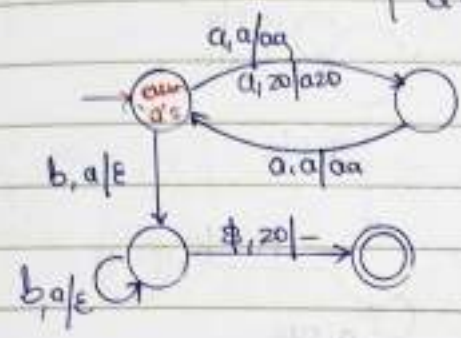
14. $\{a^m b^n \mid n \geq 1\}$



15. $\{a^m b^{3m} \mid n \geq 1\}$

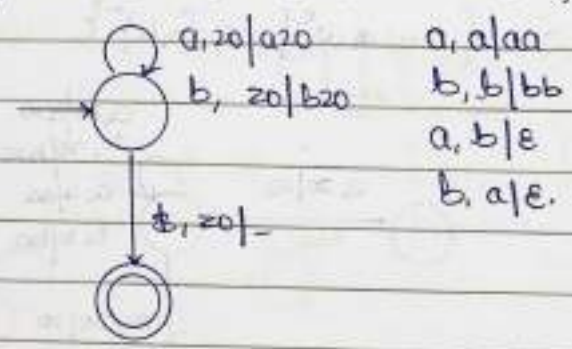


16. $\{a^m b^n \mid m, n \geq 1, m=n \text{ even}\}$
 $= \{a^2 b^2, a^4 b^4, \dots\}$

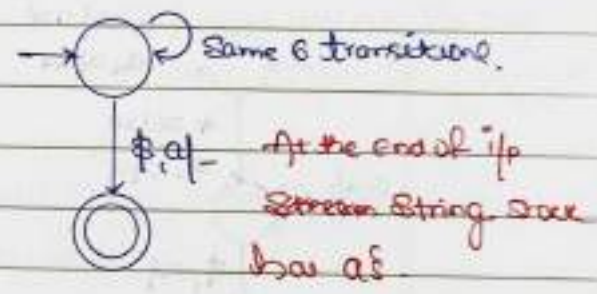


aaaa bbbb $\notin$
 push a every b, pop a.
 for even a's
 guarantee even a's

17. $\{w \mid w \in \{a, b\}^+ \wedge n_a(w) = n_b(w)\}$

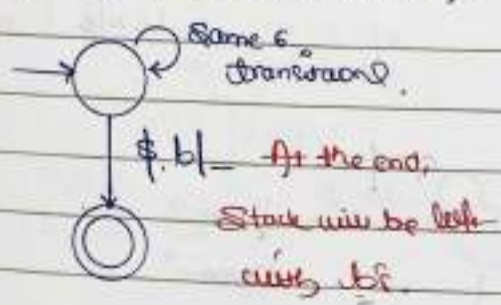


18. $\{w \mid w \in \{a, b\}^+ \wedge n_a(w) > n_b(w)\}$



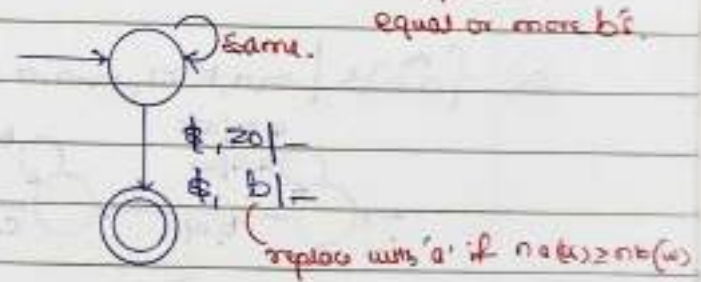
At the end if p
 string string stack
 has a's

19. $\{w \mid w \in \{a, b\}^+ \wedge n_a(w) < n_b(w)\}$



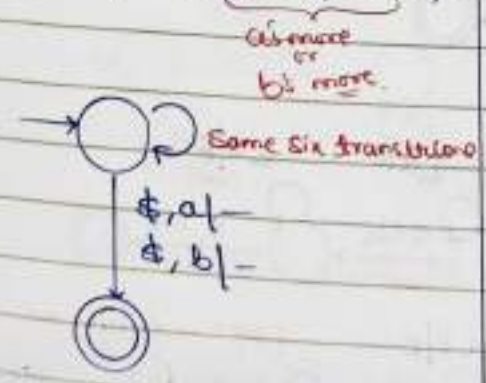
At the end,
 stack will be left
 with b's

20. $\{w \mid w \in \{a, b\}^+ \wedge n_a(w) \leq n_b(w)\}$



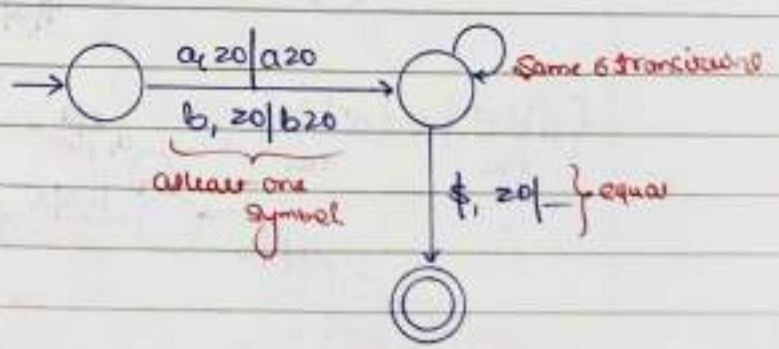
equal or more b's
 replace with 'a' if $n_a(w) > n_b(w)$

21. $\{w \mid w \in \{a, b\}^+ \wedge n_a(w) \neq n_b(w)\}$



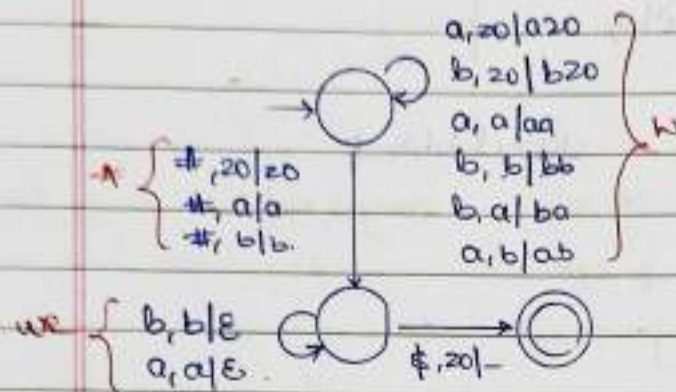
at least one
 or
 b's more
 Same six transitions

Note: $L = \{w \mid w \in \{a, b\}^+ \wedge n_a(w) = n_b(w)\}$

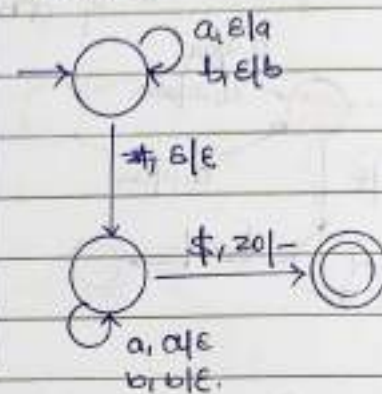


allow one symbol
 equal

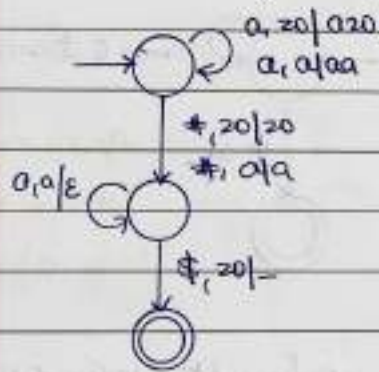
23. $L = \{ w \# w^R \mid w \in \{a,b\}^+ \}$



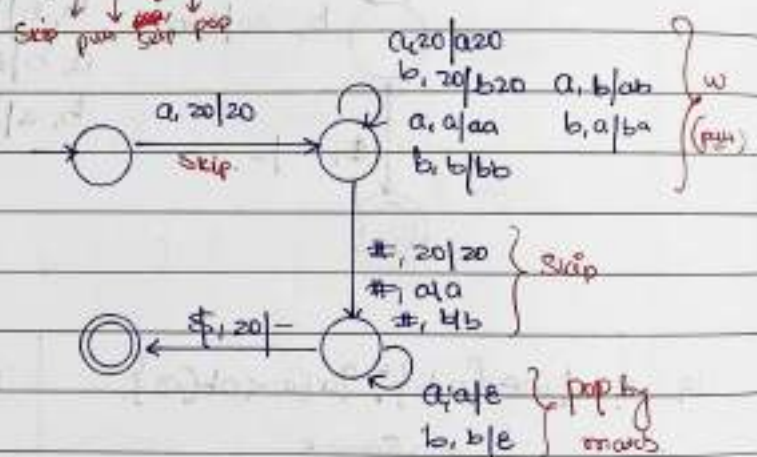
PDA ✓
DPDA ✗



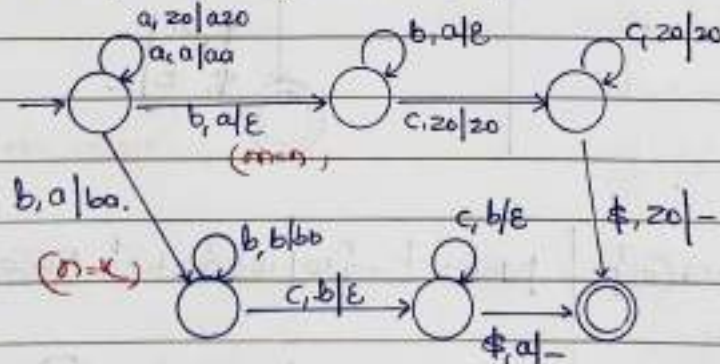
24. $L = \{ a^n \# a^n \mid n \geq 0 \}$



25. $L = \{ a^n w \# w^R \mid w \in \{a,b\}^+ \}$

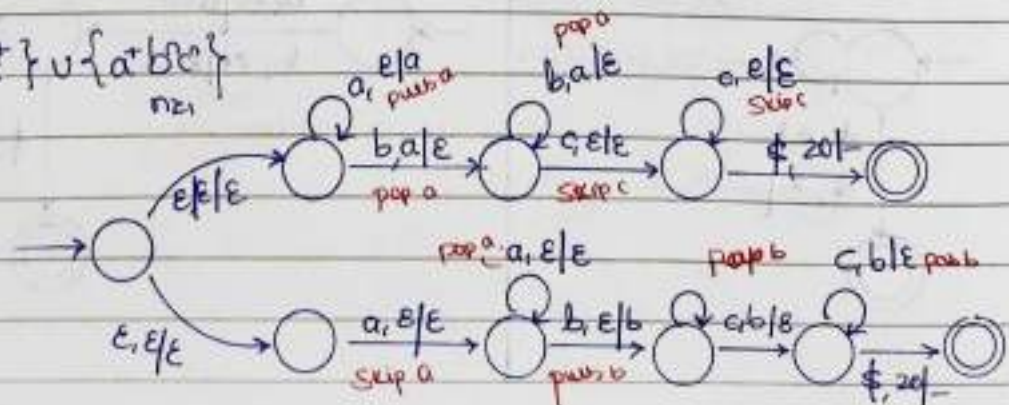


26. $\{ a^m b^k c^l \mid m,n,k \geq 1, m+n+k \geq 2 \}$



PDA ✓
DPDA ✗

$\{ a^n b^n c^n \} \cup \{ a^n b^n \}$



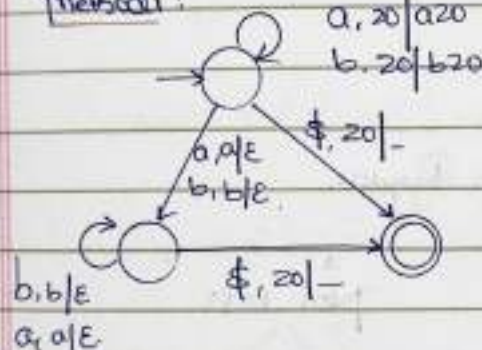
27. $\{a^m b^n c^k \mid m, n, k \geq 1, m=n \text{ or } m=k\}$
 pop a_i skip b_i $a_i = b_i$ $a_i = c_i$
 $a^n b^n c^1$ $a^n b^1 c^n$
 $n \geq 1$ $n \geq 1$

CFLS but not DFLS

28. $\{a^m b^n c^k \mid m, n, k \geq 1, m=k \text{ or } n=k\}$
 push a_i push b_i pop b_i for c_i $\#a_i = \#c_i$ $\#b_i = \#c_i$
 without reading c_i pop b_i , then every c_i pop a_i

29. $\{ww^R \mid w \in \{a,b\}^+\}$

Method 1:



$a, a/a$
 $a, a/a$
 $a, b/b$
 $b, a/a$
 $b, b/b$

When will w^R begin?

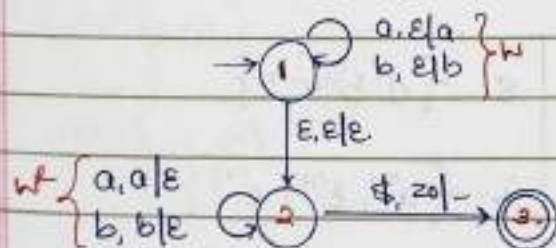
Don't know

If w^R begins, what's the logic?

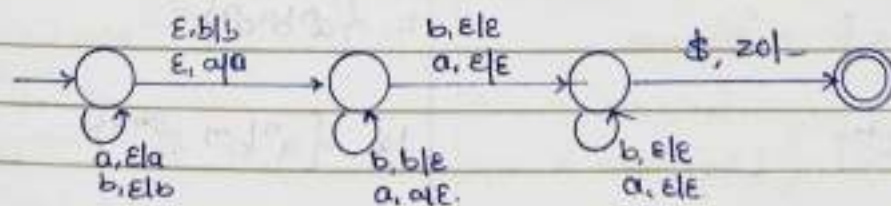
present i/p = top of previous i/p

* For valid String, atleast 1 path that ends at final

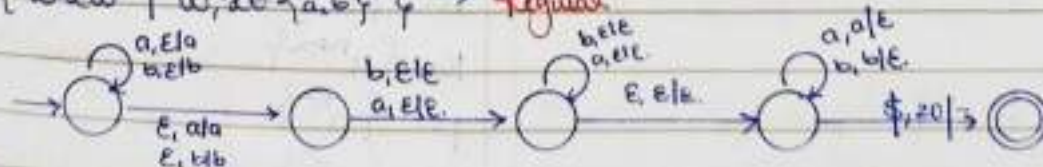
$\{ww^R \mid w \in \{a,b\}^+\} \rightarrow$ CFL but not DFL



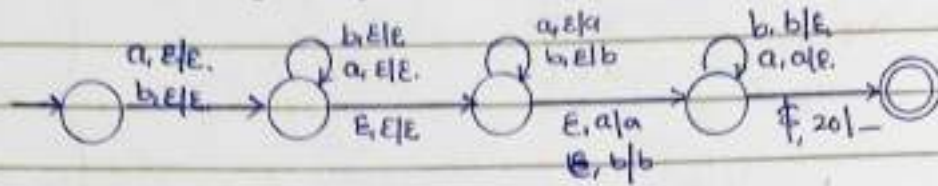
30. $\{waw^R \mid w, a \in \{a,b\}^+\} \rightarrow$ CFL but not DFL



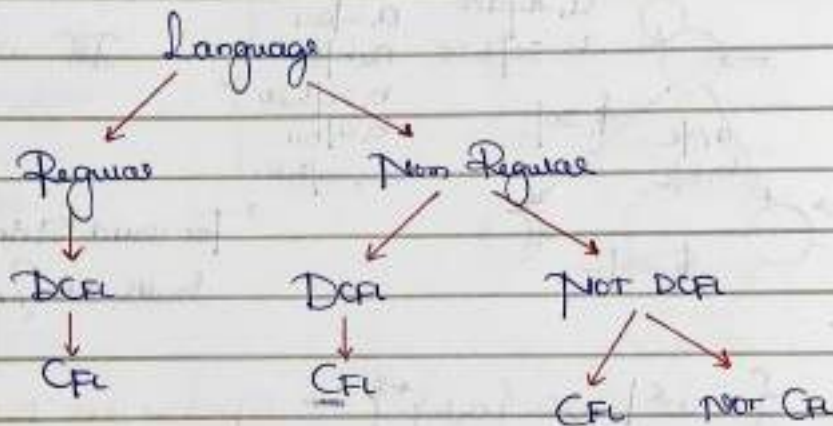
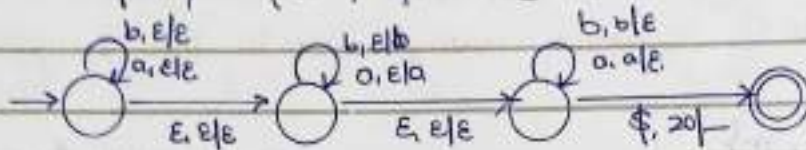
31. $\{waw^R \mid w, a \in \{a,b\}^+\} \rightarrow$ Regular



32. $\{xww^R \mid w, x \in \{a, b\}^+\} \rightarrow \text{CFL but not DCF}$



33. $\{xww^R \mid w, x \in \{a, b\}^*\} \rightarrow (a+b)^+$



1. $\{a^n b^n\} \rightarrow a^* b^* \text{ Reg. } \rightarrow \text{DCF} \rightarrow \text{CFL}$

2. $\{a^n b^n\}$ } DCF but not regular

3. $\{a^n b^{2n}\}$ }

4. $\{a^n b^n c^n\} \rightarrow \text{Regular}$

5. $\{a^n b^n c^n\}$ } DCF but not regular

6. $\{a b^n c\}$ }

7. $\{a^n b^n c^n\} \rightarrow \text{Not CFL}$

8. $\{a^n b^{2n} c^{2n}\}$ } Not CFL

9. $\{a^{n!}, b^{n!}, c^{n!}\}$ }

10. $\{a^{n!}, b^{n!}, c^{n!}\} \rightarrow \text{Regular}$

11. $\{a^n b^n d^n\} \rightarrow \text{NOT CFL}$

12. $\{a^n b^n c^{nm}\} \rightarrow \text{DCF but ! Reg}$

13. $\{a^{n!}\}$ } Not Regular $\Rightarrow$ Not CFL
14. $\{a^{n!}\}$
15. $\{a^{n^2}\}$
16. $\{a^{n^3}\}$

Note: * If L is over 1 symbol $\Rightarrow F_n = PDA$
All not regular are not CFL.

18. $\{a^n\} = a^* = \text{Regular}$

19. $\{ww \mid w \in \{a,b\}^*\}$ } not CFL

20. $\{w\#w \mid w \in \{a,b\}^*\}$ }

21. $\{ww^R \mid w \in \{a,b\}^*\} \rightarrow \text{CFL but not DCF}$

22. $\{w\#w^R \mid w \in \{a,b\}^*\} - \text{DCF but not Regular.}$

23. $\{ww \mid w \in a^*\} - \text{Reg} \Rightarrow \text{DCF} \Rightarrow \text{CFL}$

24. $\{w\#w \mid w \in a^*\} - \text{DCF but not reg.}$

25. $\{a^n b^n c^n d^n\}$ } DCF

26. $\{a^n b^n c^n d^n\}$ }

27. $\{a^n b^n c^n d^n\} - \text{not CFL}$

28. $\{a^n b^n c^{k+1} d^n\} - \text{DCF}$
($m+n = k+1$)

29. $\{a^2 b^2\} - \text{DCF}$

30. $\{a^n b^n\}$ }

31. $\{a^n b^n\}$ } Not CFL

32. $\{a^n b^n c^{2n}\}$ }

33. $\{w \mid w \in \{0,1\}^*, n_0(w) = n_1(w)\}$
DCF

34. $\{wwx \mid w, x \in \{a,b\}^*\}$
Also: $waw, waw^R, xaw, xaw^R, waw^R, waw^L$ } Regular
↓
DCF
↓
CFL

35. $\{wwx \mid w, x \in \{a,b\}^*\}$

36. $\{waw \mid " " \}$ } Not CFL
37. $\{xaw \mid " " \}$ }

38. $\{wwx \mid " \}$ } CFL but not DCF

39. $\{xaw^R \mid " \}$ } DCF

40. $\{waw^R \mid " " \} - \text{Regular}$

41. $\{ww^R x^R \mid w, x \in \{a,b\}^*\} - \text{CFL but not DCF}$

42. $\{wwx \mid w, x \in \{a,b\}^*\}$ }

43. $\{ww^R w \mid " \}$ }

44. $\{www \mid " \}$ }

45. $\{w\#w\#w \mid " \}$ }

46. $\{a^n \# a^n \# a^n\}$ } Not CFL

47. $\{a^n \# b^n \# c^n\}$ }

49. $\{w\#w\#x\#x \mid w, x \in \{a, b\}^*\}$ - DCF

46. $\{a^m\#b^n\#c^k\}$ } Regular.

50. $\{a^m\#a^n\#a^k\}$ }

51. $\{a^m b^n c^k \mid m=n \text{ or } m=k\}$

52. $\{a^m b^n c^k \mid m \leq n \text{ or } m \leq k\}$

53. $\{a^m b^n c^k \mid m \neq n \text{ or } m \neq k\}$

54. $\{a^m b^n c^k \mid m \neq n \text{ or } m \neq k\}$

55. $\{a^m b^n c^k \mid \text{if } (m \neq n) \text{ then } m=k\}$

CF but not DCF

56. $\{a^m b^n \mid m \neq n \text{ or } m=n\} = a^* b^* - \text{Regular}$

57. $\{a^m b^n c^k \mid m=n \text{ and } m=k\} = a^n b^n c^n$ } not CF

58. $\{a^m b^n c^k \mid m \neq n \text{ and } m \leq k\}$

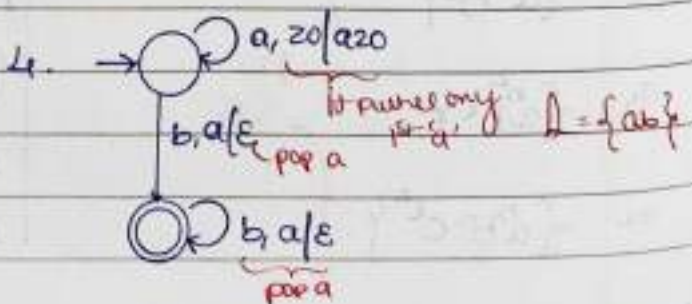
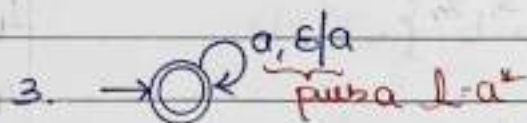
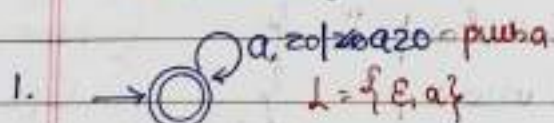
59. $\{a^m b^n c^k \mid m \neq n \text{ or } (m \leq k)\} - \text{CF but not DCF}$

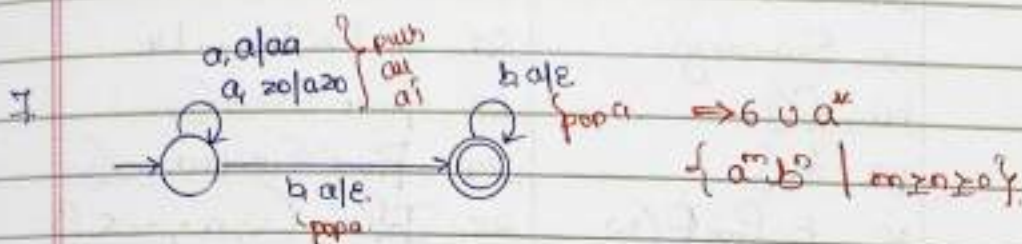
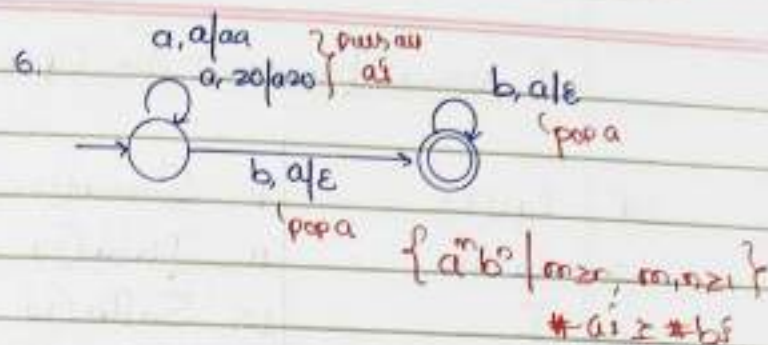
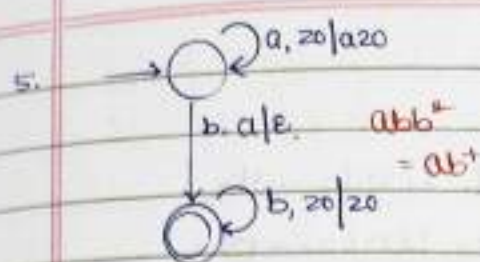
60. $\{a^n b^n\} - \text{DCF}$

61. $\{a^n b^n c^n\} - \text{CF but not DCF}$

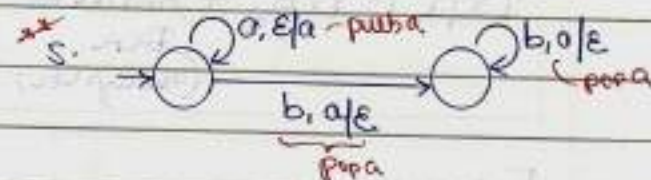
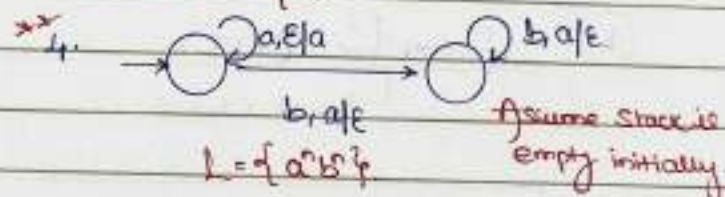
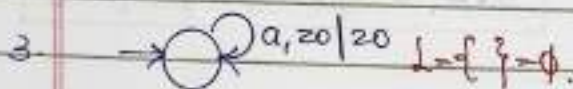
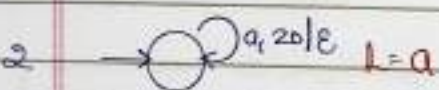
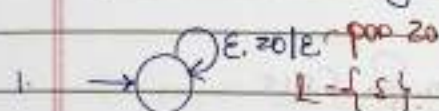
62. $\{ww \mid w \in \{a, b\}^*\} - \text{CF but not DCF}$

PDA uses final state





POA uses empty stack: (if string is valid, show that stack is empty at the end of i/p.)



Closure Properties for DCLC (Remember Closed Operations).

1. $L_1 \cup L_2$	12. Suffix(L)	23. Finite Subset(L)
2. $L_1 \cap L_2$	13. Substring(L)	24. Finite Substitutions(L)
3. $\bar{L}$	14. $f(L)$	25. Infinite $\{ \cup, \cap, -, \cdot, \subseteq, + \}$
4. $L_1 \cdot L_2$	15. $\partial(L)$	
5. $L_1 \cdot L_2$	16. ϵ -free(L)	
6. L^*	17. $\partial^{-1}(L)$	
7. L^+	18. L_1 / L_2	
8. L^+	19. $L_1 \cup L_2 \cup L_3 \cup \dots \cup L_k$	
9. $L_1 \Delta L_2$	20. $L_1 \cap L_2 \cap L_3 \dots \cap L_k$	
10. Subset(L)	21. $L_1 \cdot L_2 \cdot L_3 \dots L_k$	
11. Prefix(L)	22. $L_1 \cdot L_2 \cdot L_3 \dots L_k$	

Closure Properties for CFLs

1. $L_1 \cup L_2$	10. $\text{Subset}(L)$	19. $L_1 \cup L_2 \dots \cup L_k$
2. $L_1 \cap L_2$	11. $\text{Prefix}(L)$	20. $L_1 \cap L_2 \dots \cap L_k$
3. $\bar{L}$	12. $\text{Suffix}(L)$	21. $L_1 - L_2 - L_3 \dots - L_k$
4. $L_1 - L_2$	13. $\text{Substring}(L)$	22. $L_1 \cdot L_2 \cdot L_3 \dots L_k$
5. $L_1 \cdot L_2$	14. $f(L)$	23. $\text{Finite Subset}(L)$
6. L^{Rev}	15. $h(L)$	24. $\text{Finite Substitutions}(L)$
7. L^+	16. $\epsilon\text{-free } h(L)$	25. $\text{Infinite}(U, a, b, \dots, S, f)$
8. L^*	17. $h^{-1}(L)$	
9. $L_1 \Delta L_2$	18. L_1 / L_2	

① Union for DCFLE

→ not closed

$\text{DCFL}_1 \cup \text{DCFL}_2 \rightarrow \text{not not be DCFL}$
(Always CFL)

Example 1:

$\text{DCFL}_1 \cup \text{DCFL}_2 \rightarrow \text{not DCFL possible}$
 $\{a^m b^n c^k\} \cup \{a^m b^n c^k\} = \{a^m b^n c^k\}$
 $m=n \text{ or } n=k \}. \text{ (not DCFL)}$

Example 2:

$\text{DCFL}_1 \cup \text{DCFL}_2 \rightarrow \text{DCFL possible}$
 $a^* \cup a^+ \rightarrow a^+ \text{ DCFL}$

Union for CFLs

→ closed

$\text{CFL}_1 \cup \text{CFL}_2 \rightarrow \text{Always CFL}$

Algorithm:

$\text{CFL}_1 \Rightarrow \text{write CFG}_1 \text{ with } S_1 \text{ as start}$

$\text{CFL}_2 \Rightarrow \text{ " " " } S_2 \text{ " "}$



Add new start S

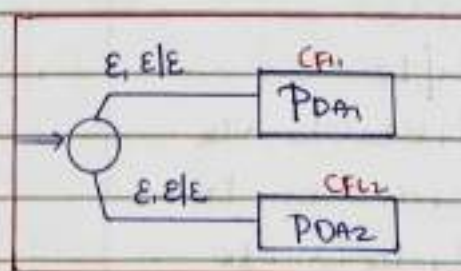
and append

$S \rightarrow S_1 / S_2$

$S \rightarrow S_1 / S_2$

CFG_1

CFG_2



$\text{CFL}_1 \cup \text{CFL}_2$

$$1. \left. \begin{array}{l} L_1 = \{a^n b^n\} \\ L_2 = \{b^n a^n\} \end{array} \right\} L_1 \cup L_2 = \{a^n b^n\} \cup \{b^n a^n\} = \text{DCFL}$$

$$2. \left. \begin{array}{l} L_1 = \{a^n b^n\} \\ L_2 = (ab)^+ \end{array} \right\} L_1 \cup L_2 = (ab)^+ \rightarrow \text{Regular}$$

$$3. \left. \begin{array}{l} L_1 = a^+ b^+ c^+ \\ L_2 = a^+ b^+ c^+ \end{array} \right\} L_1 \cup L_2 = a^+ b^+ c^+ \rightarrow \text{Regular}$$

(2) Intersection for DCFLs
→ not closed

Intersection for CFLs
→ not closed

$DCFL_1 \cap DCFL_2 \rightarrow$ May or may not be DCFL
(may or may not be CFL)

$CFL_1 \cap CFL_2 \rightarrow$ May or may not be CFL

$$\left. \begin{array}{l} L_1 = a^+ b^+ c^+ \\ L_2 = a^+ b^+ c^+ \end{array} \right\} = L_1 \cap L_2 = \{a^+ b^+ c^+\}$$

(not DCFL
not CFL (It is CFL))

$$1. L_1 = \phi, L_2 = \{a^n b^n\} \rightarrow L_1 \cap L_2 = \phi$$

$$2. L_1 = \Sigma^+, L_2 = \{a^n b^n\} \rightarrow L_1 \cap L_2 = L_2 = \{a^n b^n\}$$

$$3. L_1 = \{a^n b^n c^n\}, L_2 = \{a^n b^n c^n\} \rightarrow L_1 \cap L_2 = \{a^n b^n c^n\}$$

(3) Complement for DCFLs
→ Closed

Complement for CFLs
→ not closed

$\overline{DCFL} =$ Always DCFL
L is DCFL

$\overline{CFL} =$ either CFL or not CFL

Construct DPDA

$L = \{a^n b^n c^n\}$ is CFL

→ f.e. n.f.

Modified DPDA

$\overline{L} = \{a^n b^n c^n\}$ is not CFL
(it is CSL)

$\overline{L}$ is DCFL

$$\overline{\{a^m b^n\}} = (a+b)^+ - \{a^m b^n\}$$

$$= \Sigma^+ b a \Sigma^+ \cup \{a^m b^n \mid m \neq n\}$$

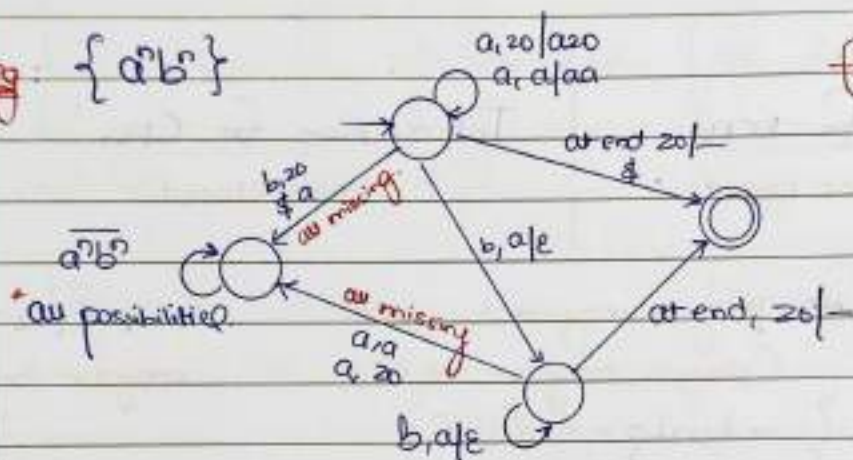
if the string
is not in $a^m b^n$

If the string is
in $a^m b^n$

$$L = a^m b^n = \{a^m b^n \mid m=n \text{ or } m \neq n\}$$

$$\rightarrow I = \Sigma^+ b a \Sigma^+$$

eg. $\{a^n b^n\}$



final $\leftrightarrow$ non final
Interchange to get
Complement

$$\begin{aligned} L = \overline{\{a^n b^n\}} &= (a+b)^+ - \{a^n b^n\} \\ &= (a+b)^+ - \{a^m b^n c^k \mid m=n=k\} \\ &= \text{CFL but not DFL} \end{aligned}$$

		DCFL	CFL
1. $\{a^n b^n\} \rightarrow$ DFL	U	x	✓
2. $\{a^n b^n c^n\} \rightarrow$ CFL but not DFL	n	x	x
3. $\{a^n b^m\} \rightarrow$ Regular	L	✓	x
4. $\{a^m b^n c^k \mid m \leq n \leq k\} \rightarrow$ CFL but not DFL			

Note: I. Complement of a CFL is either CFL or not CFL

II. Complement of not CFL is may or may not be CFL

III. Complement for CFLs is not closed.

④ Difference

- Not closed for DFLs
- Not closed for CFLs

$$L_1 \cdot L_2 = L_1 \cap \bar{L}_2$$

I. $DCFL_1 = DCFL_2 \rightarrow DCFL_1 \cap \overline{DCFL_2} \rightarrow DCFL_1 \cap DCFL_3 \rightarrow$ need not be DCF

II. $CFL_1 = CFL_2 \rightarrow CFL_1 \cap CFL_2 \rightarrow$ need not be CFL

L_1 is DCF

L_2 is CFL

$$\boxed{L \cup \bar{L} = \Sigma^+}$$

1. $L_1 \cup L_2$ is always CFL

2. $L_1 \cap L_2$ is need not be CFL

3. $\bar{L}_1 \cup \bar{L}_2$ is need not be CFL

4. $\bar{L}_1 \cup L_1$ is Σ^+ is regular

5. $\bar{L}_2 \cup L_2$ is Σ^+ is regular

(5) Concatenation:

- Not closed for DCFs
- Closed for CFLs

$DCFL_1 \cdot DCFL_2 \rightarrow$ need not be DCF (Always CFL)

$CFL_1 \cdot CFL_2 \rightarrow$ Always CFL

$$L_1 = \{a^n b^n\}$$

$$L_2 = \{a^n b^{2n}\}$$

$$L_1 \cdot L_2 = \{a^n b^n a^{2n} b^{2n}\} \rightarrow \text{not DCF}$$

aaa b
push

L_1 is CFL = $CFG_1(S_1)$

L_2 is CFL = $CFG_2(S_2)$

$\boxed{\text{Add } S \rightarrow S_1 S_2} \rightarrow S \rightarrow S_1 S_2 \quad CFG_1, CFG_2$

I. $\{a^n b^n a^k b^k\} \rightarrow$ DCF

II. $\{a^n b^n a^k b^{2k}\} \rightarrow$ CFL but not DCF

III. $\{a^n b^n a^k b^{2k} \mid n \geq 1\} \rightarrow$ DCF
always comes

(6) Reversal

- Not closed for DCFs
- Closed for CFLs

I. $(DCFL)^{rev} \rightarrow$ need not be DCF

$$L = \{a^n b^m a^k b^{2k} \mid n \geq 1, k \geq 0\} \text{ is DCFE.}$$

$$L^{rev} = \{b^{2k} a^k b^m a^n \mid n \geq 1, k \geq 0\} \text{ is not DCFE.}$$

L is CFI

$$L = a^n b^n$$

$$S \rightarrow a S b \mid \epsilon$$

$$S \rightarrow b S a \mid \epsilon$$

$$L = b^n a^n \text{ CFI}$$

7. Kleene Star: $\rightarrow$ Not Closed for DCFE
 $\rightarrow$ Closed for CFI

8. Kleene plus: $\rightarrow$ Not closed for DCFE
 $\rightarrow$ Closed for CFI

$$L \text{ is DCFE} \rightarrow L^+ \text{ need not be DCFE}$$

$$L = \{a\}$$

$$S \rightarrow a$$

add new start x and add

$$x \rightarrow x S \mid \epsilon$$

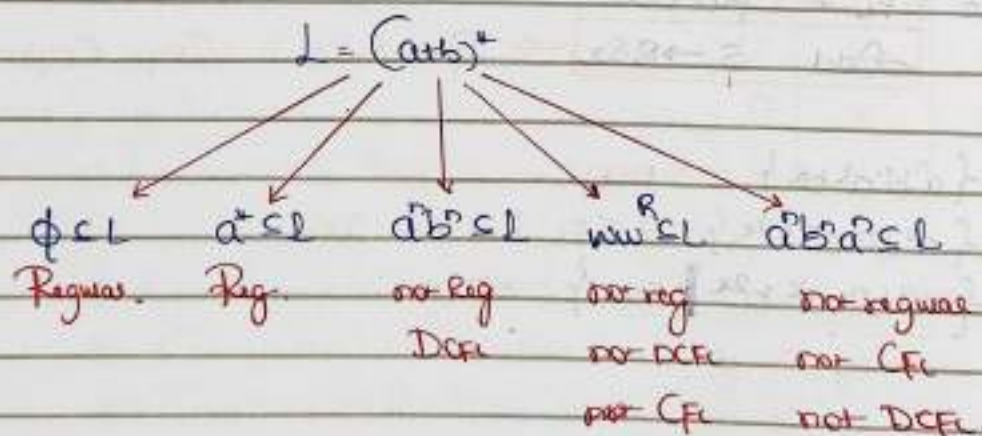
$$x \rightarrow x S \mid \epsilon$$

$$S \rightarrow a$$

$$L = a^+ \text{ is CFI}$$

9. $L_1 \Delta L_2 = (L_1 - L_2) \cup (L_2 - L_1)$
 $= (L_1 \cup L_2) - (L_1 \cap L_2)$
 $\rightarrow$ Not closed for DCFE & CFI

10. Subset of DCFE is need not be DCFE.
 Subset of CFI need not be CFI.



Push Down Automata (continued)

Regular Closures for DCFs:

1. DCF $\cup$ Regular
 2. DCF $\cap$ Regular
 3. DCF - Regular
 4. Regular - DCF
- } Always DCF
(need not be regular)

Note: * DCF $\cap$ Regular $\rightarrow$ Always DCF

* Set of all DCFs $\cap$ Set of all regulars $\rightarrow$ Set of all regulars

Regular Closures for CFLs:

1. CFL $\cup$ Regular
 2. CFL $\cap$ Regular
 3. CFL - Regular
 4. Regular - CFL $\rightarrow$ need not be CFL
- } Always CFL

I. DCF	$\cup$ $\cap$ -	Regular = DCF	II. CFL	$\cup$ $\cap$ -	Regular $\Rightarrow$ CFL
	/			/	

1. $L = \{a^m b^n\} \Rightarrow$ prefix(L) = $\{a^m b^n \mid m \geq n\}$
suff(L) = $\{a^m b^n \mid m \leq n\}$
Substring(L) = $\{a^+ b^+\}$

2. $L = \{a^m b^n\}$, $f(a) = \{0\}$, $f(b) = \{1\} \Rightarrow f(L) = 0^* 1^* = \{0^m 1^n\}$

3. $L = \{w w^R \mid w \in \{a,b\}^+\} \Rightarrow$ prefix(L) = $(a+bb)^+$
suff(L) = $(a+bb)^+$
Substring(L) = $(a+bb)^+$

Prefix (DCF) is DCF

DCF $\rightarrow$ DPDA $\rightarrow$ mod DPDA $\rightarrow$ pref(DCF)

- from initial to final, & con. come from any state.

Note: Language L satisfies Prefix property if every w in L is not Prefix of any other string in L .

Q1. $L = a^+$, does this satisfy Prefix property?

→ No

Q2. $L = \{a^n b^n \mid n \geq 1\}$

→ Satisfies Prefix property

* If any DCFL satisfies prefix property then we can make DPDA with empty stack.

$DPDA \equiv DPDA \text{ with final state}$

$\neq DPDA \text{ with empty stack}$

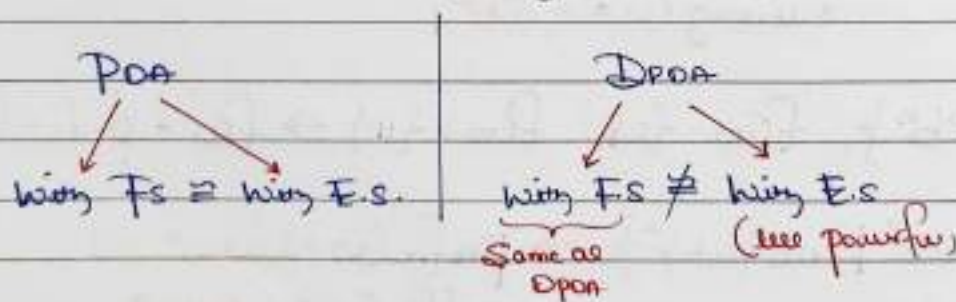
$DPDA \text{ with empty stack} < DPDA$

→ Set of all DCFLs

$\equiv$ Set of all languages accepted by DPDA

$\equiv$ Set of all languages accepted by DPDA with final state

Set of all languages generated by LR(1) CFGs



$CFG \equiv CFL \equiv POA \equiv POA \text{ with FS} \equiv POA \text{ with E.S.}$

$DCFL \equiv DPDA \equiv DPDA \text{ with FS} \equiv LR(1) \text{ CFG}$
 (Kri)

Inherently Ambiguous CFL:

Ambiguous CFG

$\exists w, \geq 1$ PT

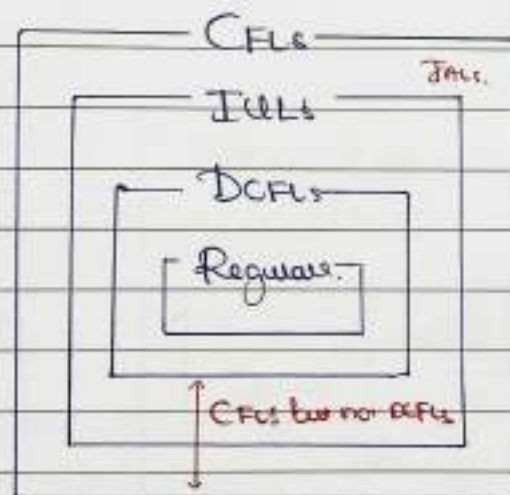
1. a^* - Regular but not IAL
2. a^*b^* - DCFE but not IAL
3. $\{ww^R \mid w \in \{a,b\}^*\}$ - CFL, not DCFE, and not IAL
4. $\{a^m b^k c^k \mid m=n \text{ or } n=k\}$ - CFL but not DCFE, it is IAL
 - there is no Unambiguous CFG in the world.

IAL

L is IAL if every CFG that represents generate L is Ambiguous if L has no Unambiguous CFG

IAL

L is IAL if some Unambiguous CFG exist for L .



- * Every Regular is IAL
- * Every DCFE is IAL
- * CFL is either IAL or IAL.

$\{ww^R \mid w \in \{a,b\}^*\}$

- CFL but not DCFE
- Unambiguous CFL

We can show Unambiguous CFG

$S \rightarrow asa \mid bsb \mid \epsilon$

Unambiguous CFG

$\{a^m b^k c^k \mid m=n \text{ or } n=k\}$

- CFL but not DCFE
- Ambiguous CFL

* There is no Unambiguous CFG for this language

If L is Unambiguous CFL $\rightarrow$ Show at least one Unambiguous CFG

If L is Ambiguous CFL $\rightarrow$ we don't have Unambiguous CFG
 Every CFG is Ambiguous

1. $L = a^*b^* \rightarrow \text{Reg} \}$ IUL
2. $L = a^*b^* \rightarrow \text{DCFL} \}$
3. $L = \{a^m b^k c^k \mid m=n \text{ or } m=k\} \rightarrow \text{CFL but not DCFL, IAL}$
4. $L = \{a^*b^*c^k\} \rightarrow \text{DCFL} \}$ IUL
5. $L = \{w \# w^R \mid w \in \{a,b\}^*\} \rightarrow \text{DCFL}$
6. $L = \{ww^R \mid w \in (a+b)^*\} \rightarrow \text{CFL, but not DCFL}$

TURNING MACHINE RECURSIVELY ENUMERABLE

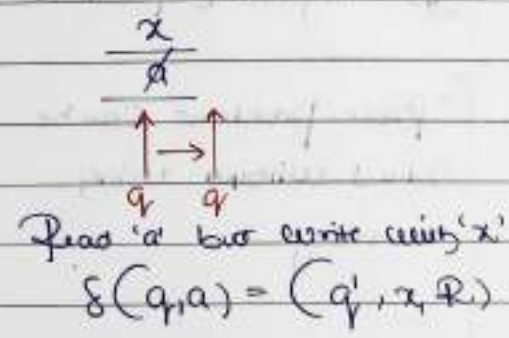
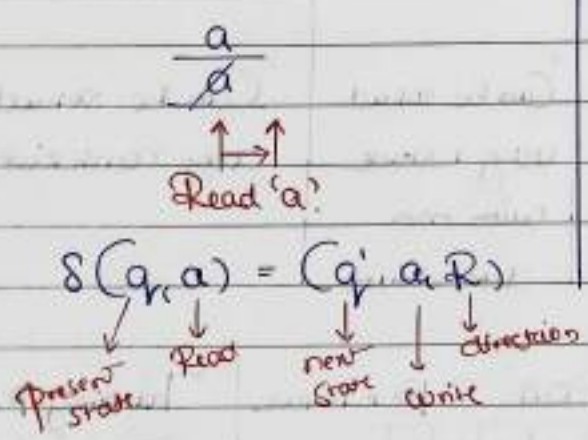
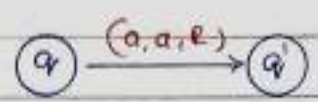
Regular	CFL	Recursively Enumerable Languages (REs)
(I) FA ↗ DFA ↘ NFA	(I) PDA	(I) TM
(II) Regular Grammar ↗ LLG ↘ RLG	(II) CFG	(II) Unrestricted Grammars
(III) Regular Expressions		
* These problems can be solved without using memory.	* Can be solved using 1 stack with non-determinism	* Can be solved without any restrictions

FA	PDA $\cong$ FA + 1 Stack	Turning Machine
	Unbounded Stack.	Read/write head
$FA = (Q, \Sigma, \delta, q_0, F)$ $\delta_{DFA}: Q \times \Sigma \rightarrow Q$ $\delta_{NFA}: Q \times \Sigma \rightarrow 2^Q$	$PDA = (Q, \Sigma, \delta, q_0, F, Z_0, \Gamma)$ $\delta_{PDA}: Q \times \Sigma \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$ $\delta_{DPDA}: Q \times \Sigma \times \Gamma \rightarrow Q \times \Gamma^*$	$TM = (Q, \Sigma, \delta, q_0, F, B, \Gamma)$ $\delta_{TM}: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ $\delta_{NTM}: Q \times \Gamma \rightarrow 2^{Q \times \Gamma \times \{L, R\}}$ $B \rightarrow$ Blank Symbol.

* PDA that uses finite stack $\cong$ FA $\cong$ Regular Language

* FA + R/w tape + Bidirectional + Infinite tape $\cong$ Turning m/c

Fn	POA	Tm
To remember one Symbol 'a':	1. Change state or	1. Change state
① $\xrightarrow{a}$ ②	2. push/pop into stack or	2. Write with new Symbol on tape
1. Change state.	3. Change state and push or pop into stack	3. Change state & write or



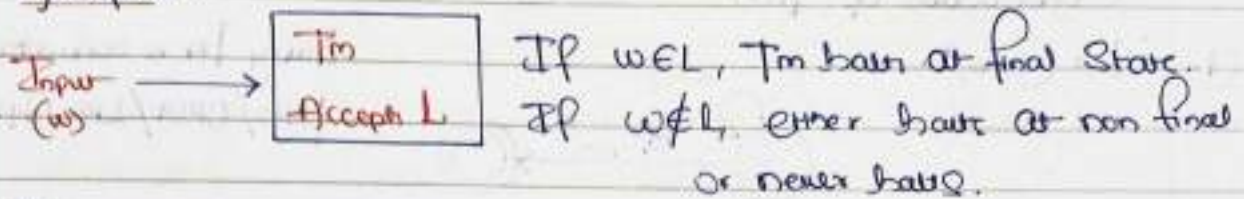
Find meaning of transitions

1. $\delta(q, B) = (s, a, \leftarrow \text{left}) \rightarrow$ From state q , by reading B , goes to s . By writing with a and moves left.
2. $\delta(q, a) = (q', a, \rightarrow \text{right}) \rightarrow$ Read 'a' move right.
3. $\delta(q, a) = (q', B, \leftarrow \text{left}) \rightarrow$ i/p = a , Replacing a with B left direction.

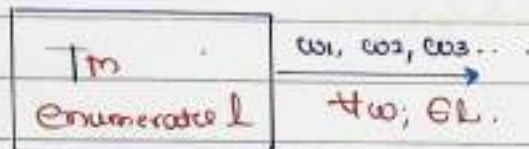
Turing Machine

- * It represents REL (Recursively Enumerable language)
- * It can accept REL
OR
- * It can enumerate REL

Acceptor:

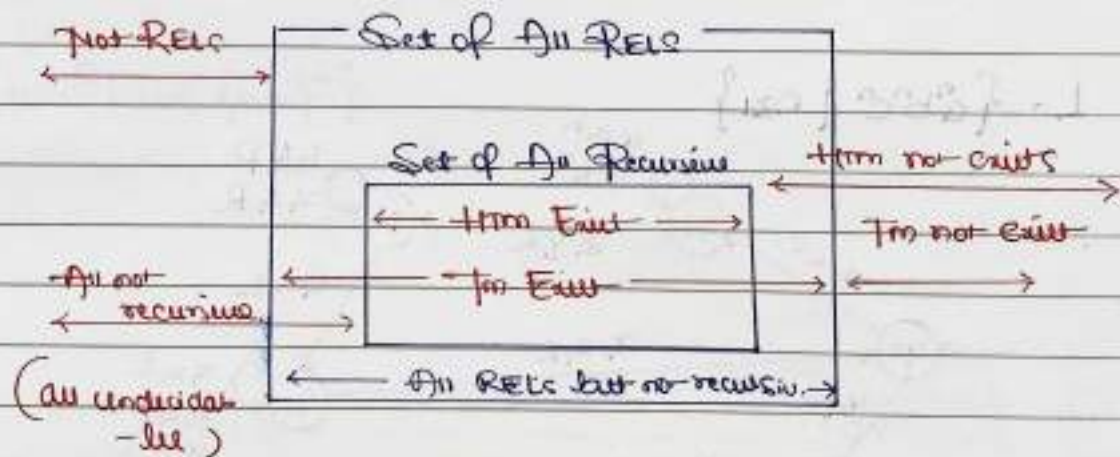


Enumerator



→ Tm ⇒ logic exist for valid strings
- Accept, REL (logic may or maynot exist for invalid string)

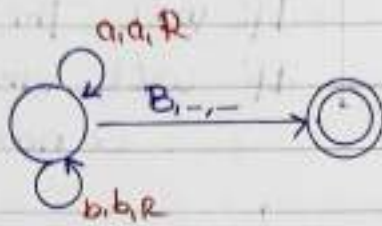
- Having Tm ← $\neg$ Tm → logic exist for valid and invalid
- Accept Recursive language (Decidable) ↓ Halt at final ↓ Halt at non final



	Tm	$\neg$ Tm
1. Recursive Language (decidable language)	✓	✓
2. REL (Semi decidable language)	✓	don't know
3. REL but not recursive	✓	x
4. Not REL	x	x
5. Undecidable language	don't know	x

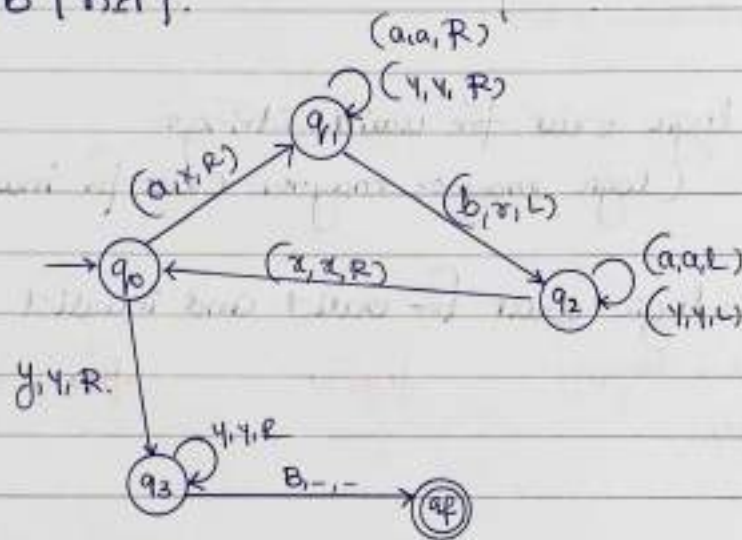
Construction of Tm:

①. $L = (arb)^+$

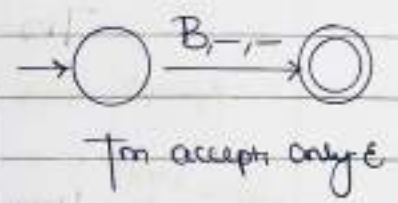


* Every FA is convertible to tm/
PDA/DPDA/LBA/HM

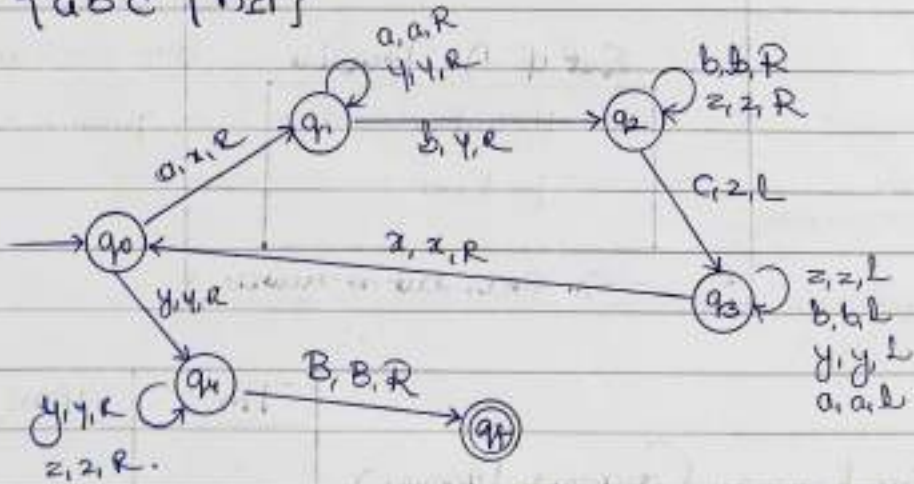
②. $\{a^n b^n \mid n \geq 1\}$



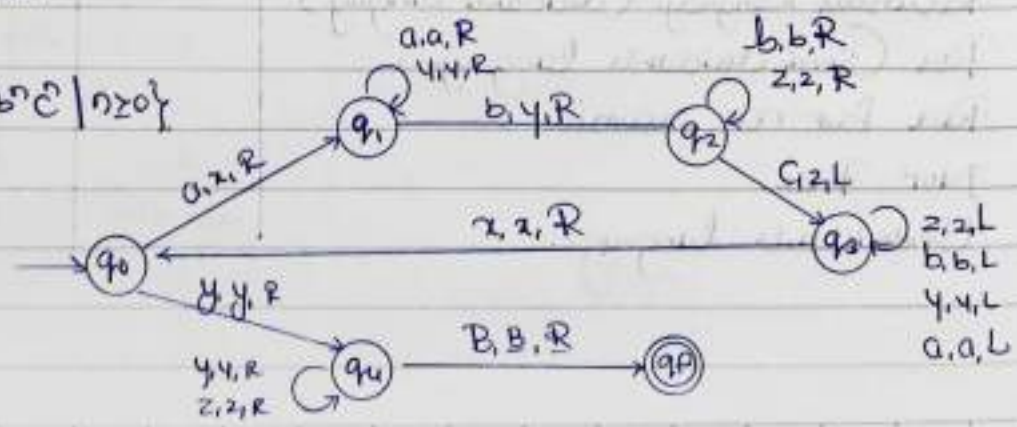
③. $L = \{\epsilon\}$

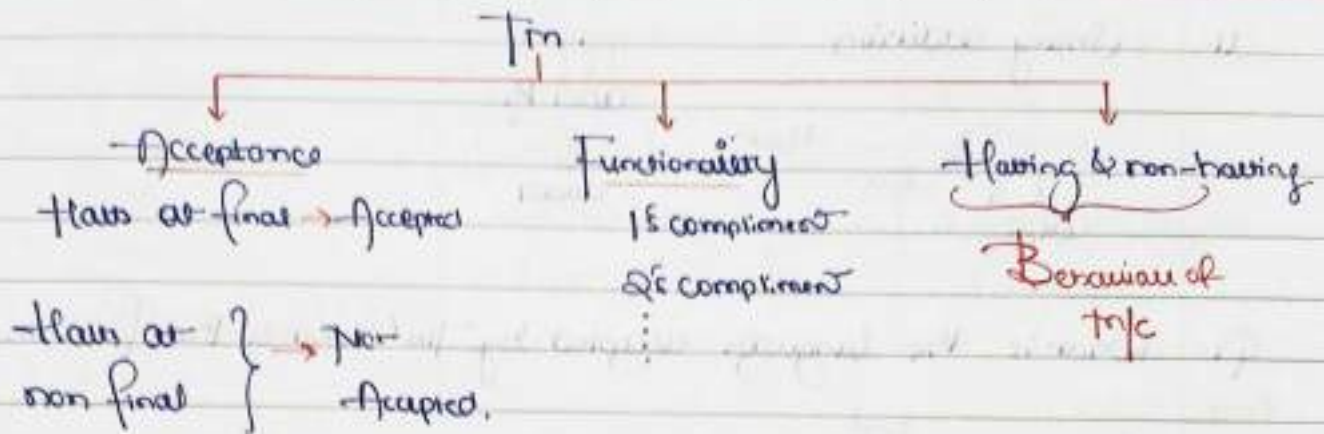


④. $L = \{a^n b^n c^n \mid n \geq 1\}$

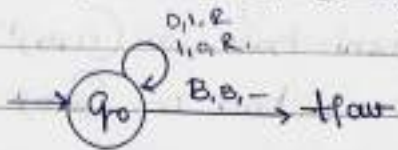


⑤. $L = \{a^n b^n c^n \mid n \geq 0\}$



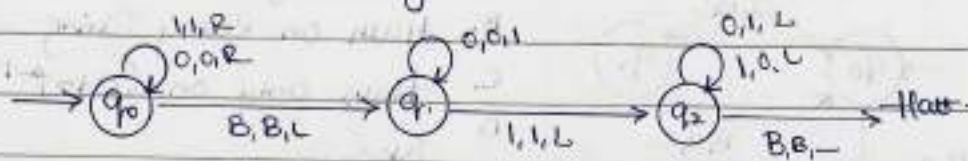


6. 1's Complement of Binary.

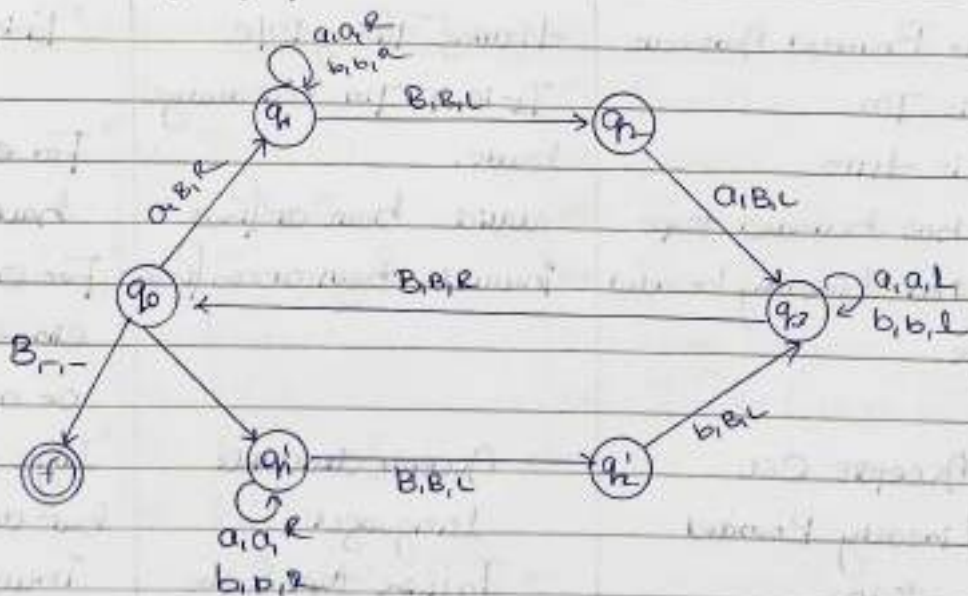


* We do not need final state. There is no concept of valid and invalid input.

7. 2's Complement of Binary.



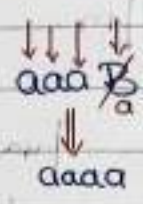
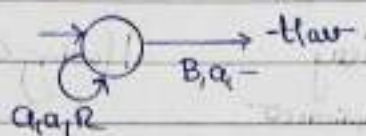
8. $\{ww^R \mid w \in \{a,b\}^+\}$



9. $\{w#w^R \mid w \in \{a,b\}^+\}$

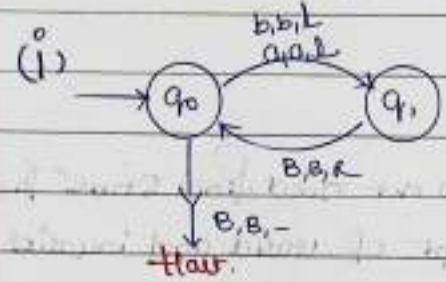
10. $\{w#w \mid w \in \{a,b\}^+\}$

11. Unary addition

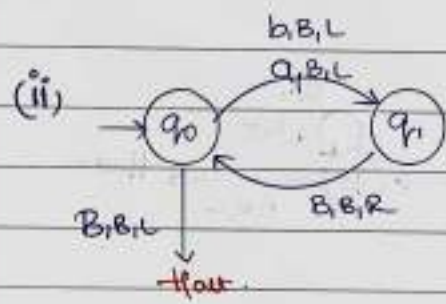


Q1. What is the language accepted by TM? What is the functionality?

Ans



- A. Halts only on E
- B. Halts only on $(abb)^+$
- C. Doesn't halt on $(abb)^+$
- D. Doesn't halt on $(abb)^*$



- A. Halts only on E
- B. Halts on every string
- C. Halts only on $(abb)^{++}$
- D. None

Lower Bounded Automata	-Halting Turing M/c	Turing M/c
* It is TM	* It is a TM, but always halts.	* For every valid string halts or final
* It is -ltm		
* It has bounded tape	→ valid halts or final	* For every invalid string, either halt or non final or never halt
* It has linearly bounded tape	→ invalid halts or non final	* Logic exist for valid but don't know about invalid.
* → Accepts CSL	→ Accepts decidable languages	* → Accepts REC
* → Linearly Bounded tape	→ Infinite tape (unbo - undel)	Infinite tape (unbo - del)
* → Always halts	→ Always halts	

CSL

- * Every regular
- * Every CFL
- * $a^m b^n c^m$
- * a^{Prime}
- * $\{ww | w \in \{a,b\}^*\}$
- * $a^{n!}$

Recursive

- * Every CSL
- * $\{ \langle n_1, n_2 \rangle \mid n_1, n_2 \text{ are regular expressions} \}$
- * $L(n_1) = L(n_2)$

REL

- * Every CSL
- * Every Recursive
- * Set of all regular languages.

Closure Properties for Recursive Languages

- | | | |
|---------------------|-----------------------------|--|
| 1. $L_1 \cup L_2$ | X Subset (L) | 17. Finite $\cup$ |
| 2. $L_1 \cap L_2$ | 10. Prefix (L) | 18. Finite $\cap$ |
| 3. $\bar{L}$ | 11. Suffix (L) | 19. Finite Diff. |
| 4. $L_1 - L_2$ | 12. Substring (L) | 20. Finite Concatenation |
| 5. $L_1 \cdot L_2$ | X $f(L)$ | 21. Finite Subset |
| 6. L^{Rev} | X $h(L)$ | 22. $f_{\text{finite}}(L)$ |
| 7. L^+ | 15. E-free $h(L)$ | X Infinite ($\cup, \cap, -, \cdot, \subseteq, f$) |
| 8. L^+ | 16. $h^{-1}(L)$ | |

Closure Properties for RE Language

- | | | |
|--------------------------|-----------------------------|--|
| 1. $L_1 \cup L_2$ | X Subset (L) | 17. Finite $\cup$ |
| 2. $L_1 \cap L_2$ | 10. Prefix (L) | 18. Finite $\cap$ |
| X $\bar{L}$ | 11. Suffix (L) | X Finite diff. |
| X $L_1 - L_2$ | 12. Substring (L) | 20. Finite Concatenation |
| 5. $L_1 \cdot L_2$ | 13. $f(L)$ | 21. Finite Subset |
| 6. L^{Rev} | 14. $h(L)$ | X Infinite ($\cup, \cap, -, \cdot, \subseteq, f$) |
| 7. L^+ | 15. E-free $h(L)$ | |
| 8. L^+ | 16. $h^{-1}(L)$ | |

$\rightarrow$ Infinite ($\cup, \cap, -, \cdot, \subseteq, f$)

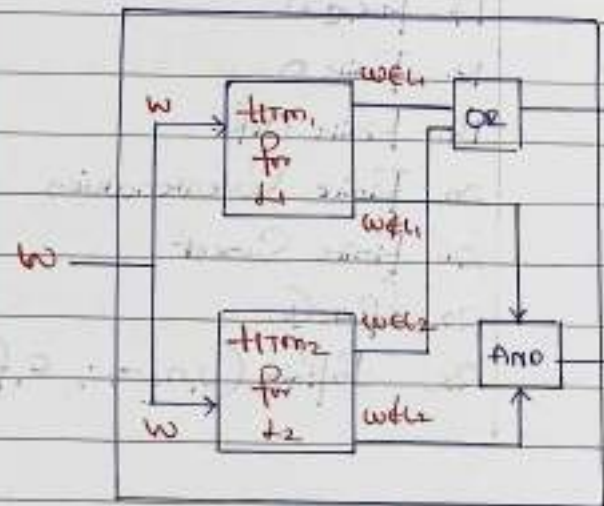
Note: * Subset are not closed for Regular languages, DFL, CFL, CSL, Recursive, REL.

* h^{-1} and finite subset are closed for regular, DFL, CFL, Recursive, REL.

1. $\text{Regular} \rightarrow \subseteq, \text{Infinite } (u, n, -, \cdot, \subseteq, f)$
2. $\text{DCFL} \rightarrow \text{I, prefix, } b^i, \text{ finite subset}$
3. $\text{CFLs} \rightarrow n, \text{I, } -, \subseteq, /, \text{FEO, Im}$
4. $\text{Recursive} \rightarrow \subseteq, f, b, \text{Inf } (u, n, -, \cdot, \subseteq, f)$
5. $\text{RELC} \rightarrow \text{I, } -, \subseteq, \text{Finite diff, Inf. (u)}$

① Union

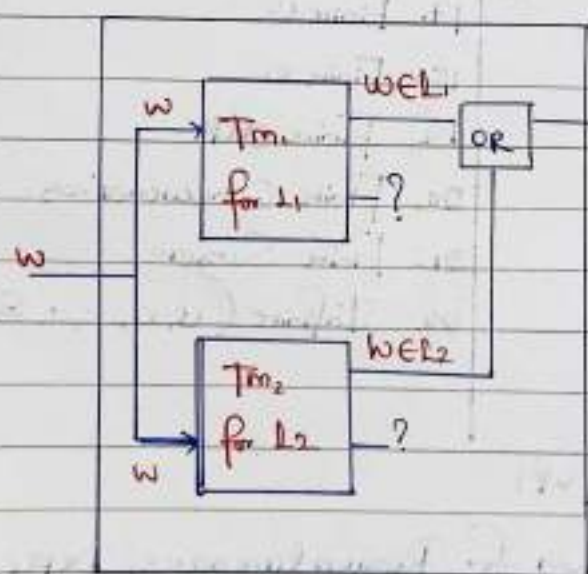
Closed for recursive languages (decidable): $\text{Rec}_1 \cup \text{Rec}_2 = \text{Rec}$
 Closed for RE language (semi-decidable): $\text{RE}_1 \cup \text{RE}_2 = \text{RE}$



$w \in (L_1 \cup L_2)$
 If any TM have at final then new TM have at final.

$w \notin (L_1 \cup L_2)$
 If both TM have at non-final then new TM rejects at non-final.

TM for $L_1 \cup L_2$



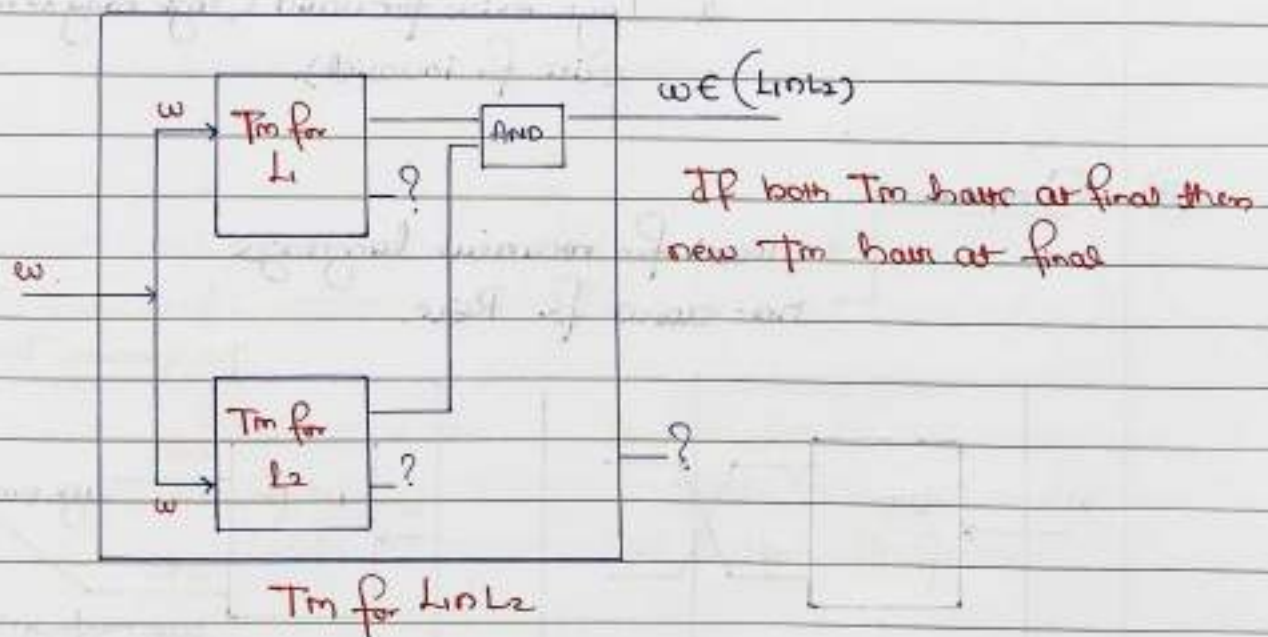
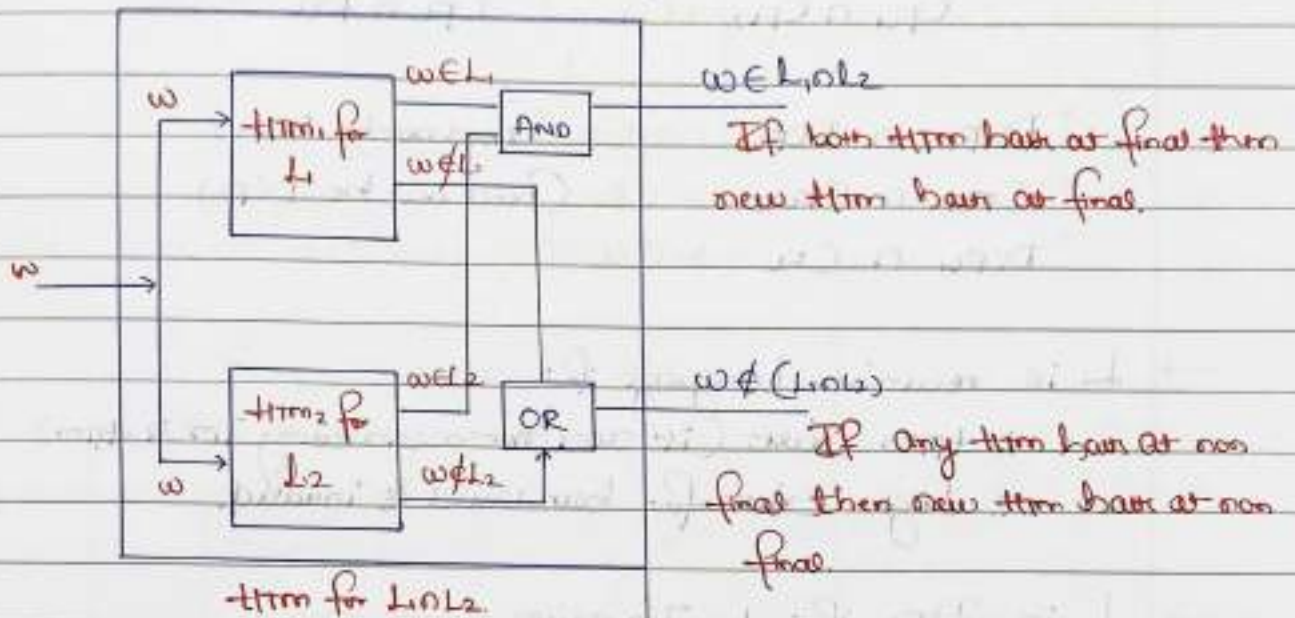
$w \in (L_1 \cup L_2)$
 If any TM have at final then new TM have at final.

?

TM for $L_1 \cup L_2$

② Intersection:

→ Closed for Recursive languages: $Rec_1 \cap Rec_2 = Rec$
 → Closed for RE languages: $RE_1 \cap RE_2 = RE$



Q1. A_1, A_2 are Recursive
 B_1, B_2 are REs.

1. $A_1 \cup A_2$ is Recursive
2. $B_1 \cup B_2$ is RE
3. $A_1 \cup B_1$ is RE
4. $A_1 \cap A_2$ is Recursive
5. $B_1 \cap B_2$ is RE
6. $A_1 \cap B_1$ is RE

Note: Reg $\cap$ DCFL $\rightarrow$ DCFL
 Reg $\cap$ CFL $\rightarrow$ CFL
 DCFL $\cap$ CFL $\rightarrow$ CSL
 CFL $\cap$ CFL₂ $\rightarrow$ CSL
 CFL $\cap$ CSL $\rightarrow$ CSL
 CSL $\cap$ Recursive $\rightarrow$ Recursive
 Rec $\cap$ REL $\rightarrow$ REL
 CFL $\cap$ REL $\rightarrow$ REL

- * DCFL₁ $\cap$ DCFL₂ $\rightarrow$ CSL (need not be DCFL)
- * CFL₁ $\cap$ CFL₂ $\rightarrow$ CSL (need not be DCFL)
- * DCFL₁ $\cap$ CFL $\rightarrow$ CSL

$\rightarrow$ L is recursive language if:

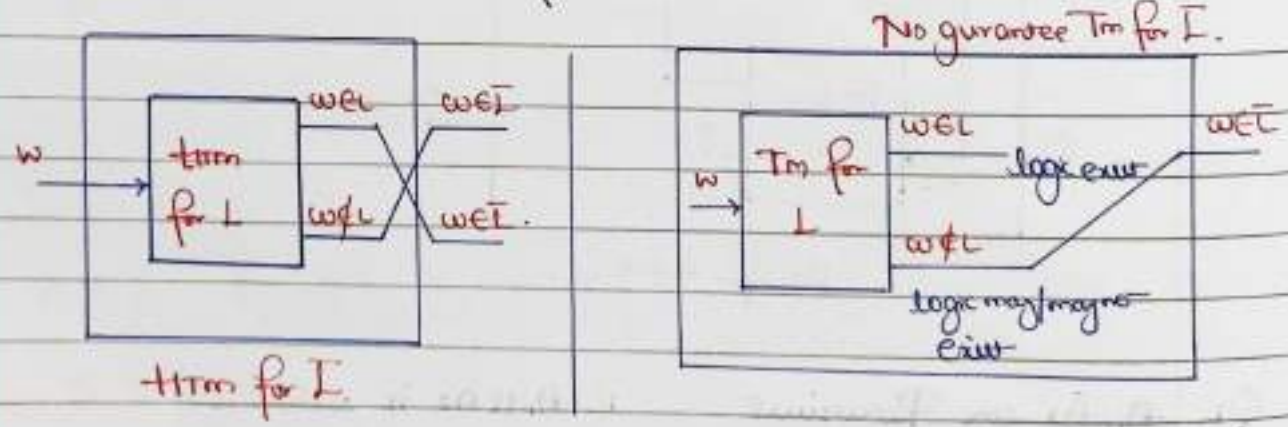
1. TM exist (it does not mean every TM is TM)
2. Logic exist for both valid & invalid.

$\rightarrow$ L is REL if:

1. TM exist.
2. Logic exist for valid (logic may or may not exist for invalid).

③ Compliment

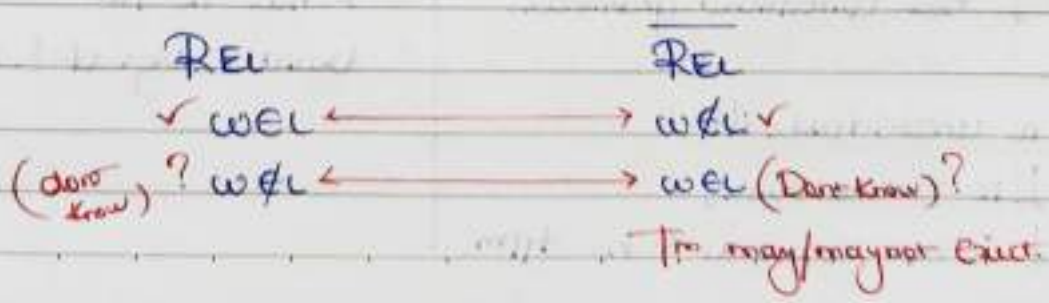
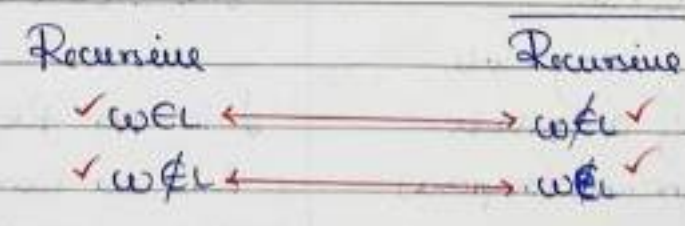
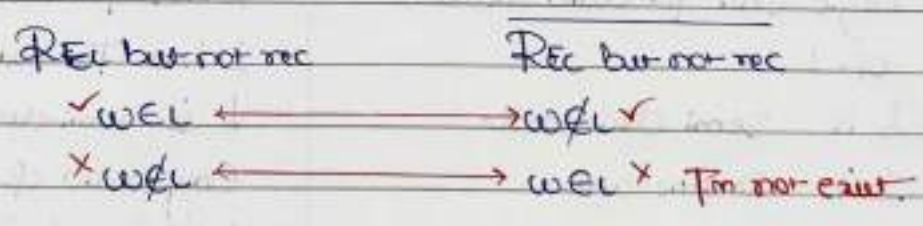
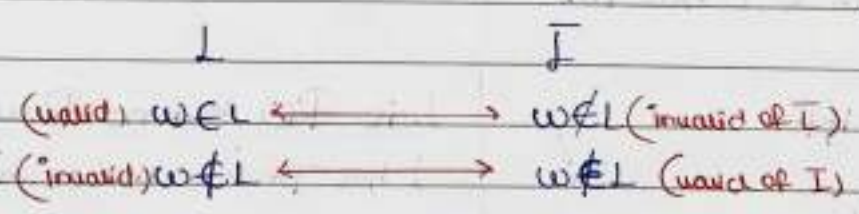
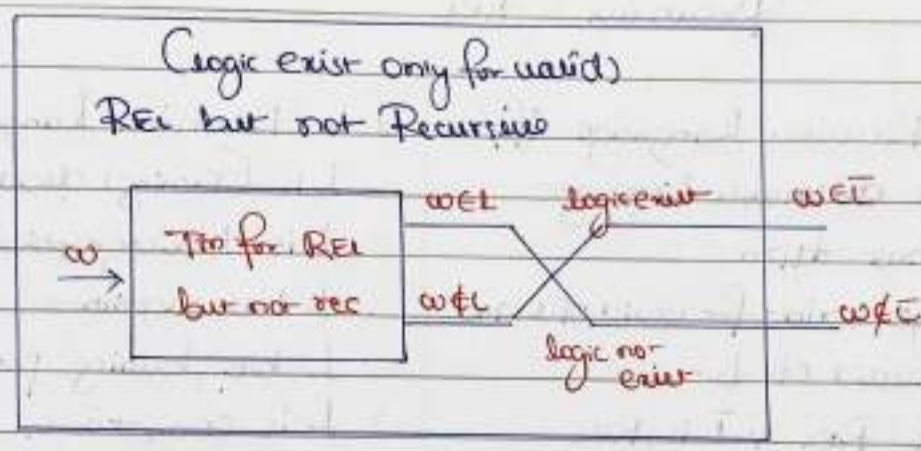
$\rightarrow$ closed for recursive languages
 $\rightarrow$ not closed for REL.



- I. Recursive Language $\rightarrow$ Recursive Language
- II. REL $\rightarrow$ Never be "REL but not rec".
- III. REL $\rightarrow$ Need not be REL
- IV. REL $\rightarrow$ Either recursive lang or "Not-REL"



III. REL but not recursive $\rightarrow$ NOT REL



Note: I. Recursive $\begin{matrix} U \\ n \\ / \\ - \end{matrix}$ Regular $\rightarrow$ Recursive

II. REL $\begin{matrix} U \\ n \\ / \\ - \end{matrix}$ Regular $\rightarrow$ REL

- * Recursive is REL but REL need not be Recursive.
- * Decidable is REL but REL need not be Decidable.
- * Recursive $\neq$ Decidable
- * Recursive is REL

$\rightarrow$ L is Recursive language iff:

1. L is decidable language
2. L has TM
3. Logic exist for valid of L and invalid of L.
4. L is REL & $\bar{L}$ is REL
5. L has TM & $\bar{L}$ has TM

L is Recursive language if:

1. L is (Turing) decidable
2. L is Lexicographically Enumerable
3. L has algorithm
4. L has halting program
5. L is Computable.

$\rightarrow$ L is REL iff:

1. L has TM
2. Logic exist for valid members of L
3. L is Semi Decidable
4. L is Enumerable
5. L has Program
6. L is Turing recognisable
7. L is Turing acceptable
8. L is Recognizable (acceptable)
9. L has Unrestricted grammar.

L is REL but not Recursive if:

1. L has TM but not recursive
2. L has TM but L has no TM
3. L has TM but $\bar{L}$ has no TM
4. L is Semi-decidable & undecidable
5. Only valid of L has logic.
6. L is REL & $\bar{L}$ is not REL

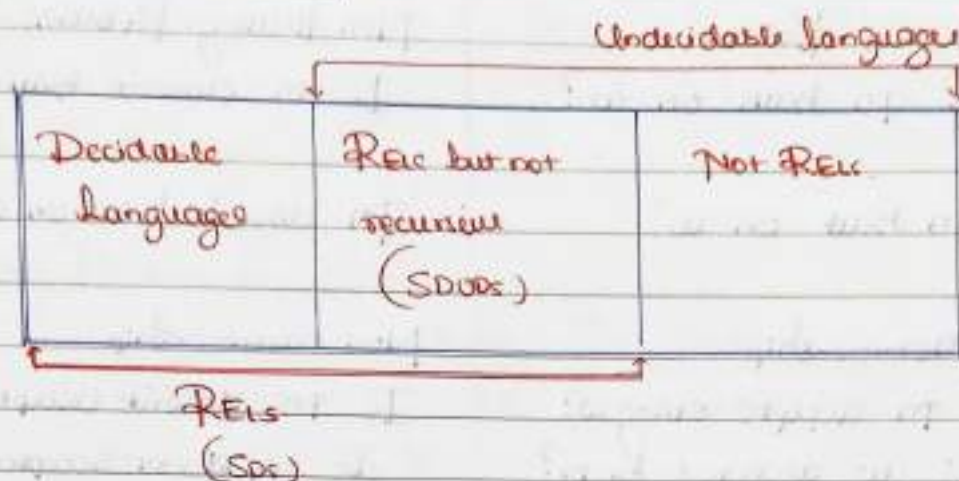
L is not REL iff:

1. L is not Semi Decidable
2. L has no TM
3. Valid strings of L has no logic

L is undecidable iff:

1. L is not decidable,
2. L is not recursive & no TM.

DECIDABILITY



	D	SDUO	NR
Members	valid $\rightarrow$ logic exist invalid $\rightarrow$ logic exist	valid $\rightarrow$ logic exist invalid $\rightarrow$ logic not exist	valid $\rightarrow$ logic not exist
Machine	TM exist	TM exist but TM not exist	TM not exist
Language	Decidable language	SDUO (Rec but not rec)	NOT REC (NOT SO)
Program (Computer)	Halting program exist (Algorithm exist)	Program exist but no algorithm	Program not exist

Decision Properties Table (Decision Problems)

Problem	FA, Regular lang, Reg Exp, RL	DPOA, DCFGs	PDA, CFLs, CFGs	TM, Recursive lang	TM, REs, UG
1. Halting	D	D	D	D	UD
2. Membership	D	D	D	D	UD
3. Emptiness	D	D	D	UD	UD
4. Finiteness	D	D	D	UD	UD
5. Totality	D	D	UD	UD	UD
6. Equivalence	D	D	UD	UD	UD
7. Disjoint	D	UD	UD	UD	UD
8. Set Containment	D	UD	UD	UD	UD

① Halting Problem
 If M halt on w ?
 Or
 " M halt on w ."

Non-Halting Problem.
 Is M doesn't halt on w ?
 Or
 " M doesn't halt on w ."

② Membership
 Is M accepts string w ?
 Or Is w accepted by M ?
 Or Is $w \in L(M)$?
 Or " M accepts w "

Non membership
 Is M doesn't accept w ?
 Or Is w not accepted by M ?
 Or Is $w \notin L(M)$?
 Or " M doesn't accept w "

③ Emptiness
 Is M accepts ϕ ?
 Or Is M accepts nothing?
 Or Is $L(M) = \phi$?

Non Emptiness
 Is M not-accepts ϕ ?
 Or Is M accepts something?
 Or Is $L(M) \neq \phi$?

④ Finiteness
 Is M accepts finite language?
 Is $L(M) = \text{finite language}$?

Non-Finiteness
 Is $L(M) \neq \text{finite language}$?
 Is $L(M) = \text{infinite language}$?

⑤ Totality
 Is M accepts everything?
 Is $L(M) = \Sigma^*$?

Non Totality
 Is M not accepts everything?
 Is M not accepting atleast one string?
 Is $L(M) \neq \Sigma^*$?

⑥ Equivalence
 Is $M_1 \approx M_2$?
 Is $L(M_1) = L(M_2)$?

Non Equivalence
 Is $M_1 \neq M_2$?
 Is $L(M_1) \neq L(M_2)$?

⑦ Disjointness
 Is $L(M_1) \cap L(M_2) = \phi$?

Non Disjointness
 Is $L(M_1) \cap L(M_2) \neq \phi$?

8.

Set Containment

Is $L(M_1) \subseteq L(M_2)$?

Is $L_1 \subseteq L_2$?

Non Set Containment

Is $L_1 \not\subseteq L_2$?

Halting Problem for TM.

Is TM halt on ϵ ?
 Yes: TM halt on ϵ (logic exist)
 No: TM doesn't halt on ϵ (logic not exist)
 Undecidable SPUD

Not Halting: Is TM doesn't halt on w ?
 Yes: logic not exist
 No:
 Undecidable NOT RE

Membership for FA/RegL/RegExp/PL

FA $\rightarrow$ Is $w \in L(FA)$?
 $w \rightarrow$
 Yes: $w \in L(FA)$: FA halt at final (logic exist)
 No: $w \notin L(FA)$: FA halt at non final (logic exist)

Similarly, membership problem is decidable for FA/DPDA/PDA/LBA/TM.

Membership for TM $\rightarrow$ SPUD

TM $\rightarrow$ Is TM
 accept w ?
 $w \rightarrow$
 Yes: $w \in L(TM)$ given TM halt at final (logic exist)
 No: $w \notin L(TM)$ given TM either halt at non final or never halt (logic not exist)

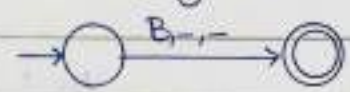
Non membership for TM
 Not RE

Problems:

1. Is FA accepts ϵ ? $\rightarrow$ Membership for FA $\rightarrow$ Decidable
2. Is PDA accepts ϵ ? $\rightarrow$ D
3. Is TM accepts ϵ ? $\rightarrow$ SPUD

4. Is $FA_1 \cong FA_2$? $\rightarrow$ SUD $\exists$ Some String $\exists$ Pr (logic exist)
5. Is CFG Ambiguous ? $\rightarrow$ Yes: CFG is Ambiguous
 $\rightarrow$ No: CFG is Unambiguous
 $\exists$ every string, 1 pr (logic not exist)
6. Is grammar CFG? $\rightarrow$ Decidable
7. Is CFG unambiguous $\rightarrow$ Not RE

(I) Can we design Tm that accepts ϵ ?



we can design Tm that accepts ϵ

(II) Can we verify arbitrary Tm that accepts ϵ ? $\rightarrow$ Yes: logic $\checkmark$
 $\rightarrow$ No: logic $\times$
undecidable & Semi decidable

Languages:

1. $\{ FA \mid FA \text{ accepts } \epsilon \} \rightarrow D$
2. $\{ \langle FA, w \rangle \mid FA \text{ accepts } w \} \rightarrow$ Decidable
3. $\{ FA \# w \mid FA \text{ accepts } w \} \rightarrow$ Decidable
4. $\{ Tm \mid Tm \text{ accepts } \epsilon \} \rightarrow$ SUD
5. $\{ Tm \mid Tm \text{ accepts } w \} \rightarrow$ SUD
6. $\{ Tm \mid Tm \text{ accepts some string} \} \rightarrow$ SUD
7. $\{ Tm \mid Tm \text{ accepts only } w \} \rightarrow$ Yes: Tm is accepting only w
 $\rightarrow$ No: $\exists (Tm) \neq \{w\}$ Tm not only accepts w.
 $\rightarrow$ SUD
8. $\{ Tm \mid Tm \text{ accepts 2 length string} \} \rightarrow$ SUD
9. $\{ Tm \mid Tm \text{ reached state } q \} \rightarrow$ SUD
10. $\{ Tm \mid Tm \text{ reached state } q \text{ within 5 steps} \} \rightarrow$ Decidability
11. $L = a^*b^*$
12. Finite language
13. Regular language $\rightarrow$ Decidable
14. DFL
15. CFL
16. CSL
17. Recursive language $\rightarrow D$
18. RE $\rightarrow D$ or SUD
19. RE but not Recursive $\rightarrow$ SUD
20. Not RE $\rightarrow$ NR
21. Not Recursive $\rightarrow$ NR or SUD
22. Not Regular Language $\rightarrow D$ or SUD or NR

23. Not CFL $\rightarrow$ D/SND/NR
24. $a^n b^n \rightarrow$ D
25. Set of (all) finite languages } SND
26. Set of (all) regular languages }
27. Set of DFL, CFL, CSL, Decidable language, RE, SUND } SND
28. Set of not RE
29. Set of infinite languages } NR
30. $\{LL \text{ is decidable}\} \cup \{L | L \text{ is NR}\}$

Reducibility: $A \leq B$

A is reducible to B

- * Every instance of A is mapped into some instance of B
- * B is equal or harder than A
- * B is atleast as hard as A
 - $\rightarrow A \leq_p B$ (A is polynomially reducible to B)
 - $\rightarrow A \leq_m B$ (A is many to one reducible to B)

Let $A \leq B$:

1. If A is undecidable then B is undecidable
2. If B is decidable then A is decidable
3. If B is undecidable then A is decidable/undecidable
4. If A is decidable then B is decidable/undecidable
5. If B is RE then A is RE
6. If A is not RE then B is not RE
7. If A takes 100 days then B takes min 100 days
8. If B takes 100 days then A takes max 100 days

If $A \leq B$ and $C \leq B$:

1. If A is decidable then B is ? and C is ?
2. If B is decidable then A is decidable and C is decidable
3. If C is decidable then A is ? and B is ?

$\rightarrow$ If $A \leq B$ and $B \leq A$ then $A = B$

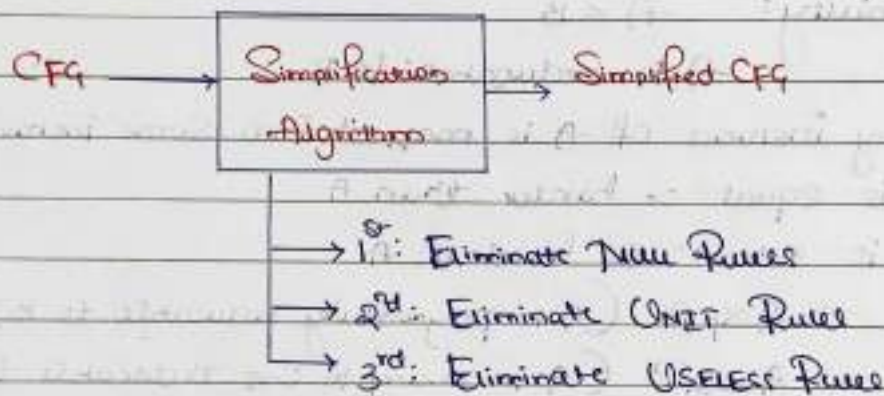
Let $A \leq B \leq C$ then:

1. If C is decidable then A is decidable and B is decidable
2. If A is decidable then B is undecidable and C is undecidable

→ L is SDUD → $\bar{L}$ is NR

→ L is Set of all SDUD → $\bar{L}$ = Set of all decidable $\cup$ Set of all NR

Simplification of CFG



eg: $S \rightarrow AB | \epsilon$
 $A \rightarrow aA | bB | c$
 $B \rightarrow Ba | \epsilon | bS$

Step 1: $S \rightarrow \epsilon$

$S \rightarrow AB$

$A \rightarrow aA | bB | c$

$B \rightarrow Ba | \epsilon | bS | b$

Step 2: $B \rightarrow \epsilon$

$S \rightarrow AB | A$

$A \rightarrow aA | bB | c | b$

$B \rightarrow Ba | bS | b | a$

eg 2: $S \rightarrow Sa | Ab | Bc | AB$
 $A \rightarrow Ae | \epsilon$
 $B \rightarrow Bb | \epsilon$

Step 1: $A \rightarrow \epsilon$ delete

$S \rightarrow Sa | Ab | Bc | AB | b | B$

$A \rightarrow Ae | \epsilon$

$B \rightarrow Bb | \epsilon$

Step 2: Delete $B \rightarrow \epsilon$

$S \rightarrow Sa | Ab | Bc | AB | b | B$

$c | A | \epsilon$

$A \rightarrow Ae | \epsilon$

$B \rightarrow Bb | b$

Step 3: Delete $S \rightarrow \epsilon$

$S \rightarrow Sa | Ab | Bc | AB | b | B | c | A | a$

$A \rightarrow Ae | \epsilon$

$B \rightarrow Bb | b$

$X \rightarrow \epsilon$ delete

In whole grammar, add all new productions by putting X with ϵ

egs: $S \rightarrow aA$ | $A \rightarrow bB$ | AS | B | $B \rightarrow aA$ | S

Step 1: Delete $S \rightarrow A$	Step 2: Delete $S \rightarrow B$
$S \rightarrow aA$ bB AS B	$S \rightarrow aA$ bB AS $\cancel{aA}$ $\cancel{S}$
$A \rightarrow bB$ AS B	$A \rightarrow bB$ AS B
$B \rightarrow aA$ S	$B \rightarrow aA$ S

Step 3: Delete $A \rightarrow R$	Step 4: Delete $A \rightarrow S$	Step 5:
$S \rightarrow aA$ bB AS	$S \rightarrow aA$ bB aA	$S \rightarrow aA$ bB aA
$A \rightarrow bB$ AS aA S	$A \rightarrow bB$ AS aA	$A \rightarrow bB$ aA aA
$B \rightarrow aA$ S	$B \rightarrow aA$ S	$B \rightarrow aA$ bB AS

* In given G, if we delete $X \rightarrow Y \rightarrow$ Replace Y with all Y productions.

Eliminate USELESS Productions:

Step 1: Find the nonterminals which derive nothing and eliminate it.

Step 2: Find the unreachable nonterminals and delete them.
 $S \rightarrow \cancel{aA}$ | $b \Rightarrow S \rightarrow b$

$S \rightarrow aA$ | Bb | CDe
 $A \rightarrow a$ | sa
 $B \rightarrow bB$
 $D \rightarrow bD$ | eE
 $F \rightarrow g$
 $H \rightarrow ea$

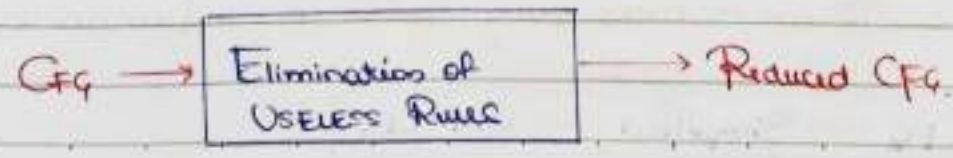
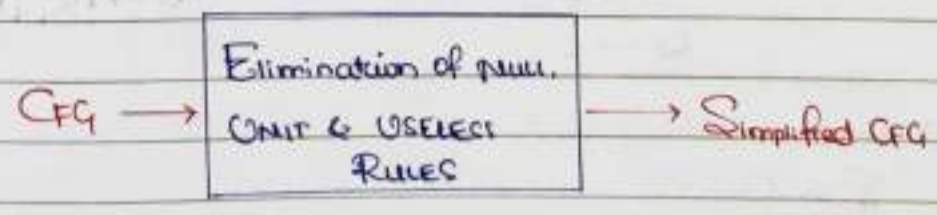
Step 1: Identify non-terminals derived nothing:

S	A	B	D	E	F	H
aa	a	x	x	x	g	ea

Delete all productions associated with B, D, E.

Step 2: Delete all unreachable symbols

$S \rightarrow aH$
 $A \rightarrow a$ | sa



Normal Forme:

- CNF CFG (Chomsky normal form)
- GNF CFG (Greibach normal form)

CNF CFG	GNF CFG
$V \rightarrow UV \mid t$ only 1 terminal exactly 2 non terminal. Example: $S \rightarrow SS \mid AS \mid a$ $A \rightarrow b \mid SA$	$V \rightarrow Tv^*$ Example: $S \rightarrow a \mid aSSSA$ $A \rightarrow bA \mid CAAA SA$

	CNF	GNF
1. $S \rightarrow a$	✓	✓
2. $S \rightarrow ab$	✗	✗
3. $S \rightarrow SS \mid a$	✓	✗
4. $S \rightarrow as \mid bsc \mid c$	✗	✓

Conversion of CFG to CNF CFG:

- $S \rightarrow ab \Rightarrow S \rightarrow G_1 G_2$
 $G_1 \rightarrow a$
 $G_2 \rightarrow b$
- $S \rightarrow abc \Rightarrow S \rightarrow G_1 G_2 G_3$
 $S \rightarrow G_1 X$
 $X \rightarrow G_2 G_3$
 $G_1 \rightarrow a$
 $G_2 \rightarrow b$
 $G_3 \rightarrow c$
- $S \rightarrow as \mid bca \mid bB$
 $A \rightarrow a s c \mid e$
 $B \rightarrow A a \mid b$

$X \rightarrow \dots$

if in CNF ✓

if no → make every symbol as non-terminal then divide into 2 length

CFG → Simplified → CNF CFG

$X \rightarrow \square$
 if terminal $\rightarrow$ erase non-terminal
 if non-terminal, substitute with production

Conversion of CFG to GNF CFG?

CFG $\rightarrow$ Simplified CFG $\rightarrow$ non left Recursive CFG $\rightarrow$ GNF CFG

- ① $S \rightarrow ab \rightarrow S \rightarrow aG$
 $G \rightarrow b$

② $S \rightarrow abc \rightarrow S \rightarrow aGG$
 $G \rightarrow b$
 $G \rightarrow c$
- ③ $S \rightarrow Aab \mid baB \Rightarrow S \rightarrow fG_1 \mid bG_2B$
 $A \rightarrow f$ $A \rightarrow f$ ✓ *unique*
 $B \rightarrow gh$ $B \rightarrow gG_3$
 $G_1 \rightarrow a$
 $G_2 \rightarrow b$
 $G_3 \rightarrow c$

$\rightarrow$ How long does it take the derivation takes place for deriving n length string using GNF CFG?

- ① $n=1$ $w=a$

$\textcircled{S}$
 $\downarrow$
 a = 1 Step

② $n=2$ $w=aa$

$\textcircled{S}$
 $\downarrow$
 $\textcircled{AB}$ = 3 Steps
 $\downarrow$
 $a \textcircled{g}$
 $\downarrow$
 ab
- ③ $n=3$ $w=abb$
- ```

graph TD
 S --> x
 S --> y
 x --> a
 y --> w
 y --> z
 w --> b
 z --> b

```

$\textcircled{S}$   
 $\downarrow$   
 $\textcircled{xy}$   
 $\downarrow$   
 $a \textcircled{y}$   
 $\downarrow$   
 $a \textcircled{wz}$   
 $\downarrow$   
 $ab \textcircled{z}$   
 $\downarrow$   
 $abb$

= 5 Steps

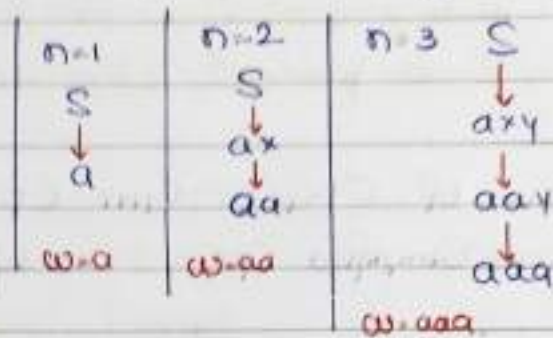
= 5 non leaf nodes

= 5 Substitutions

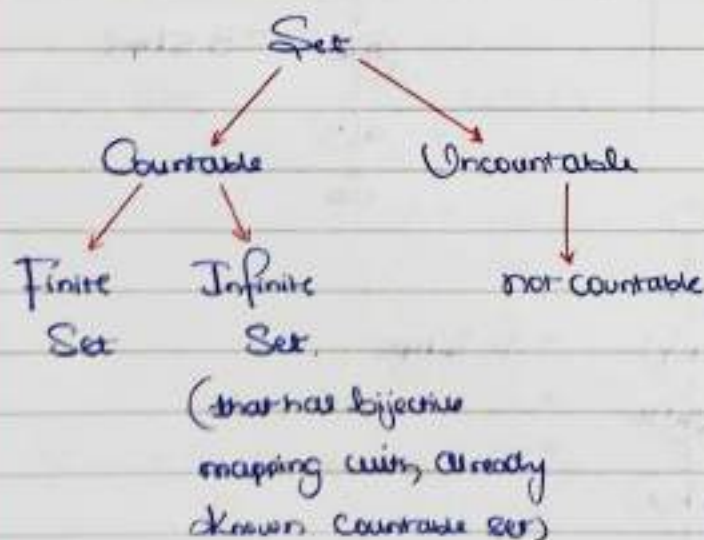
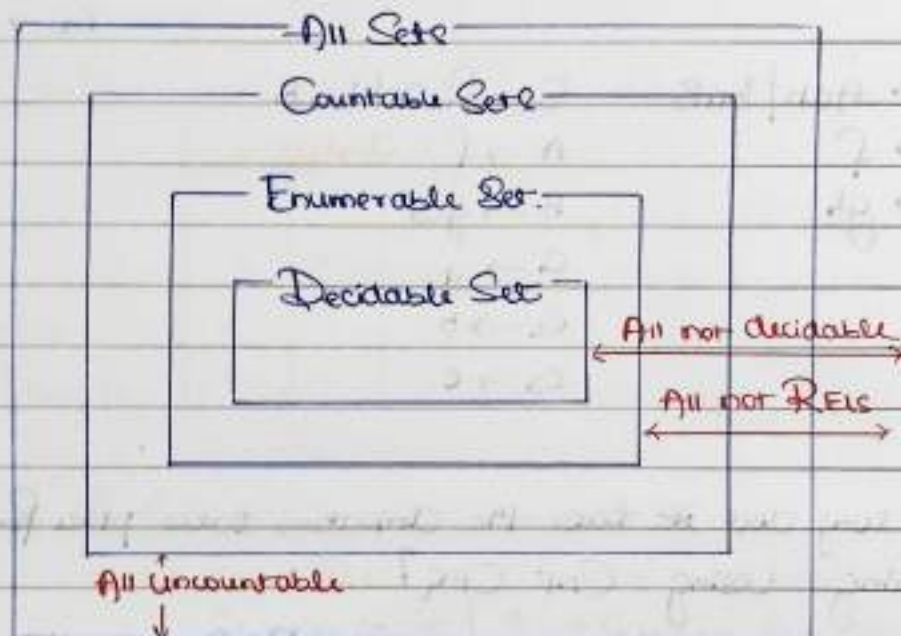
\* n length string  $\rightarrow (2n-1)$  steps using GNF CFG

How long the derivation takes place for deriving  $n$  length string using GNF CFG?

1.  $n=1 \rightarrow \# \text{ steps } = 1$
2.  $n=2 \rightarrow \# \text{ steps } = 2$
3.  $n=3 \rightarrow \# \text{ steps } = 3$



For  $n$  length string  
' $n$ ' steps using GNF CFG



$N$  = Natural Set

$Z$  = Integer Set

$2N$  = Even Set

$2N+1$  = Odd Set

$\Sigma$  branches into:

- Finite
- REL
- Decidable
- Countable

$\Sigma$  = Set of finite no. of symbols

$\Sigma^*$  branches into:

- Infinite set
- Regular set
- Decidable
- Countable
- REL

$\Sigma^*$  = Set of all strings

$2\Sigma^*$  branches into:

- Infinite Set
- not countable

$2\Sigma^*$  = Set of all languages

|    |                | Regular set. | DCFLs | CFLs | Recurr. Lang. | REs                        |
|----|----------------|--------------|-------|------|---------------|----------------------------|
| 1. | $L_1 \cup L_2$ | Regular      | CFL   | CFL  | Recursive     | RE                         |
| 2. | $L_1 \cap L_2$ | Regular      | CSL   | CSL  | Recursive set | RE                         |
| 3. | $\bar{L}$      | Regular      | DCFL  | CSL  | Recursive     | either Recursive or not RE |
| 4. | $L_1 - L_2$    | Regular      | CSL   | CSL  | Recursive     | Any.                       |

Rice Theorem : Every non trivial property is undecidable.

non trivial property  $\leftarrow L = \{M \mid M \text{ is TM, } M \text{ accepts Regular}\} \rightarrow \text{not RE}$   
 $\bar{L} = \{M \mid M \text{ is TM, } M \text{ accepts not regular}\} \rightarrow \text{not RE}$

Trivial  $\leftarrow L = \{TM \mid TM \text{ accepts RE}\}$

= Set of all TMs

$\bar{L} = \{TM \mid TM \text{ accepts not RE}\}$   
 =  $\phi$